AI505

Optimization

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

List of Lectures

1. Introduction

Who is here?

44 in total registered in ItsLearning

- **AI505 (7.5 ECTS)**
  31 from Bachelor in AI

- **AI801 (10 ECTS)** or **AI505 (7.5 ECTS)** + **IAAI501 (2.5 ECTS)**
  13 from Master Educations (which ones?)

Prerequisites

- Calculus
- Linear Algebra
- Programming

Outline

1. Course Organization

2. Introduction

Aims of the course

Learn about **optimization**:

- continuous multivariate optimization
- discrete optimization

Optimization is an important tool in **machine learning**, **decision making** and in analysing physical systems.
In mathematical terms, an **optimization problem** is the problem of finding the **best solution** from the set of all **feasible solutions**.

The first step in the optimization process is constructing an appropriate **mathematical formulation**. The second is devising an **algorithm** for solving the mathematical formulation.

Contents of the Course (Pensum)

Unit	Main topic
1	Introduction, Univariate Problems
2	Multivariate Problems, Gradient-Based Methods
3	Derivative-Free Methods
4	Optimization for Machine Learning
5	Constrained Optimization, Linear Programming
6	Sampling Methods
7	Discrete Optimization and Heuristics

Practical Information

Teacher: Marco Chiarandini (imada.sdu.dk/u/marco/)
Instructor: Sai Ganesh Nagarajan (H21)

Schedule, alternative views:

- mitsdu.sdu.dk, SDU Mobile
- Official course description (skemaplanen)
- ItsLearning
- https://ai-505.github.io

Schedule (16 weeks):

- Introductory classes: 16 classes (32 hours)
- Training classes: 16 classes (32 hours)
- Scheduled: $(16 \times 2) + 2$ classes $\times 2 = 68$ hours

Communication Means

- ItsLearning $\Leftrightarrow$ External Web Page
  (link `https://ai-505.github.io`)
- Announcements + Discussion Board + Submissions in ItsLearning
- Write to Marco (`marco@imada.sdu.dk`) or to instructor
- Collaborate with peers

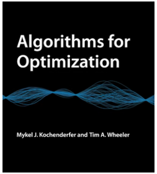
$\leadsto$ **It is good to ask questions!!**

$\leadsto$ **Let me know if you think we should do things differently!**

Sources — Reading Material

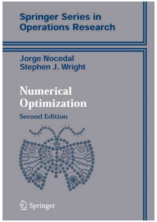
Main reference:

[KW] Mykel J. Kochenderfer, Tim A. Wheeler. Algorithms for Optimization 2019. The MIT Press.



Others

[NW] J. Nocedal and S. J. Wright, Numerical Optimization, Second Edition. Springer Series in Operations Research, 2006



Course Material

External Web Page is the main reference for list of contents (ie[1], syllabus, pensum).

It will collect:

- slides
- list of topics and references
- exercises
- links
- tutorials for programming tasks

[1] ie = id est = that is; eg = exempli gratia = for example; wrt = with respect to; et al. = et alii = and others

Assessment

Portfolio consisting of:

- mandatory assignments in groups of 2:
  1. assignment
  2. assignment

  Time line to be announced.

- oral exam on June 29-30, 2026

The oral examination consists of questions on the basis of the handed in assignments and can be extended to other parts of the course curriculum.

Both elements must individually meet the learning objectives.

Final grade: primarily based on element 1, but element 2 may adjust the grade up or down by one grade level.

(language: Danish and/or English)

Exercise Sessions (1/2)

For exercises in general:

- Both theoretical, modelling and programming tasks
- No mandatory hand-ins, voluntary participation, but all lectures and exercises are relevant for the exam
- Help each other! Teaching others is the best form of learning
- Sai will be there to help. First hour: you work and Sai gives personalize help. Second hour: Sai presents a subset of solutions in plenary.
- Exercises should help you to learn - find the setting that works best for you, don't hesitate to extend with other material that works better for you. You can use Gen AI tools.

Exercises are group work:

- Exercises best done in pairs of 2-3 people
- Try to gather in different groups every now and then

Exercise Sessions (1/2)

Notation: $^{+}$, *, ' '

- plus exercises are to be done before the class
- starred exercises are done in class. They are good examples of assignment questions
- unmarked exercises are for self study

Coding

Set up your local Python programming environment and use the available tutorial to:

- Brush up some knowledge on Python-IDEs and Git
- Grasp some basics on jupyter notebooks
- Recap basics of data processing and visualization

We will span several programming modes: functional, imperative, object oriented, small scripts, large code bases, use of modules and libraries. We will use git.

Outline

Introduction

- Applications of Optimization
  - Physics
  - Business
  - Biology
  - Engineering
  - Machine Learning
  - Logistics and Scheduling
- Objectives to Optimize
  - Efficiency
  - Safety
  - Accuracy
- Constraints
  - Cost
  - Weight
  - Structural Integrity
- Challenges
  - High-Dimensional Search Spaces
  - Multiple Competing Objectives
  - Model Uncertainty

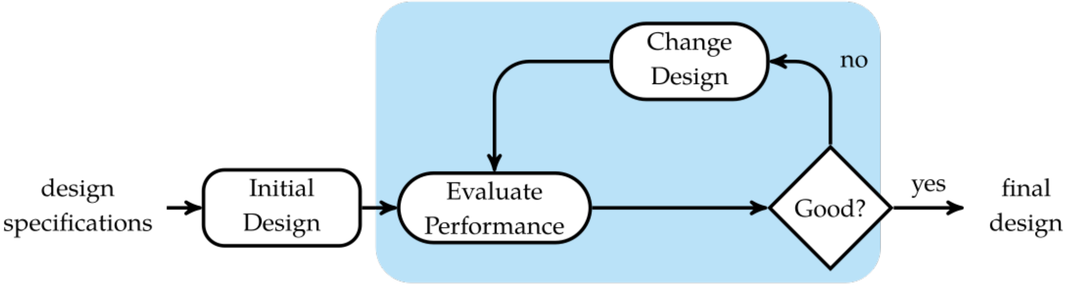
Real World, Model, Representation, Implementation

> The Real World: That messy thing we are trying to study (with computers).
>
> Model: Mathematical object in some class M.
>
> Representation: An object of an abstract data type R used to store the model.
>
> Implementation: An object of a concrete type used to store the model.

Any object from the real world might have different models.
Any model might have several representations (exact).
And representation might have different implementations (exact).

We will focus on the algorithmic aspects of optimization that arise after the problem has been properly formulated

Optimization Process



An optimization algorithm is used to incrementally improve the design until it can no longer be improved or until the budgeted time or cost has been reached.

Optimization Process

Search the space of possible designs with the aim of finding the best one.

Depending on the application, this search may involve:

- evaluating an analytical expression (white or glass box)
- running physical experiments, such as wind tunnel tests (black box)
- running computer simulations

Modern optimization techniques can be applied to problems with millions of variables and constraints.

Definitions – Functions

- Univariate single-valued function: $f : \mathbb{R} \to \mathbb{R}$ is a function that takes as input a single real number, say x, and produces a single real number as output.
- Multivariate single-valued function: $f : \mathbb{R}^n \to \mathbb{R}$ is a function that takes as input a vector $\boldsymbol{x}$ of n real numbers and produces a single real number as output.
- (Multivariate) Vector-valued function: $f : \mathbb{R}^n \to \mathbb{R}^m$ is a function that takes as input a vector $\boldsymbol{x}$ of n real numbers and produces a vector $\boldsymbol{y}$ of m real numbers as output.

We will work primarily with multivariate single-valued functions.

Basic Optimization Problem

$$\boxed{\begin{aligned}&\underset{\boldsymbol{x}}{\text{minimize}}\; f(\boldsymbol{x})\\ &\text{subject to } \boldsymbol{x} \in \mathcal{X}\end{aligned}}$$

- Feasible Set $\mathcal{X} \subseteq \mathbb{R}^n$
- Design Point $\boldsymbol{x}$
- Design Variables
- Objective Function: $f : \mathbb{R}^n \to \mathbb{R}$ (scalar-valued function)
- Minimizer

Any value of $\boldsymbol{x}$ from among all points in the feasible set $\mathcal{X}$ that minimizes the objective function is called a **solution** or **minimizer**. A particular solution is written $\boldsymbol{x}^*$.

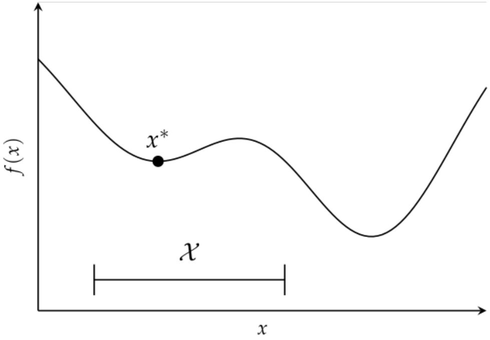
$$\boxed{\boldsymbol{x}^* = \operatorname{argmin} f(\boldsymbol{x}) \text{ subject to } \boldsymbol{x} \in \mathcal{X}}$$

There is only one minimum but there can be many minimizers

maximize $f(\boldsymbol{x})$ subject to $\boldsymbol{x} \in \mathcal{X} \equiv$ minimize $-f(\boldsymbol{x})$ subject to $\boldsymbol{x} \in \mathcal{X}$

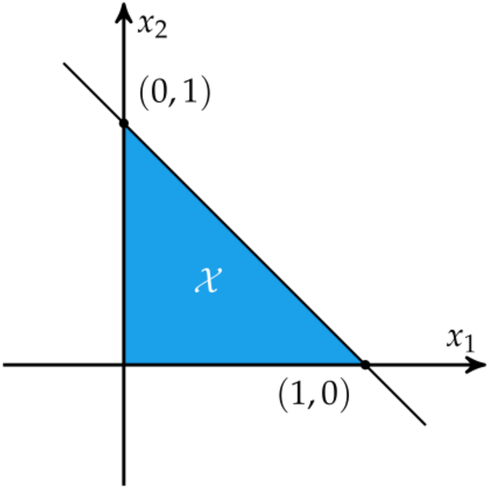
Basic Optimization Problem

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{x} \in \mathcal{X} \end{aligned}$$



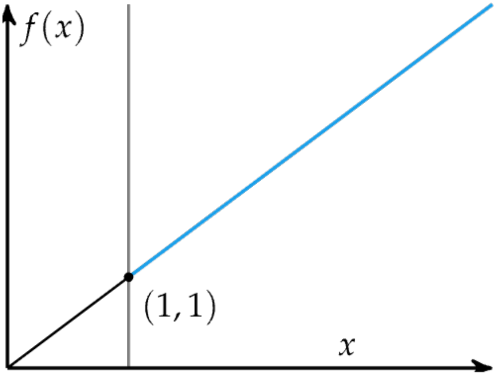
Constraints

$$\begin{aligned} \underset{x_1, x_2}{\text{minimize}} \quad & f(x_1, x_2) \\ \text{subject to} \quad & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_1 + x_2 \leq 1 \end{aligned}$$



Constraints

$$\underset{x}{\text{minimize}} \quad f(x)$$
$$\text{subject to} \quad x > 1$$



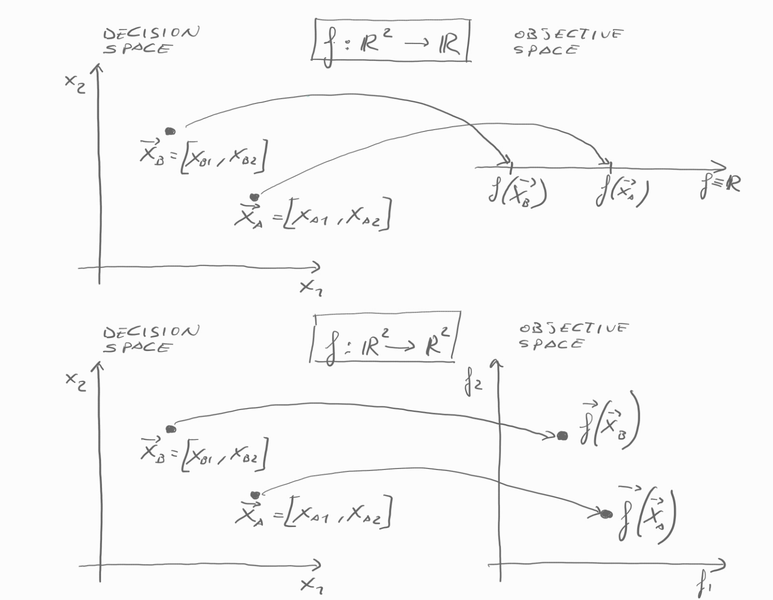
The problem has no solution.
$x = 1$ would not be feasible.

$x = 1$ would be the solutions to

$$\underset{x}{\text{infimum}}\, f(x) \text{ subject to } x > 1$$

infimum of a subset $\mathcal{X}$ of a partially ordered set $\mathcal{P}$ is the greatest element in P that is less than or equal to each element of $\mathcal{X}$, if such an element exists. Aka, **greatest lower bound**.

Decision Space and Objective Space

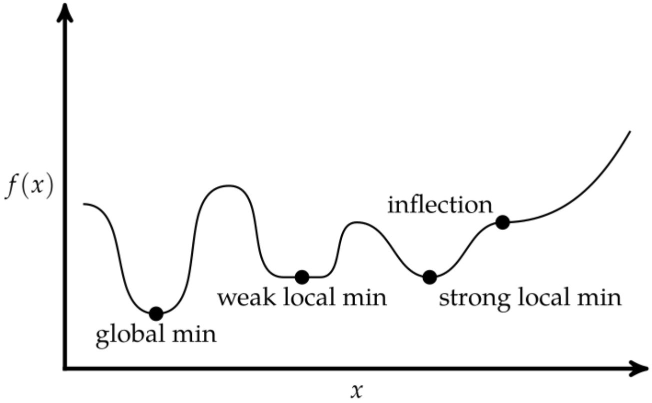


Critical Points

Univariate function

global vs local minimum

<u>Def.</u> A point $\boldsymbol{x}^*$ is at a **local minimum** (or is a local minimizer) if there exists a $\delta > 0$ such that $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x}$ with $\|\boldsymbol{x} - \boldsymbol{x}^*\| < \delta$.

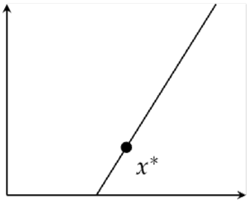


Conditions for Local Minima

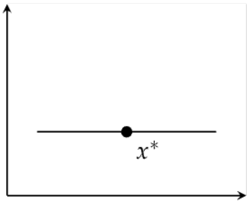
Univariate objective functions, assuming they are differentiable (Derivatives exist), without constraints

Local minimum: Necessary condition but not sufficient condition:

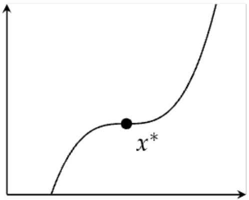
1. $f'(x^*) = 0$, the first-order necessary condition (FONC)
2. $f''(x^*) \geq 0$, the second-order necessary condition (SONC)



SONC but not FONC



FONC and SONC



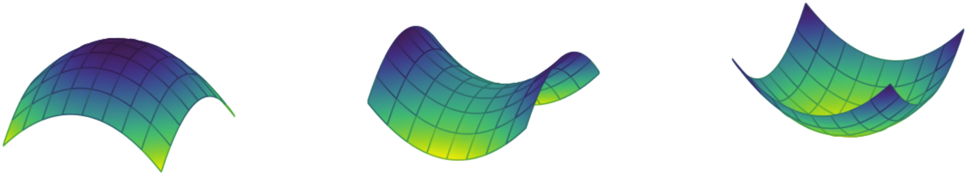
FONC and SONC

Conditions for Local Minima

Multivariate objective functions, assuming they are differentiable (gradients and Hessians exist), without constraints

Local minimum: Necessary condition but not sufficient condition:

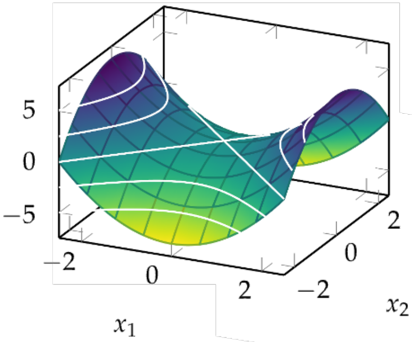
1. $\nabla f(\boldsymbol{x}^*) = 0$, the first-order necessary condition (FONC)
2. $\nabla^2 f(\boldsymbol{x}^*) \geq 0$, the second-order necessary condition (SONC)

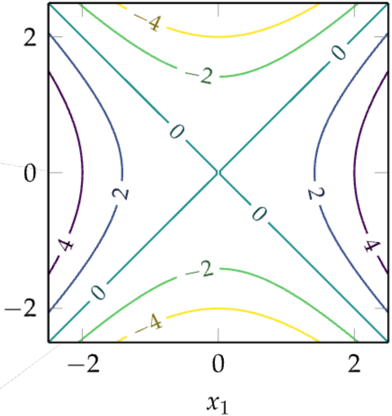


Contour Plots

$$f(x_1, x_2) = x_1^2 - x_2^2$$

can be rendered in a 3D space but convenient to represent it also in 2D showing the lines of constant output value





Taylor Expansion

From the *first fundamental theorem of calculus*,[2] we know that

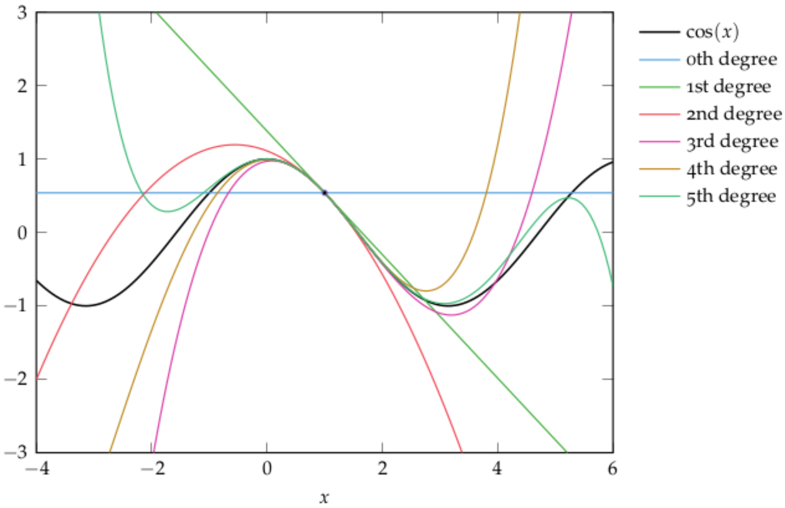
$$f(x+h) = f(x) + \int_0^h f'(x+a)\,da$$

Nesting this definition produces the Taylor expansion of f about x:

$$\begin{aligned}
f(x+h) &= f(x) + \int_0^h \left(f'(x) + \int_0^a f''(x+b)\,db \right) da \\
&= f(x) + f'(x)h + \int_0^h \int_0^a f''(x+b)\,db\,da \\
&= f(x) + f'(x)h + \int_0^h \int_0^a \left(f''(x) + \int_0^b f'''(x+c)\,dc \right) db\,da \\
&= f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \int_0^h \int_0^a \int_0^b f'''(x+c)\,dc\,db\,da \\
&\;\;\vdots \\
&= f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \ldots \\
&= \sum_{n=0}^{\infty} \frac{f^{(n)}(x)}{n!}h^n
\end{aligned}$$

Taylor Expansion

A smooth function looks like a polynomial when you zoom in closely enough.



Taylor Expansion for Multivariate Scalar-Valued Functions

The Taylor expansion of a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$ around a point $\boldsymbol{a}$ is given by:

In multiple dimensions, the Taylor expansion about $\mathbf{a}$ generalizes to

$$f(\mathbf{x}) = f(\mathbf{a}) + \nabla f(\mathbf{a})^\top (\mathbf{x} - \mathbf{a}) + \frac{1}{2}(\mathbf{x} - \mathbf{a})^\top \nabla^2 f(\mathbf{a})(\mathbf{x} - \mathbf{a}) + \ldots$$

- $\nabla f(\boldsymbol{a})$ is the gradient vector of f at $\boldsymbol{a}$, and
- $\nabla^2 f(\boldsymbol{a})$ is the Hessian matrix of f at $\boldsymbol{a}$.
- $O(||\boldsymbol{h}||^2)$ denotes the error by omitting higher-order terms. It becomes negligible as h approaches 0.

Example: Rosenbrock function

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

It has a global minimum at $(x, y) = (a, a^2)$, where $f(x, y) = 0$. Usually, these parameters are set such that $a = 1$ and $b = 100$. Only in the trivial case where $a = 0$ the function is symmetric and the minimum is at the origin.

Multivariate generalization
sum of $N/2$ uncoupled 2D Rosenbrock problems, and defined only for even N:

$$f(\mathbf{x}) = f(x_1, x_2, \ldots, x_N) = \sum_{i=1}^{N/2} \left[100(x_{2i-1}^2 - x_{2i})^2 + (x_{2i-1} - 1)^2\right].$$

This variant has predictably simple solutions.

Example: Rosenbrock function

Consider the Rosenbrock banana function,

$$f(\mathbf{x}) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$$

Does the point $(1,1)$ satisfy the FONC and SONC?

The gradient is:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2\left(10x_1^3 - 10x_1x_2 + x_1 - 1\right) \\ 10(x_2 - x_1^2) \end{bmatrix}$$

and the Hessian is:

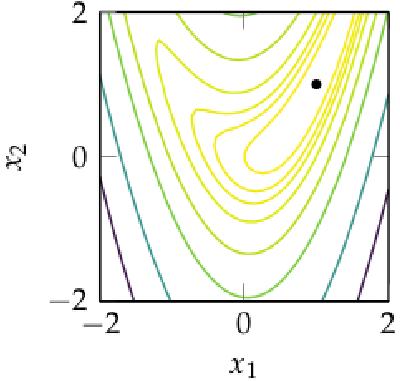
$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_2} \end{bmatrix} = \begin{bmatrix} -20(x_2 - x_1^2) + 40x_1^2 + 2 & -20x_1 \\ -20x_1 & 10 \end{bmatrix}$$

We compute $\nabla(f)([1,1]) = 0$, so the FONC is satisfied. The Hessian at $[1,1]$ is:

$$\begin{bmatrix} 42 & -20 \\ -20 & 10 \end{bmatrix}$$

which is positive definite, so the SONC is satisfied.

Example: Rosenbrock function



Overview

2. Derivatives → 4. Descent → 5. First-Order, 6. Second-Order

3. Bracketing

7. Direct

8. Stochastic → 9. Population → 20. Expressions

10. Constraints → 11. Linear → 19. Discrete

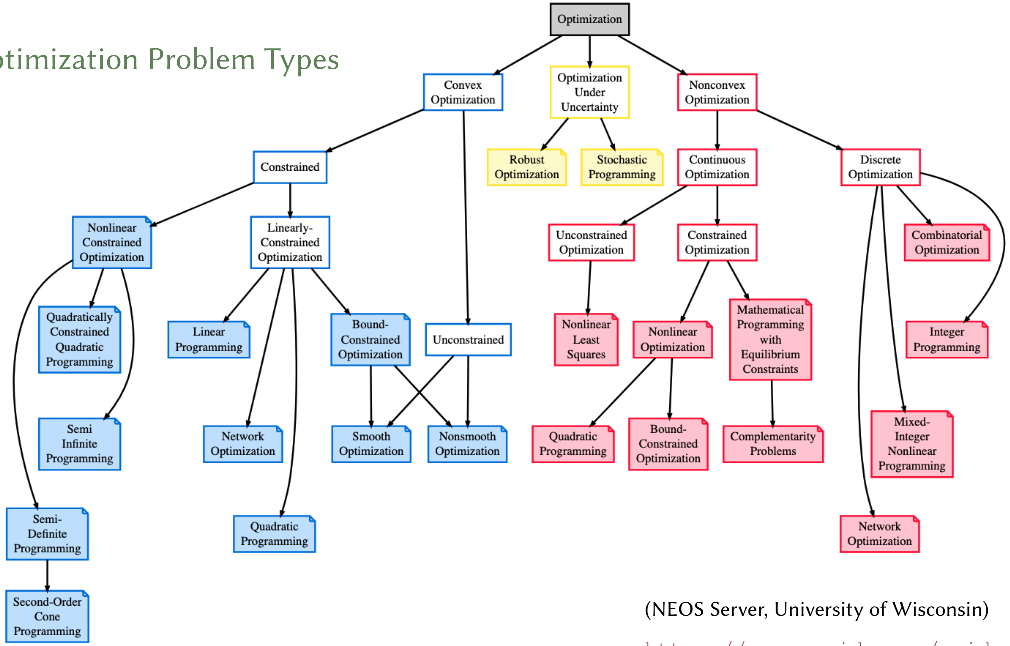
10. Constraints → 12. Multiobjective

13. Sampling Plans → 14. Surrogate Models → 15. Probabilistic Surrogate Models → 16. Surrogate Optimization

17. Optimization under Uncertainty → 18. Uncertainty Propagation

21. Multidisciplinary Design Optimization

Optimization Problem Types



(NEOS Server, University of Wisconsin)

https://neos-guide.org/guide/types/

Problem classification

Different classification factors:

- Univariate $f : \mathbb{R} \to \mathbb{R}$ vs Multivariate $f : \mathbb{R}^n \to \mathbb{R}$
- Scalar-valued $f : \mathbb{R}^n \to \mathbb{R}$ vs vector-vector functions $f : \mathbb{R}^n \to \mathbb{R}^m$
- Linear vs Nonlinear
- Nonlinear: Convex vs Nonconvex, unimodal vs multimodal
- Constrained vs unconstrained
- Smooth (differentiable) vs non smooth (non differentiable)
- Continuous vs Discrete
- Deterministic vs Uncertain

Application Example

In robotics, an autonomous agent (e.g., a drone, robotic arm, or self-driving car) needs to determine an **optimal path** from a starting position to a target while satisfying constraints such as avoiding obstacles, minimizing energy consumption, or optimizing smoothness.

Formulating the Optimization Problem

The goal is to find a smooth trajectory $x(t)$ that minimizes a cost function:

$$J = \int_{t_0}^{t^f} C(\boldsymbol{x}(t), \boldsymbol{u}(t))dt$$

$\boldsymbol{x}(t)$ is the state (e.g., position, velocity), $\boldsymbol{u}(t)$ is the control input (e.g., force, acceleration), $C(\boldsymbol{x}, \boldsymbol{u})$ is the cost function, which could represent energy usage, time, or distance.

Constraints:

- Dynamic Constraints: Governed by the system's physics (e.g., Newton's laws for a robot arm or quadcopter): $\boldsymbol{x}' = f(\boldsymbol{x}, \boldsymbol{u})$
- Obstacle Avoidance: Ensures that the trajectory does not collide with obstacles: $g(\boldsymbol{x}(t)) \geq 0, \forall t$
- Boundary Conditions: The system must start and end at given positions with certain velocities.

Summary

- Optimization is the process of finding the best system design subject to a set of constraints
- Optimization is concerned with finding global minima of a function
- Minima can occur where the gradient is zero, but zero-gradient does not imply optimality

2. Derivatives and Gradients

Definitions

- $[a, b] = \{x \in \mathbb{R} \mid a \leq x \leq b\}$ closed interval
  $(a, b) = \{x \in \mathbb{R} \mid a < x < b\}$ open interval
- We denote vectors in bold, eg, $\boldsymbol{x}$ and matrices in uppercase letters, eg, A
- Given a vector $\boldsymbol{x} \in \mathbb{R}^n$, x_i is its i-th component
- We invariably assume that $\boldsymbol{x}$ is a **column vector**,
  which we also write with parentheses as $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$ or $\boldsymbol{x} = [x_1, x_2, \ldots, x_n]$.
- The transpose $\boldsymbol{x}^T$ is a row vector

$$\boldsymbol{x}^T = \begin{bmatrix} x_1 & x_2 & \ldots & x_n \end{bmatrix}$$

- Scalar product: $\boldsymbol{v} \cdot \boldsymbol{x} = \langle \boldsymbol{v}, \boldsymbol{x} \rangle = \boldsymbol{y}^T \boldsymbol{x} = \sum_{i=1}^n y_i x_i$

Definitions

- **linear combination**

$$\begin{array}{r} \boldsymbol{v}_1, \boldsymbol{v}_2 \ldots, \boldsymbol{v}_k \in \mathbb{R}^n \\ \boldsymbol{\lambda} = [\lambda_1, \ldots, \lambda_k]^T \in \mathbb{R}^k \end{array} \qquad \boldsymbol{x} = \lambda_1 \boldsymbol{v}_1 + \cdots + \lambda_k \boldsymbol{v}_k = \sum_{i=1}^{k} \lambda_i \boldsymbol{v}_i$$

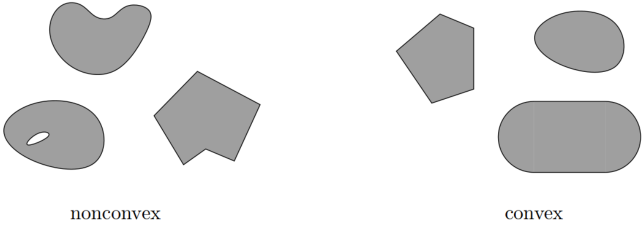
  moreover:

$\boldsymbol{\lambda} \geq \mathbf{0}$		**conic combination**	
	$\boldsymbol{\lambda}^T \mathbf{1} = 1$	**affine combination**	$\left(\sum_{i=1}^{k} \lambda_i = 1 \right)$
$\boldsymbol{\lambda} \geq \mathbf{0}$ and	$\boldsymbol{\lambda}^T \mathbf{1} = 1$	**convex combination**	

- $A\boldsymbol{x}$ column vector combination of the columns of A;
  $\boldsymbol{u}^T A$ row vector combination of the rows of A

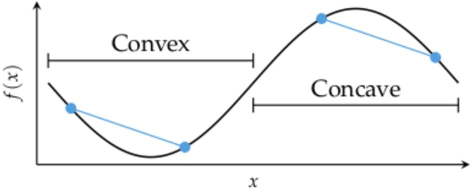
Definitions

- **convex set**: if $\boldsymbol{x}, \boldsymbol{y} \in S$ and $0 \leq \lambda \leq 1$ then $\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y} \in S$



Alternative definition:

- **convex function**
  - if its epigraph $\{(x, y) \in \mathbb{R}^2 : y \geq f(x)\}$ is a convex set or
  - if $f : \mathbb{R}^n \to \mathbb{R}$ and
    if $\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n, \alpha \in [0, 1]$ it holds that
    $f(\alpha \boldsymbol{x} + (1-\alpha)\boldsymbol{y}) \leq \alpha f(\boldsymbol{x}) + (1-\alpha) f(\boldsymbol{y})$



Definitions: Convexity

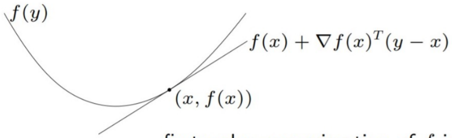
f is **differentiable** if $\mathbf{dom}\, f$ is open and the gradient

$$\nabla f(x) = \left(\frac{\partial f(x)}{\partial x_1}, \frac{\partial f(x)}{\partial x_2}, \ldots, \frac{\partial f(x)}{\partial x_n} \right)$$

exists at each $x \in \mathbf{dom}\, f$

1st-order condition: differentiable f with convex domain is convex iff

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) \quad \text{for all } x, y \in \mathbf{dom}\, f$$



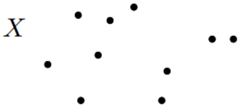
first-order approximation of f is global underestimator

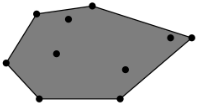
Alternative definition: $(\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y})) \cdot (\boldsymbol{x} - \boldsymbol{y}) \geq 0$ for all $\boldsymbol{x}, \boldsymbol{y}$ in the domain of f.

Definitions

- For a set of points $S \subseteq \mathbb{R}^n$

$$
\begin{aligned}
\mathrm{lin}(S) &\quad \text{linear hull (span)}\\
\mathrm{cone}(S) &\quad \text{conic hull}\\
\mathrm{aff}(S) &\quad \text{affine hull}\\
\mathrm{conv}(S) &\quad \text{convex hull}
\end{aligned}
$$





the convex hull of X

$$\boxed{\mathbf{conv}(X) = \{\lambda_1 \boldsymbol{x}_1 + \lambda_2 \boldsymbol{x}_2 + \ldots + \lambda_n \boldsymbol{x}_n \mid \boldsymbol{x}_i \in X,\ \lambda_1, \ldots, \lambda_n \geq 0 \ \text{ and } \sum_i \lambda_i = 1\}}$$

Norms

Def. A **norm** is a function that assigns a length to a vector.

A function f is a norm if:

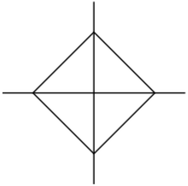
1. $f(\boldsymbol{x}) = 0$ if and only if $\boldsymbol{x}$ is the zero vector
2. $f(a\boldsymbol{x}) = |a| f(\boldsymbol{x})$, such that lengths scale
3. $f(\boldsymbol{x} + \boldsymbol{y}) \leq f(\boldsymbol{x}) + f(\boldsymbol{y})$, also known as triangle inequality

L_p norms are commonly used set of norms parameterized by a scalar $p \geq 1$:

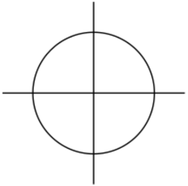
$$\|\boldsymbol{x}\|_p = \lim_{\rho \to p} \left(|x_1|^\rho + |x_2|^\rho + \ldots + |x_n|^\rho\right)^{\frac{1}{\rho}}$$

L_∞ is also called the max norm, Chebyshev distance or chessboard distance.

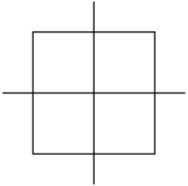
L_1: $\|\mathbf{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|$



L_2: $\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$



L_∞: $\|\mathbf{x}\|_\infty = \max(|x_1|, |x_2|, \cdots, |x_n|)$



Matrix Norms

$$\|A\| \stackrel{\text{def}}{=} \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}. \tag{A.7}$$

The matrix norms defined in this way are said to be *consistent* with the vector norms (A.2). Explicit formulae for these norms are as follows:

$$\|A\|_1 = \max_{j=1,\ldots,n} \sum_{i=1}^{m} |A_{ij}|, \tag{A.8a}$$

$$\|A\|_2 = \text{largest eigenvalue of } (A^T A)^{1/2}, \tag{A.8b}$$

$$\|A\|_\infty = \max_{i=1,\ldots,m} \sum_{j=1}^{n} |A_{ij}|. \tag{A.8c}$$

The Frobenius norm $\|A\|_F$ of the matrix A is defined by

$$\|A\|_F = \left(\sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij}^2 \right)^{1/2}. \tag{A.9}$$

Condition Number

For a nonsingular matrix A, the **condition number** $\kappa(A)$ is defined as

$$\kappa(A) = \|A\| \left\|A^{-1}\right\|$$

where $\|\cdot\|$ is any matrix norm. The condition number of a matrix is a measure of how close the matrix is to being singular. A matrix with a large condition number is said to be ill-conditioned, and small changes in the input can lead to large changes in the output when solving linear systems or performing matrix operations involving that matrix.

Outline

3. Derivatives

4. Symbolic Differentiation

5. Numerical Differentiation

6. Automatic Differentiation

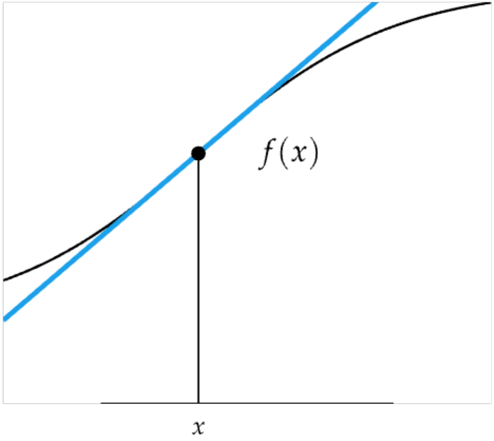
Derivatives

- Derivatives tell us which direction to search for a solution

- Slope of Tangent Line

$$f'(x) := \frac{\mathrm{d}f(x)}{\mathrm{d}x}$$

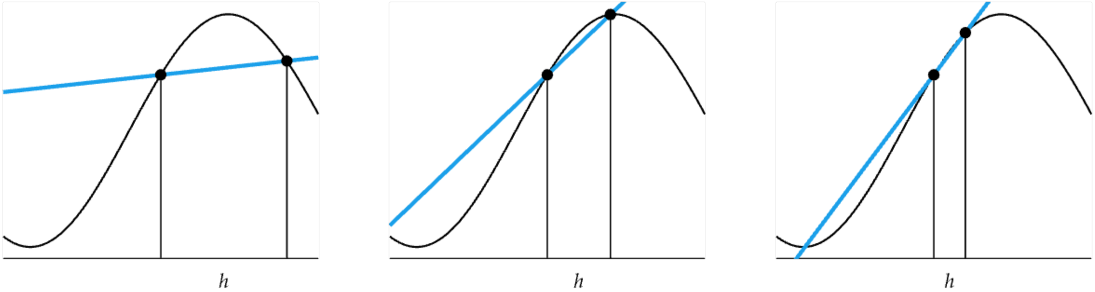
(Leibniz notation)



Derivatives

$$f(x+\Delta x) \approx f(x) + f'(x)\Delta x$$

$$f'(x) = \frac{\Delta f(x)}{\Delta x}$$



Symbolic Differentiation

$$f'(x) \equiv \underbrace{\lim_{h\to 0} \frac{f(x+h) - f(x)}{h}}_{\text{forward difference}} = \underbrace{\lim_{h\to 0} \frac{f(x+h/2) - f(x-h/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h\to 0} \frac{f(x) - f(x-h)}{h}}_{\text{backward difference}}$$

Symbolic Differentiation

```python
import sympy as sp

# Define the variable
x = sp.symbols('x')

# Define the function
f = x**2 + x/2 - sp.sin(x)/x

# Compute the derivative
df_dx = sp.diff(f, x)

# Display the result
print("The symbolic derivative of f is:")
print(df_dx)
```

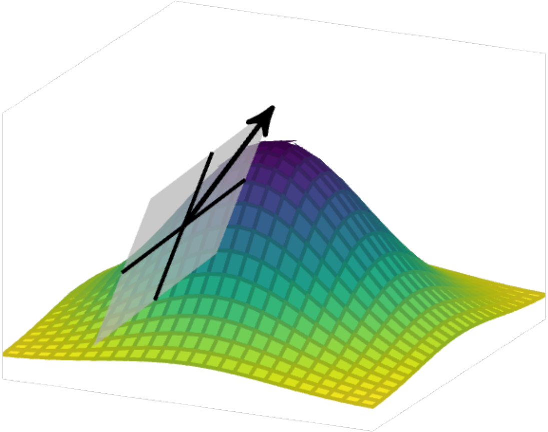
Derivatives in Multiple Dimensions

- Gradient Vector

$$\nabla f(\boldsymbol{x})^T = \begin{bmatrix} \frac{\partial f(\boldsymbol{x})}{\partial x_1} & \frac{\partial f(\boldsymbol{x})}{\partial x_2} & \dots & \frac{\partial f(\boldsymbol{x})}{\partial x_n} \end{bmatrix}$$

- Hessian Matrix

$$\nabla^2 f(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1\,\partial x_1} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1\,\partial x_2} & \dots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1\,\partial x_n} \\ \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1\,\partial x_2} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2\,\partial x_2} & \dots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2\,\partial x_n} \\ \vdots & \ddots & & \vdots \\ \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1\,\partial x_n} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2\,\partial x_n} & \dots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_n\,\partial x_n} \end{bmatrix}$$



Directional derivative

The **directional derivative** $\nabla_s f(x)$ of a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$ is the instantaneous rate of change of $f(\boldsymbol{x})$ as $\boldsymbol{x} = [x_1, x_2, \ldots, x_n]$ is moved with velocity $\boldsymbol{s} = [s_1, s_2, \ldots, s_n]$.

$$\nabla_{\mathbf{s}} f(\mathbf{x}) \equiv \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{s}) - f(\mathbf{x})}{h}}_{\text{forward difference}} = \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{s}/2) - f(\mathbf{x} - h\mathbf{s}/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x}) - f(\mathbf{x} - h\mathbf{s})}{h}}_{\text{backward difference}}$$

To compute $\nabla_s f(\boldsymbol{x})$:

- compute $\boxed{\nabla_s f(\boldsymbol{x}) = \frac{\partial f(\boldsymbol{x})}{\partial x_1} s_1 + \frac{\partial f(\boldsymbol{x})}{\partial x_2} s_2 + \ldots + \frac{\partial f(\boldsymbol{x})}{\partial x_n} s_n = \nabla f(\boldsymbol{x})^T \boldsymbol{s} = \nabla f(\boldsymbol{x}) \cdot \boldsymbol{s}}$

- $g(\alpha) := f(\boldsymbol{x} + \alpha \boldsymbol{s})$ and then compute $g'(0)$

We wish to compute the directional derivative of $f(\mathbf{x}) = x_1 x_2$ at $\mathbf{x} = [1, 0]$ in the direction $\mathbf{s} = [-1, -1]$:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \quad \frac{\partial f}{\partial x_2}\right] = [x_2, x_1]$$

$$\nabla_{\mathbf{s}} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{s} = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} = -1$$

We can also compute the directional derivative as follows:

$$g(\alpha) = f(\mathbf{x} + \alpha \mathbf{s}) = (1 - \alpha)(-\alpha) = \alpha^2 - \alpha$$

$$g'(\alpha) = 2\alpha - 1$$

$$g'(0) = -1$$

Matrix Calculus

Common gradient:

$$\nabla_{\boldsymbol{x}} \boldsymbol{b}^T \boldsymbol{x} = ?$$

$$\boldsymbol{b}^T \boldsymbol{x} = [b_1 x_1 + b_2 x_2 + \ldots + b_n x_n]$$

$$\frac{\partial \boldsymbol{b}^T \boldsymbol{x}}{\partial x_i} = b_i$$

$$\nabla_{\boldsymbol{x}} \boldsymbol{b}^T \boldsymbol{x} = \nabla_{\boldsymbol{x}} \boldsymbol{x}^T \boldsymbol{b} = \boldsymbol{b}$$

Matrix Calculus

Common gradient:

$$\nabla_{\boldsymbol{x}} \boldsymbol{x}^T A \boldsymbol{x} = ?$$

$$\boldsymbol{x}^T A \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}^T \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}^T \begin{bmatrix} x_1 a_{11} + x_2 a_{12} + \dots + x_n a_{1n} \\ x_1 a_{21} + x_2 a_{22} + \dots + x_n a_{2n} \\ \vdots \\ x_1 a_{n1} + x_2 a_{n2} + \dots + x_n a_{nn} \end{bmatrix}$$

$$= \begin{matrix} x_1^2 a_{11} + x_1 x_2 a_{12} + \dots + x_1 x_n a_{1n} + \\ x_1 x_2 a_{21} + x_2^2 a_{22} + \dots + x_2 x_n a_{2n} + \\ \vdots \\ x_1 x_n a_{n1} + x_2 x_n a_{n2} + \dots + x_n^2 a_{nn} \end{matrix}$$

$$\frac{\partial}{\partial x_i}\mathbf{x}^T A\mathbf{x} = \sum_{j=1}^{n} x_j\left(a_{ij} + a_{ji}\right)$$

$$\nabla_x \mathbf{x}^T A\mathbf{x} = \begin{bmatrix} \sum_{j=1}^n x_j\left(a_{1j} + a_{j1}\right) \\ \sum_{j=1}^n x_j\left(a_{2j} + a_{j2}\right) \\ \vdots \\ \sum_{j=1}^n x_j\left(a_{nj} + a_{jn}\right) \end{bmatrix} = \begin{bmatrix} a_{11} + a_{11} & a_{12} + a_{21} & \dots & a_{1n} + a_{n1} \\ a_{21} + a_{12} & a_{22} + a_{22} & \dots & a_{2n} + a_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} + a_{1n} & a_{n2} + a_{2n} & \dots & a_{nn} + a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \left(A + A^T\right)\mathbf{x}$$

Smoothness

Def. The **smoothness** of a function is a property measured by the number of continuous derivatives (differentiability class) it has over its domain.

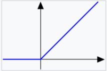
A function of **class** C^k is a function of smoothness at least k; that is, a function of class C^k is a function that has a kth derivative that is continuous in its domain.

The term **smooth function** refers to a C^∞-function. However, it may also mean “sufficiently differentiable” for the problem under consideration.

Smoothness

- Let U be an open set on the real line and a function f defined on U with real values. Let k be a non-negative integer.
- The function f is said to be of **differentiability class** C^k if the derivatives $f', f'', \ldots, f^{(k)}$ exist and are continuous on U.
- If f is k-differentiable on U, then it is at least in the class C^{k-1} since $f', f'', \ldots, f^{(k-1)}$ are continuous on U.
- The function f is said to be **infinitely differentiable**, **smooth**, or of **class** C^∞, if it has derivatives of all orders (continuous) on U.
- The function f is said to be of **class** C^ω, or **analytic**, if f is smooth and its Taylor series expansion around any point in its domain converges to the function in some neighborhood of the point.
- There exist functions that are smooth but not analytic; C^ω is thus strictly contained in C^∞. Bump functions are examples of functions with this property.

Example: continuous (C^0) but not differentiable [edit]



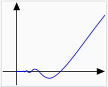
The C^0 function $f(x) = x$ for $x \geq 0$ and 0 otherwise.

The function

$$f(x) = \begin{cases} x & \text{if } x \geq 0, \\ 0 & \text{if } x < 0 \end{cases}$$

is continuous, but not differentiable at $x = 0$, so it is of class C^0, but not of class C^1.

Example: finitely-times differentiable (C^k) [edit]



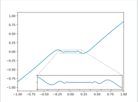
The function $g(x) = x^2 \sin(1/x)$ for $x > 0$.

For each even integer k, the function

$$f(x) = |x|^{k+1}$$

is continuous and k times differentiable at all x. At $x = 0$, however, f is not $(k + 1)$ times differentiable, so f is of class C^k, but not of class C^j where $j > k$.

Example: differentiable but not continuously differentiable (not C^1) [edit]



The function $f : \mathbb{R} \to \mathbb{R}$ with $f(x) = x^2 \sin\left(\frac{1}{x}\right)$ for $x \neq 0$ and $f(0) = 0$ is differentiable. However, this function is not continuously differentiable.

The function

$$g(x) = \begin{cases} x^2 \sin\left(\frac{1}{x}\right) & \text{if } x \neq 0, \\ 0 & \text{if } x = 0 \end{cases}$$

is differentiable, with derivative

$$g'(x) = \begin{cases} -\cos\left(\frac{1}{x}\right) + 2x \sin\left(\frac{1}{x}\right) & \text{if } x \neq 0, \\ 0 & \text{if } x = 0. \end{cases}$$

Because $\cos(1/x)$ oscillates as $x \to 0$, $g'(x)$ is not continuous at zero. Therefore, $g(x)$ is differentiable but not of class C^1.

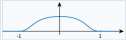
Example: differentiable but not Lipschitz continuous [edit]

The function

$$h(x) = \begin{cases} x^{4/3} \sin\left(\frac{1}{x}\right) & \text{if } x \neq 0, \\ 0 & \text{if } x = 0 \end{cases}$$

is differentiable but its derivative is unbounded on a compact set. Therefore, h is an example of a function that is differentiable but not locally Lipschitz continuous.

Example: analytic (C^ω) [edit]



A smooth function that is not analytic.

The exponential function e^x is analytic, and hence falls into the class C^ω (where ω is the smallest transfinite ordinal). The trigonometric functions are also analytic wherever they are defined, because they are linear combinations of complex exponential functions e^{ix} and e^{-ix}.

Example: smooth (C^∞) but not analytic (C^ω) [edit]

The bump function

$$f(x) = \begin{cases} e^{-\frac{1}{1-x^2}} & \text{if } |x| < 1, \\ 0 & \text{otherwise} \end{cases}$$

is smooth, so of class C^∞, but it is not analytic at $x = \pm 1$, and hence is not of class C^ω. The function f is an example of a smooth function with compact support.

Positive Definteness

Def. A square, symmetric matrix A is **positive definite** if $\mathbf{x}^T A\mathbf{x}$ is positive for all points other than the origin: $\mathbf{x}^T A\mathbf{x} > 0$ for all $x \neq 0$.
Def. A square, symmetric matrix A is **positive semidefinite** if $\mathbf{x}^T A\mathbf{x}$ is always non-negative: $\mathbf{x}^T A\mathbf{x} \geq 0$ for all $\mathbf{x}$.

A matrix A is positive definite if and only if all its eigenvalues are positive. (This can be used for recognition.)

If the matrix A is positive definite in the function $f(\mathbf{x}) = \mathbf{x}^T A\mathbf{x}$, then f has a unique global minimum.

Recall that the second order Taylor approximation of a twice-differentiable function f at $\mathbf{x}_0$ is

$$f(\mathbf{x}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^T(\mathbf{x} - \mathbf{x}_0) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^T H_0(\mathbf{x} - \mathbf{x}_0)$$

where H_0 is the Hessian evaluated at $\mathbf{x}_0$. If $(\mathbf{x} - \mathbf{x}_0)^T H_0(\mathbf{x} - \mathbf{x}_0)$ has a unique global minimum, then the overall approximation has a unique global minimum.

Non-symmetric matrices can be positive definite:
Every real matrix A decomposes uniquely as a symmetric matrix and a skew-symmetric matrix:

$$A = S + K, \quad S = \frac{1}{2}(A + A^T), \quad K = \frac{1}{2}(A - A^T).$$

Then:

$$\boldsymbol{x}^T A x = \boldsymbol{x}^T S \boldsymbol{x},$$

because

$$\boldsymbol{x}^T K \boldsymbol{x} = 0 \text{ for all } \boldsymbol{x}.$$

However, all local optimization algorithms are based, explicitly or implicitly, on the second-order Taylor expansion. The matrix governing local curvature is the Hessian and Hessians are necessarily symmetric

LU Decomposition

The LU factorization of a matrix $A \in \mathbb{R}^{n \times n}$ is defined as

$$PA = LU, \tag{A.20}$$

where

P is an $n \times n$ permutation matrix (that is, it is obtained by rearranging the rows of the $n \times n$ identity matrix),

L is unit lower triangular (that is, lower triangular with diagonal elements equal to 1, and

U is upper triangular.

- This factorization can be used to solve efficiently a linear system of the form $A\boldsymbol{x} = \boldsymbol{b}$ by first solving $L\boldsymbol{y} = \boldsymbol{b}$ for y and then solving $U\boldsymbol{x} = \boldsymbol{y}$ for $\boldsymbol{x}$.
- It can be found by using Gaussian elimination with row partial pivoting, an algorithm that requires approximately $2n^3/3$ floating-point operations when A is dense. Done by standard software LAPACK.

Cholesky Decomposition

When $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite, it is possible to compute a similar but more specialized factorization at about half the cost—about $n^3/3$ operations. This factorization, known as the Cholesky factorization, produces a matrix L such that

$$A = LL^T. \tag{A.23}$$

(If we require L to have positive diagonal elements, it is uniquely defined by this formula.)

- It can also be used to solve systems of linear equations
- The Cholesky factorization can also be used to verify positive definiteness of a symmetric matrix A.

Outline

Symbolic Derivatives

- Symbolic derivatives can give valuable insight into the structure of the problem domain and, in some cases, produce analytical solutions of extrema (e.g., solving for $\frac{d}{dx} f(x) = 0$) that can eliminate the need for derivative calculation altogether.

- But they do not lend themselves to efficient runtime calculation of derivative values, as they can get exponentially larger than the expression whose derivative they represent

Outline

Numerical Differentiation

Finite Difference Method

- Neighboring points are used to approximate the derivative

$$f'(x) \approx \underbrace{\frac{f(x+h)-f(x)}{h}}_{\text{forward difference}} \approx \underbrace{\frac{f(x+h/2)-f(x-h/2)}{h}}_{\text{central difference}} \approx \underbrace{\frac{f(x)-f(x-h)}{h}}_{\text{backward difference}}$$

- h too small causes numerical cancellation errors (square root or cube root of the machine precision for floating point values: `sys.float_info.epsilon` difference between 1 and closest representable number)

Derivation

from Taylor series expansion:

$$f(x+h) = f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \cdots$$

We can rearrange and solve for the first derivative:

$$f'(x)h = f(x+h) - f(x) - \frac{f''(x)}{2!}h^2 - \frac{f'''(x)}{3!}h^3 - \cdots$$

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{f''(x)}{2!}h - \frac{f'''(x)}{3!}h^2 - \cdots$$

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

- forward difference has error term $O(h)$, linear error as h approaches zero
- central difference has error term is $O(h^2)$

```python
import sys
import numpy as np

def diff_forward(f, x: float, h: float=np.sqrt(sys.float_info.epsilon)) -> float:
    return (f(x+h) - f(x))/h

def diff_central(f, x: float, h: float=np.cbrt(sys.float_info.epsilon)) -> float:
    return (f(x+h/2) - f(x-h/2))/h

def diff_backward(f, x: float, h: float=np.sqrt(sys.float_info.epsilon)) -> float:
    return (f(x) - f(x-h))/h

# Example usage
def func(x):
    return x**2 + np.sin(x)

x0 = 1.0
print(f"The derivative at x = {x0} is {diff_forward(func, x0)}")
```

Numerical Differentiation

Complex step method

Uses one single function evaluation after taking a step in the imaginary direction.

$$f(x+ih) = f(x) + ihf'(x) - h^2\frac{f''(x)}{2!} - ih^3\frac{f'''(x)}{3!} + \cdots$$

$$\begin{aligned}
\mathrm{Im}(f(x+ih)) &= hf'(x) - h^3\frac{f'''(x)}{3!} + \cdots \\
\Rightarrow f'(x) &= \frac{\mathrm{Im}(f(x+ih))}{h} + h^2\frac{f'''(x)}{3!} - \cdots \\
&= \frac{\mathrm{Im}(f(x+ih))}{h} + O(h^2) \text{ as } h \to 0
\end{aligned}$$

$$\begin{aligned}
\mathrm{Re}(f(x+ih)) &= f(x) - h^2\frac{f''(x)}{2!} + \ldots \\
\Rightarrow f(x) &= \mathrm{Re}(f(x+ih)) + h^2\frac{f''(x)}{2!} - \cdots
\end{aligned}$$

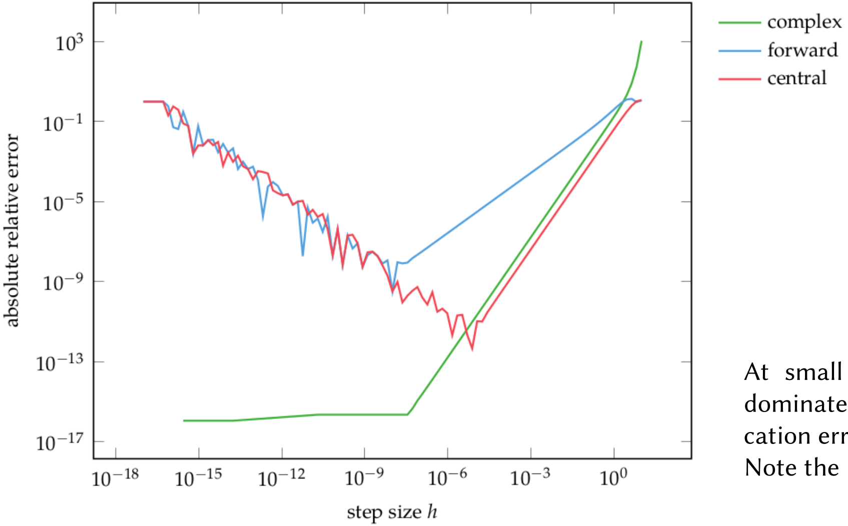
```python
import numpy as np

def diff_complex(f, x: float, h: float=1e-20) -> float:
    return np.imag(f(x + h * 1j)) / h

# Example usage
def func(x):
    return x**2 + np.sin(x)

x0 = 1.0
print(f"The derivative at x = {x0} is {diff_complex(func, x0)}")
```

Numerical Differentiation Error Comparison



At small h, round off errors dominate, and at large h, truncation errors dominate.
Note the log transformation.

Numerical Differentiation in ML

- Approximation errors would be tolerated in a deep learning setting thanks to the well-documented error resiliency of neural network architectures (Gupta et al., 2015).

- The $O(n)$ complexity of numerical differentiation for a gradient in n dimensions is the main obstacle to its usefulness in machine learning, where n can be as large as millions or billions in state-of-the-art deep learning models (Shazeer et al., 2017).

Outline

Automatic Differentiation

Automatic differentiation techniques are founded on the observation that any function is evaluated by performing a sequence of simple elementary operations involving just one or two arguments at a time:

- addition
- multiplication
- division
- power operation a^b
- trigonometric functions
- exponential functions
- logarithmic
- chain rule:

Chain Rule

The chain rule is a formula that expresses the derivative of the composition of two differentiable functions. More precisely, if $h = z \circ y$ is the composition such that $h(x) = z(y(x))$ for every x, then the chain rule is:

in Lagrange's notation:

$$h'(x) = z'(y(x))y'(x).$$

in Leibniz's notation:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx},$$

and

$$\left.\frac{dz}{dx}\right|_{x} = \left.\frac{dz}{dy}\right|_{y(x)} \cdot \left.\frac{dy}{dx}\right|_{x},$$

for indicating at which points the derivatives have to be evaluated.

- Forward Accumulation is equivalent to expanding a function using the chain rule and computing the derivatives inside-out
- Requires n-passes to compute n-dimensional gradient
- Example:

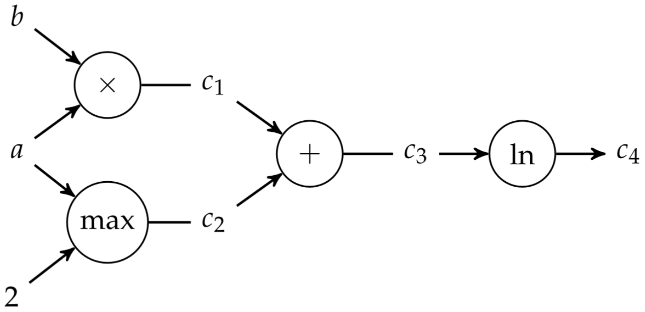
$$f(a, b) = \ln(ab + \max(a, 2))$$

$$\begin{aligned}
\frac{\partial f}{\partial a} &= \frac{\partial}{\partial a} \ln(ab + \max(a,2)) \\
&= \frac{1}{ab + \max(a,2)} \frac{\partial}{\partial a}(ab + \max(a,2)) \\
&= \frac{1}{ab + \max(a,2)} \left[\frac{\partial(ab)}{\partial a} + \frac{\partial \max(a,2)}{\partial a} \right] \\
&= \frac{1}{ab + \max(a,2)} \left[\left(b\frac{\partial a}{\partial a} + a\frac{\partial b}{\partial a} \right) + \left((2 > a)\frac{\partial 2}{\partial a} + (2 < a)\frac{\partial a}{\partial a} \right) \right] \\
&= \frac{1}{ab + \max(a,2)} [b + (2 < a)]
\end{aligned}$$

Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

Forward accumulation for $f(a,b) = \ln(ab + \max(a,2))$



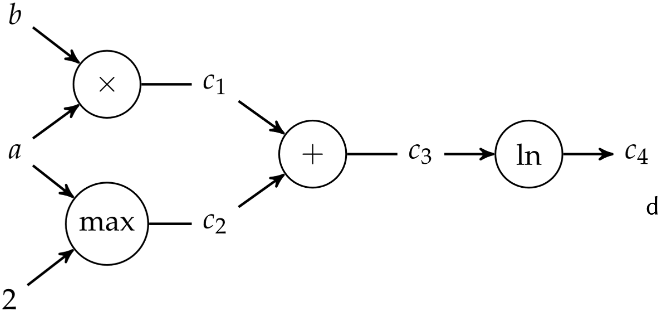
Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

Forward accumulation for $f(a,b) = \ln(ab + \max(a,2))$

$b = 2$
$\dot{b} = 0$

$a = 3$
$\dot{a} = 1$



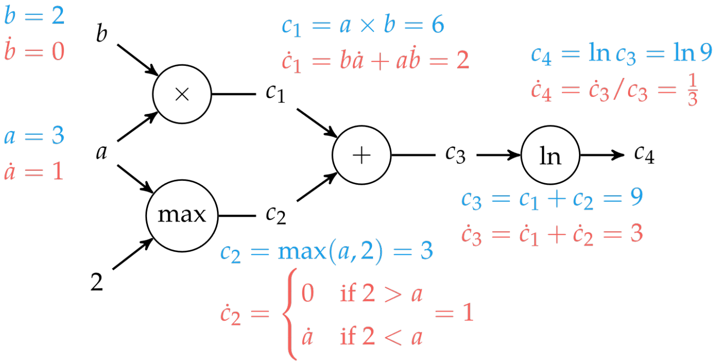
dual numbers

$\frac{\partial b}{\partial a} = \dot{b}$ Newton notation

Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

Forward accumulation for $f(a,b) = \ln(ab + \max(a,2))$



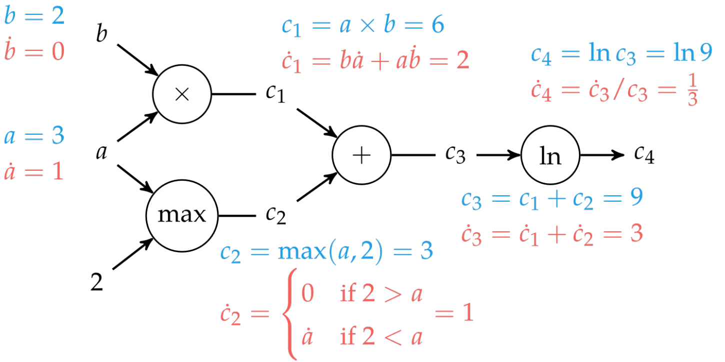
dual numbers

$\frac{\partial b}{\partial a} = \dot{b}$ Newton notation

Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

Forward accumulation for $f(a,b) = \ln(ab + \max(a,2))$



dual numbers

$\frac{\partial b}{\partial a} = \dot b$ Newton notation

for $\frac{\partial f}{\partial b}$ set $\dot a = 0, \dot b = 1$

Dual numbers

- Dual numbers can be expressed mathematically by including the abstract quantity ϵ, where ϵ^2 is defined to be 0.

- Like a complex number, a dual number is written $a + b\epsilon$ where a and b are both real values.

- $(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$
  $(a + b\epsilon) \times (c + d\epsilon) = (ac) + (ad + bc)\epsilon$

- by passing a dual number into any smooth function f, we get the evaluation and its derivative. We can show this using the Taylor series:

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(x - a)^k$$

$$f(a + b\epsilon) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(a + b\epsilon - a)^k$$

$$= \sum_{k=0}^{\infty} \frac{f^{(k)}(a)b^k\epsilon^k}{k!}$$

$$= f(a) + bf'(a)\epsilon + \epsilon^2 \sum_{k=2}^{\infty} \frac{f^{(k)}(a)b^k}{k!}\epsilon^{(k-2)}$$

$$= f(a) + bf'(a)\epsilon$$

Note that

$$
\begin{aligned}
(v + \dot{v}\epsilon) + (u + \dot{u}\epsilon) &= (v + u) + (\dot{v} + \dot{u})\epsilon \\
(v + \dot{v}\epsilon)(u + \dot{u}\epsilon) &= (vu) + (v\dot{u} + \dot{v}u)\epsilon \ ,
\end{aligned}
$$

satisfies the rules of differentiation

Setting:

$$
f(v + \dot{v}\epsilon) = f(v) + f'(v)\dot{v}\epsilon
$$

The chain rule follows:

$$
\begin{aligned}
f(g(v + \dot{v}\epsilon)) &= f(g(v) + g'(v)\dot{v}\epsilon) \\
&= f(g(v)) + f'(g(v))g'(v)\dot{v}\epsilon \ .
\end{aligned}
$$

Automatic Differentiation

- **Reverse accumulation** is performed in a single run using two passes $O(m \cdot \text{ops}(f))$ (forward and back) for $f : \mathbb{R}^n \to \mathbb{R}^m$

- Note: this is central to the back-propagation algorithm used to train neural networks because it needs only one pass for the n-dimensional function to find the gradient.

- implemented through two different operation overloading functions (for forward and backward)

- Many open-source software implementations are available: eg, Tensorflow

Forward implements:

$$\frac{df}{dx} = \frac{df}{dc_4}\frac{dc_4}{dx} = \frac{df}{dc_4}\left(\frac{dc_4}{dc_3}\frac{dc_3}{dx}\right) = \frac{df}{dc_4}\left(\frac{dc_4}{dc_3}\left(\frac{dc_3}{dc_2}\frac{dc_2}{dx} + \frac{dc_3}{dc_1}\frac{dc_1}{dx}\right)\right)$$

Backward implements:

$$\frac{df}{dx} = \frac{df}{dc_4}\frac{dc_4}{dx} = \left(\frac{df}{dc_3}\frac{dc_3}{dc_4}\right)\frac{dc_4}{dx} = \left(\left(\frac{df}{dc_2}\frac{dc_2}{dc_3} + \frac{df}{dc_1}\frac{dc_1}{dc_3}\right)\frac{dc_3}{dc_4}\right)\frac{dc_4}{dx}$$

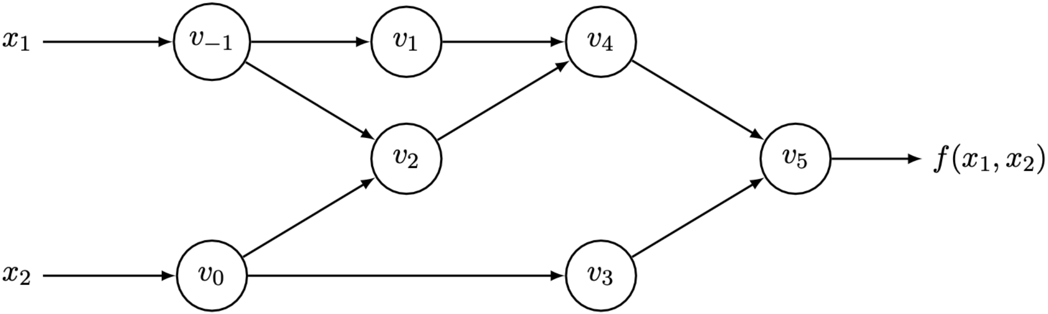
Complementing each intermediate variable v_i with an **adjoint**

$$\bar{v}_i = \frac{\partial y_j}{\partial v_i}$$

which represents the sensitivity of a considered output y_j with respect to changes in v_i.

Example

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - sin(x_2)$$



Example: Forward Accumulation

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - sin(x_2)$$

Forward Primal Trace		
v_{-1}	$= x_1$	$= 2$
v_0	$= x_2$	$= 5$
v_1	$= \ln v_{-1}$	$= \ln 2$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$
v_3	$= \sin v_0$	$= \sin 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$
y	$= v_5$	$= 11.652$

Forward Tangent (Derivative) Trace		
$\dot{v}_{-1}$	$= \dot{x}_1$	$= 1$
$\dot{v}_0$	$= \dot{x}_2$	$= 0$
$\dot{v}_1$	$= \dot{v}_{-1} / v_{-1}$	$= 1/2$
$\dot{v}_2$	$= \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1}$	$= 1 \times 5 + 0 \times 2$
$\dot{v}_3$	$= \dot{v}_0 \times \cos v_0$	$= 0 \times \cos 5$
$\dot{v}_4$	$= \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$
$\dot{v}_5$	$= \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$
$\boldsymbol{\dot{y}}$	$\boldsymbol{= \dot{v}_5}$	$\mathbf{= 5.5}$

$$O(n \cdot \text{ops}(f))$$

Example: Reverse Accumulation

Forward Primal Trace

v_{-1}	$= x_1$	$= 2$
v_0	$= x_2$	$= 5$
v_1	$= \ln v_{-1}$	$= \ln 2$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$
v_3	$= \sin v_0$	$= \sin 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$
y	$= v_5$	$= 11.652$

Reverse Adjoint (Derivative) Trace

$\bar{\boldsymbol{x}}_1$	$= \bar{\boldsymbol{v}}_{-1}$		$= \mathbf{5.5}$
$\bar{\boldsymbol{x}}_2$	$= \bar{\boldsymbol{v}}_0$		$= \mathbf{1.716}$
$\bar{v}_{-1}$	$= \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}}$	$= \bar{v}_{-1} + \bar{v}_1 / v_{-1}$	$= 5.5$
$\bar{v}_0$	$= \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0}$	$= \bar{v}_0 + \bar{v}_2 \times v_{-1}$	$= 1.716$
$\bar{v}_{-1}$	$= \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}}$	$= \bar{v}_2 \times v_0$	$= 5$
$\bar{v}_0$	$= \bar{v}_3 \frac{\partial v_3}{\partial v_0}$	$= \bar{v}_3 \times \cos v_0$	$= -0.284$
$\bar{v}_2$	$= \bar{v}_4 \frac{\partial v_4}{\partial v_2}$	$= \bar{v}_4 \times 1$	$= 1$
$\bar{v}_1$	$= \bar{v}_4 \frac{\partial v_4}{\partial v_1}$	$= \bar{v}_4 \times 1$	$= 1$
$\bar{v}_3$	$= \bar{v}_5 \frac{\partial v_5}{\partial v_3}$	$= \bar{v}_5 \times (-1)$	$= -1$
$\bar{v}_4$	$= \bar{v}_5 \frac{\partial v_5}{\partial v_4}$	$= \bar{v}_5 \times 1$	$= 1$
$\bar{v}_5$	$= \bar{y}$	$= 1$	

$O(m \cdot \mathrm{ops}(f))$

Summary

- Derivatives are useful in optimization because they provide information about how to change a given point in order to improve the objective function

- For multivariate functions, various derivative-based concepts are useful for directing the search for an optimum, including the gradient, the Hessian, and the directional derivative

- computation of derivatives in computer programs can be classified into four categories:

  1. manually working out derivatives and coding them (error prone and time consuming)

  2. numerical differentiation using finite difference approximations
     Complex step method can eliminate the effect of subtractive cancellation error when taking small steps

  3. symbolic differentiation using expression manipulation in computer algebra systems

  4. automatic differentiation, (aka algorithmic differentiation)
     forward and reverse accumulation on computational graphs

3. Bracketing

Solutions and Recognizing Them for Smooth Functions

Definition

A point $\boldsymbol{x}^*$ is a global minimizer if $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x}$.

Definition

A point $\boldsymbol{x}^*$ is a local minimizer if there is a neighborhood N of $\boldsymbol{x}^*$ such that $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x} \in N$.

Solutions and Recognizing Them for Smooth Functions

Theorem (Taylor's Theorem)

Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ *is continuously differentiable and that* $\boldsymbol{p} \in \mathbb{R}^n$*. Then we have that*

$$f(\boldsymbol{x} + \boldsymbol{p}) = f(\boldsymbol{x}) + \nabla f(\boldsymbol{x} + t\boldsymbol{p})^T \boldsymbol{p},$$

for some $t \in (0, 1)$*. Moreover, if* f *is twice continuously differentiable, we have that*

$$\nabla f(\boldsymbol{x} + \boldsymbol{p}) = \nabla f(\boldsymbol{x}) + \int_0^1 \nabla^2 f(\boldsymbol{x} + t\boldsymbol{p})\boldsymbol{p}\, dt,$$

and that

$$f(\boldsymbol{x} + \boldsymbol{p}) = f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^T \boldsymbol{p} + \frac{1}{2}\boldsymbol{p}^T \nabla^2 f(\boldsymbol{x} + t\boldsymbol{p})\boldsymbol{p},$$

for some $t \in (0, 1)$*.*

Solutions and Recognizing Them for Smooth Functions

Theorem (First-Order Necessary Conditions)

If $\boldsymbol{x}^*$ is a local minimizer and f is continuously differentiable in an open neighborhood of $\boldsymbol{x}^*$, then $\nabla f(\boldsymbol{x}^*) = 0$.

Definition (Stationary Point)

We call $\boldsymbol{x}^*$ a stationary point if $\nabla f(\boldsymbol{x}^*) = 0$.

Theorem (Second-Order Necessary Conditions)

If $\boldsymbol{x}^*$ is a local minimizer of f and $\nabla^2 f$ exists and is continuous in an open neighborhood of $\boldsymbol{x}^*$, then $\nabla f(\boldsymbol{x}^*) = 0$ and $\nabla^2 f(\boldsymbol{x}^*)$ is positive semidefinite.

Theorem (Second-Order Sufficient Conditions)

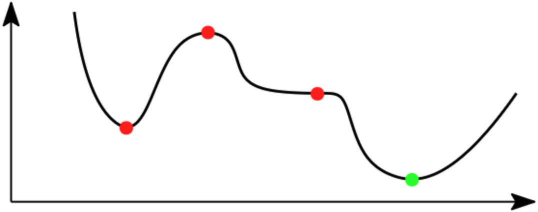
Suppose that $\nabla^2 f$ is continuous in an open neighborhood of $\boldsymbol{x}^*$ and that $\nabla f(\boldsymbol{x}^*) = 0$ and $\nabla^2 f(\boldsymbol{x}^*)$ is positive definite. Then $\boldsymbol{x}^*$ is a strict local minimizer of f.

Solutions and Recognizing Them for Smooth Functions

Theorem (Local Minimizers in Convex Functions)

Assume that $f : \mathbb{R}^n \text{ß} \mathbb{R}$ *is convex, then* $\boldsymbol{x}^*$ *is a global minimizer of* f *if and only if it is a local minimizer. If in addition* f *is differentiable, then* $\boldsymbol{x}^*$ *is a global minimizer of* f *if and only if it is a stationary point of* f*, i.e.,* $\nabla f(\boldsymbol{x}^*) = 0$.

Thus, for convex functions, we only need to look for stationary points. This is not the case for potentially nonconvex functions. For example, in one dimension below, all red points are stationary points that are not the global minimum (shown is in green)



Unconstrained Optimization of Smooth Functions

Two main strategies for finding local minima of smooth functions are:

- Line search methods (gradient descent and its variants)

- Trust region methods

A component of both strategies is the ability to find a local minimum of a one-dimensional function, which is the topic of this lecture.

Bracketing: A derivative-free method to identify an interval containing a local minimum and then successively shrinking that interval

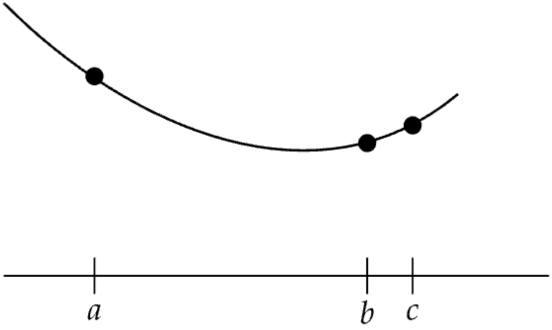
Unimodality

The algorithms in this class assume **unimodality**:

There exists a unique optimizer $\boldsymbol{x}^*$ such that f is monotonically decreasing for $\boldsymbol{x} \leq \boldsymbol{x}^*$ and monotonically increasing for $\boldsymbol{x} \geq \boldsymbol{x}^*$

Finding an Initial Bracket

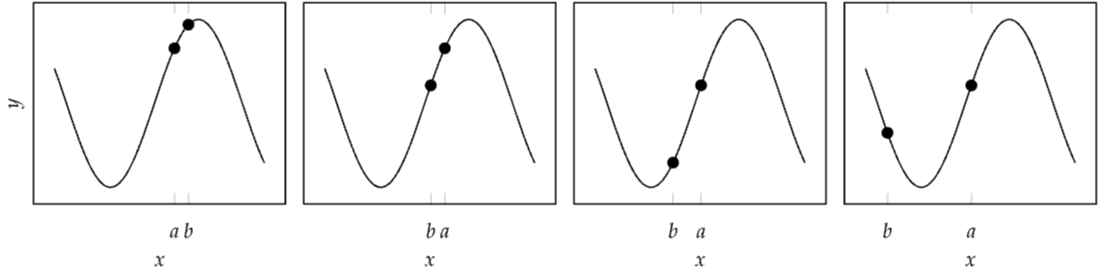
Given a unimodal function, the global minimum is guaranteed to be inside the interval $[a, c]$ if we can find three points $a < b < c$ such that $f(a) > f(b) < f(c)$




```julia
function bracket_minimum(f, x=0; s=1e-2, k=2.0)
    a, ya = x, f(x)
    b, yb = a + s, f(a + s)
    if yb > ya
        a, b = b, a
        ya, yb = yb, ya
        s = -s
    end
    while true
        c, yc = b + s, f(b + s)
        if yc > yb
            return a < c ? (a, c) : (c, a)
        end
        a, ya, b, yb = b, yb, c, yc
        s *= k
    end
end
```

Finding an Initial Bracket

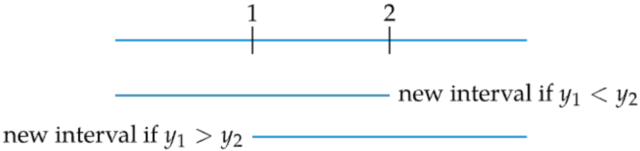
Example of `bracket_minimum` on a function



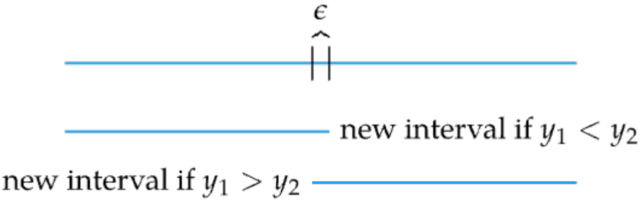
reverses direction between the first and second iteration and expands until a minimum is bracketed in the fourth iteration.

For unimodal functions, when function evaluations are limited, what is the maximal shrinckage we can achieve?

When restricted to only 2 function evaluations (queries) the most we can guarantee to shrink our interval is by just under a factor of 2.

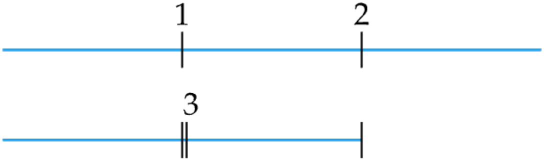


yeilds a factor of 3.



for $\epsilon \to 0$ yields a factor of just less than 2

When restricted to only 3 function evaluations (queries) the most we can guarantee to shrink our interval is by a factor of 3.

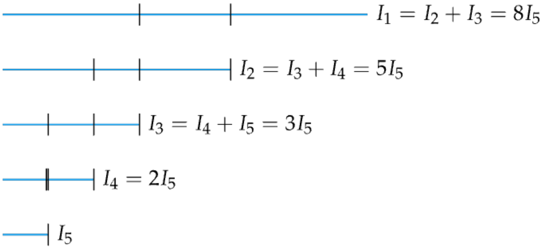


Fibonacci Search

When restricted to n functions evaluations following the previous strategy, we are guaranteed to shrink our interval by a factor of F_{n+1}.

Fibonacci numbers: sum of previous two,
$1, 1, 2, 3, 5, 8, 13, \ldots$

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1, 2 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$



The length of every interval constructed can be expressed in terms of the final interval times a Fibonacci number.
- final, smallest interval has length I_n,
- second smallest interval has length $I_{n-1} = F_3 I_n$
- third smallest interval has length $I_{n-2} = F_4 I_n$,
and so forth.

Fibonacci Search Algorithm

For a unimodal function f in the interval $[a, b]$, we want to shrink the interval within n iterations.
(At each iteration we want to shrink by a factor ϕ).

$$b_{k+1} - a_{k+1} = \frac{F_{n-k+1}}{F_{n-k+2}}(b_k - a_k)$$

Therefore:

$$\begin{aligned} b_n - a_n &= \frac{F_2}{F_3}(b_{n-1} - a_{n-1}) \\ &= \frac{F_2}{F_3}\frac{F_3}{F_4}\dots\frac{F_n}{F_{n+1}}(b_1 - a_1) \\ &= \frac{1}{F_{n+1}}(b_1 - a_1) \end{aligned}$$

Closed-form expression (Binet's formula):

$$F_n = \frac{\phi^n - (1-\phi)^n}{\sqrt{5}},$$

$\phi = (1+\sqrt{5})/2 \approx 1.61803$ is the golden ratio.

$$\frac{F_{n+1}}{F_n} = \phi\frac{1-s^{n+1}}{1-s^n}, \quad s = (1-\sqrt{5})(1+\sqrt{5}) \approx -0.382$$

Suppose we have a unimodal function f in the interval $[a, b]$ and a tolerance $\epsilon = 0.01$. Let $k = 1$.

$$\frac{F_n}{F_{n+1}} = \rho_n = \frac{1 - s^n}{\phi(1 - s^{n+1})} \approx 0.6$$

1. $d_k = a_k + \frac{F_{n-k+1}}{F_{n-k+2}}(b_k - a_k)$

2. if $k \neq n - 1$:

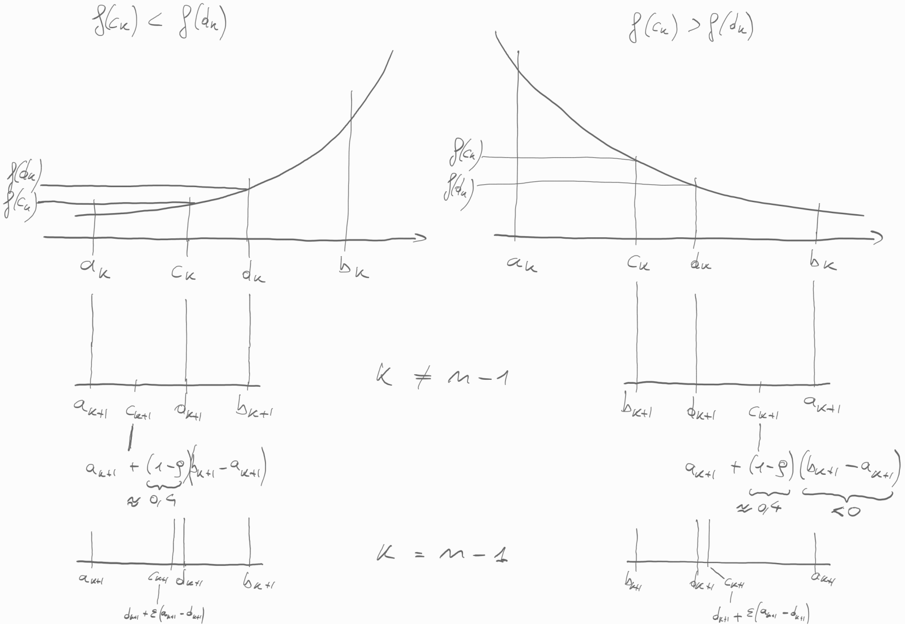
$$c_k = a_k + \left(1 - \frac{F_{n-k+1}}{F_{n-k+2}}\right)(b_k - a_k)$$

   otherwise: $c_k = d_k + \epsilon(a_k - d_k)$

3. if $f(c_k) < f(d_k)$: $\quad b_{k+1} = d_k, \quad d_{k+1} = c_k, \quad a_{k+1} = a_k$
   otherwise: $a_{k+1} = b_k, \quad b_{k+1} = c_k, \quad d_{k+1} = d_k$

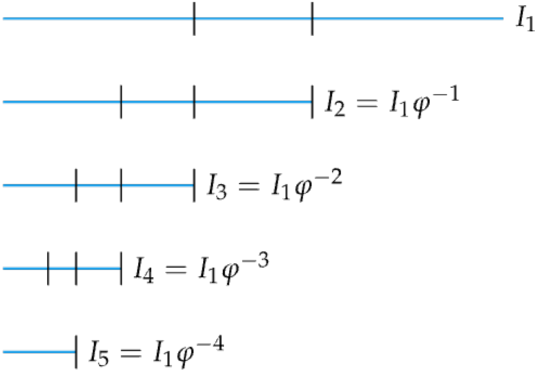
4. $k = k + 1$, if $k = n$ go to step 5, else go to step 2

5. return (a_k, b_k) if $(a_k < b_k)$ else (b_k, a_k)

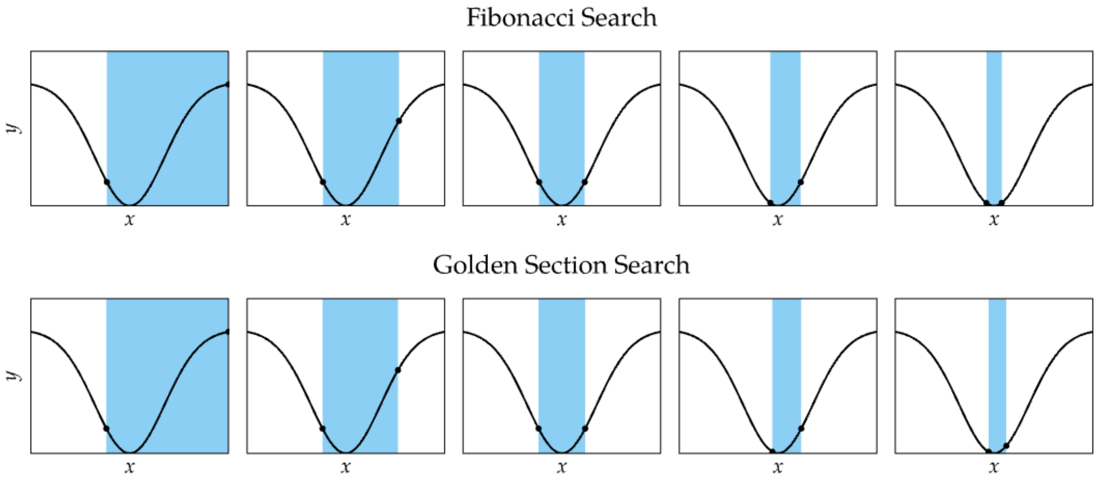

f(c_k) < f(d_k)
f(c_k) > f(d_k)
f(d_k)
f(c_k)
a_k c_k d_k b_k
a_{k+1} c_{k+1} d_{k+1} b_{k+1}
a_{k+1} + (1-ρ)(b_{k+1} - a_{k+1})
≈ 0,4
< 0
k ≠ n - 1
k = n - 1
d_{k+1} + ε(a_{k+1} - d_{k+1})

Golden Section Search

$$\lim_{n\to\infty} \frac{F_{n+1}}{F_n} = \lim_{n\to\infty} \frac{1}{\rho_n} = \lim_{n\to\infty} \phi \frac{1 - s^{n+1}}{1 - s^n} = \phi \approx 1.61803 \qquad \frac{1}{\phi} \approx 0.618$$

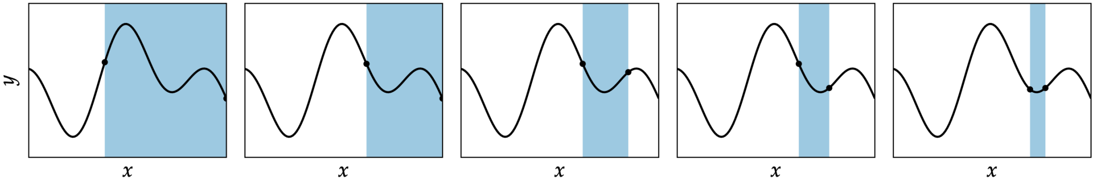


Comparison

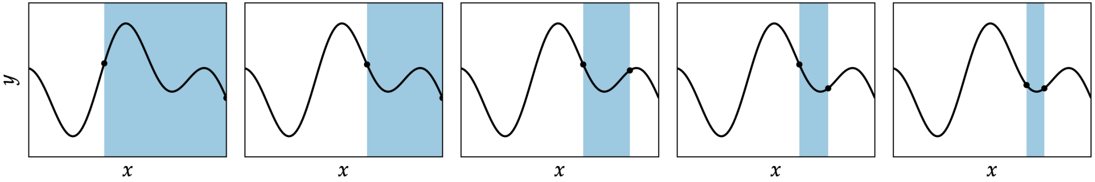


Comparison

Fibonacci Search

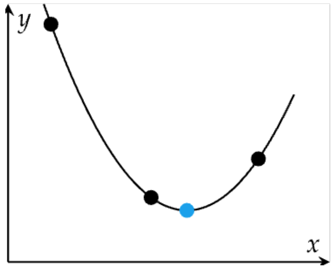


Golden Section Search



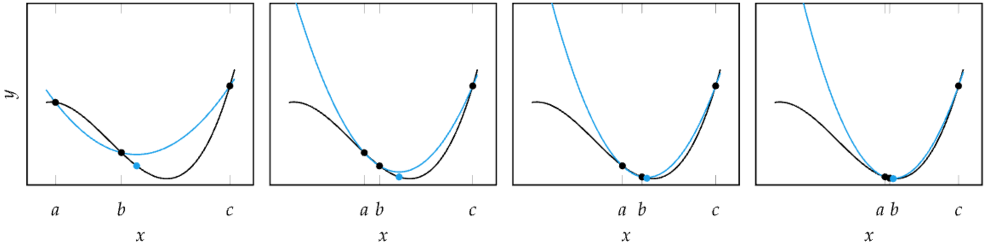
Quadratic Fit Search

- Leverages ability to analytically minimize quadratic functions
- Iteratively fits quadratic function to three bracketing points



Quadratic Fit Search

- If a function is locally nearly quadratic, the minimum can be found after several steps



Maintain a bracketing interval $[a, c]$ with $a < b < c$ and $f(a) > f(b) < f(c)$.

```
if x > b
  if yx > yb
    c, yc = x, yx
  else
    a, ya, b, yb = b, yb, x, yx
```

```
elif x < b
  if yx > yb
    a, ya = x, yx
  else
    c, yc, b, yb = b, yb, x, yx
```

Using Linear Algebra

- We assume that the variable y is related to $x \in \mathbb{R}$ quadratically, so for some constants b_0, b_1, b_2:

  $$y = b_0 + b_1 x + b_2 x^2$$

- Given the set of 3 points $(y_1, x_1,), \ldots, (y_3, x_3)$ in the ideal case, we have that $y_i = b_0 + b_1 x_i + b_2 x_i^2$, for all $i = 1, 2, 3$. In matrix form:

  $$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

  This can be written as $A\boldsymbol{z} = \boldsymbol{y}$ to emphasize that $\boldsymbol{z}$ are our unknowns and A and $\boldsymbol{y}$ are given.

In Python

In polynomial regression, the $m \times (n+1)$ matrix A is called a **Vandermonde matrix** (a matrix with entries $a_{ij} = x_i^{n+1-j}$, $j = 1..n+1$).
NumPy's `np.vander()` is a convenient tool for quickly constructing a Vandermonde matrix, given the values x_i, $i = 1..m$, and the number of desired columns $(n+1)$.

```
>>> print(np.vander([2, 3, 5], 2))
[[2 1]                              # [[2**1, 2**0]
 [3 1]                              #  [3**1, 3**0]
 [5 1]]                             #  [5**1, 5**0]]

>>> print(np.vander([2, 3, 5, 4], 3))
[[ 4  2  1]                         # [[2**2, 2**1, 2**0]
 [ 9  3  1]                         #  [3**2, 3**1, 3**0]
 [25  5  1]                         #  [5**2, 5**1, 5**0]
 [16  4  1]]                        #  [4**2, 4**1, 4**0]
```

In Python

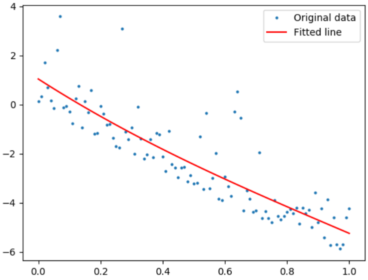
```
A = np.vander(x,4)

coeff = np.linalg.solve(A,y) ## Error!! Why?

B = A.T @ A
z = np.linalg.inv(B) @ A.T @ y

coeff = np.linalg.lstsq(A, y)[0]
np.allclose(z,coeff)

f=np.poly1d(coeff)
plt.plot(x, y, 'o', label='Original data', ↪
    ↪markersize=2)
plt.plot(x, f(x), 'r', label='Fitted line')
plt.legend()
plt.show()
```

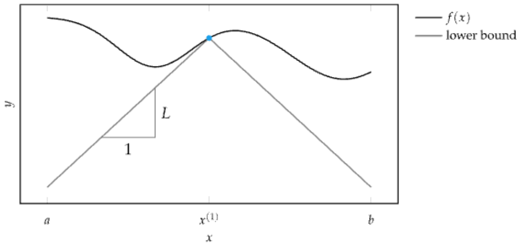


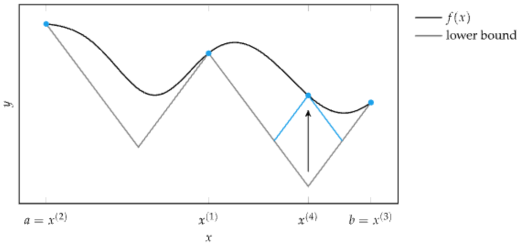
However, for three points the system $Az = y$ is solvable, also in closed form!

Shubert-Piyavskii Method

- The Shubert-Piyavskii method is guaranteed to find the global minimum of any bounded function
- but requires that the function be Lipschitz continuous
- A function is **Lipschitz continuous** if there is an upper bound on the magnitude of its derivative. A function f is Lipschitz continuous on $[a, b]$ if there exists an $\ell > 0$ such that:

$$|f(x) - f(y)| \leq \ell |x - y|, \quad \forall x, y \in [a, b]$$





The Algorithm

Given a valid Lipschitz constant ℓ and an initial interval $[a, b]$:

Initialize:
Sample the midpoint, $x^{(1)} = (a + b)/2$.
Construct a sawtooth lower bound using lines of slope $\pm\ell$ from $x^{(1)}$. These lines will always lie below f if ℓ is a valid Lipschitz constant

Iterate:
Upper vertices in the sawtooth correspond to sampled points. Lower vertices correspond to intersections between the Lipschitz lines originating from each sampled point.
Further iterations find the minimum point in the sawtooth $(x^{(i)}, y^{(i)})$, evaluate the function at that $x^{(i)}$ value, $f(x^{(i)})$, and then use the result to update the sawtooth.

Terminate:

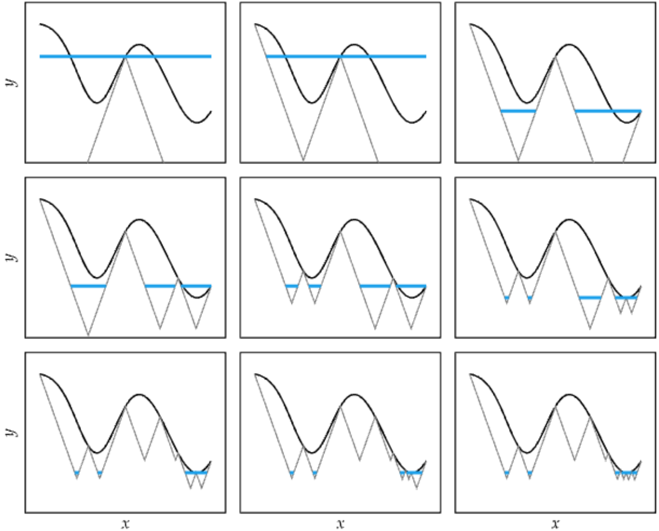
when: $|y^{(i)} - f(x^{(i)})| < \epsilon$.

For every peak, an uncertainty region (blue segments) can be computed according to:

$$\left[x^{(i)} - \frac{1}{\ell}\left(y_{\min} - y^{(i)}\right),\; x^{(i)} + \frac{1}{\ell}\left(y_{\min} - y^{(i)}\right)\right]$$

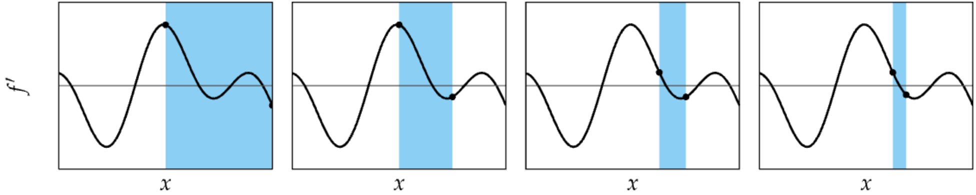
$\left(x^{(i)}, y^{(i)}\right)$ sawtooth lower vertex

$\left(x_{\min}, y_{\min}\right)$ minimum sawtooth upper vertex



Bisection Method

- **Intermediate value theorem**: If f is continuous on $[a, b]$, and there is some $y \in [f(a), f(b)]$, then there exists at least one $x \in [a, b]$, such that $f(x) = y$.
- Used in root-finding methods
- When applied to $f'(x)$, can be used to find minimum of f
- if $\text{sign}(f'(a)) \neq \text{sign}(f'(b))$, or equivalently, $f'(a)f'(b) \leq 0$ then $[a, b]$ is guaranteed to contain a zero.



Bisection Method

- Cut the bracketed region $[a, b]$ in half with every iteration
- Evaluate the midpoint $(a + b)/2$
- form a new bracket from the midpoint and whichever side that continues to bracket a zero.
- Terminate after a fixed number of iterations.
- Guaranteed to converge within ϵ of x^* within $\lg_2(|b - a|/\epsilon)$

Summary

- Many optimization methods shrink a bracketing interval, including Fibonacci search, golden section search, and quadratic fit search
- The Shubert-Piyavskii method outputs a set of bracketed intervals containing the global minima, given the Lipschitz constant
- Root-finding methods like the bisection method can be used to find where the derivative of a function is zero

4. Local Descent

Preface

For multivariate functions, we have argued that:

- derivatives can have exponential growth in the resulting analytical expression
- calculating zeros might be challenging

Hence, minimizing by solving $\nabla f(\boldsymbol{x}) = 0$ may be computationally demanding.

Outline

Descent Direction Iteration

Descent Direction Methods use a local model to incrementally improve design point until some convergence criteria is met.

1. Check termination conditions at $\boldsymbol{x}_k$; if not met, continue.
2. Decide **descent direction** $\boldsymbol{d}_k$ using local information, commonly required that $\boldsymbol{d}_k^T \nabla f(\boldsymbol{x}_k) < 0$.
3. Decide **step size** (ie, magnitude of the overall step that depends on α_k, sometimes but not always $\|\boldsymbol{d}_k\|_2 = 1$)
4. Compute next design point $\boldsymbol{x}_{k+1}$

$$\boxed{\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k}$$

Descent Direction

The search direction often has the form

$$\boldsymbol{d}_k = -B_k^{-1}\nabla f(\boldsymbol{x}_k)$$

where B_k is a symmetric and nonsingular matrix.

- in the steepest descent method, B_k is the identity matrix I
- in Newton's method, B_k is the exact Hessian $\nabla^2 f(\boldsymbol{x}_k)$.
- in quasi-Newton methods, B_k is an approximation to the Hessian that is updated at every iteration by means of a low-rank formula.

When $\boldsymbol{d}_k = -B_k^{-1}\nabla f(\boldsymbol{x}_k)$ and B_k is positive definite, we have

$$\boldsymbol{d}_k^T\nabla f(\boldsymbol{x}_k) = -\nabla f(\boldsymbol{x}_k)^T B_k^{-1}\nabla f(\boldsymbol{x}_k) < 0$$

and therefore $\boldsymbol{d}_k$ is a descent direction. In fact, it is a double implication!

Outline

- We discuss how to choose α_k and $\boldsymbol{d}_k$ to promote convergence from remote starting points.
- We also consider the rate of convergence of steepest descent, quasi-Newton, and Newton methods.

Line Search for Step Size

Assuming we have the search direction:

- Use it to compute α
- Using the techniques discussed in the previous class, find the minimum of a univariate function:

$$\phi(\alpha) = f(\boldsymbol{x} + \alpha \boldsymbol{d}) \qquad \text{minimize}_{\alpha \geq 0}\ \phi(\alpha)$$

We assume that $\boldsymbol{p}_k$ is a descent direction, that is, $\phi'(0) < 0$, so that our search can be confined to positive values of α.

```
def line_search(f, x, d)
  objective = lambda alpha: f(x + alpha * d)
  a, b = bracket_minimum(objective)
  alpha = minimize(objective, a, b)
  return x + alpha * d
```

Often computationally costly, so approximate line search is used instead

Line Search: Alternatives

α is called to the **learning rate** or **step factor**:

Equal to the **step size** only when $\|\boldsymbol{d}_k\|_2 = 1$.

- Fixed learning rate
- **Decaying step factor**

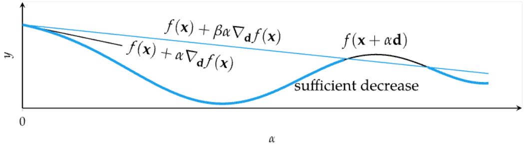
$$\alpha_k = \alpha_1 \gamma^{k-1} \qquad \text{for } \gamma \in [0, 1]$$

Decaying step factor is often required in convergence proofs

Approximate Line Search

- If function calls are expensive, rather than finding the minimum along a search direction, find a point of sufficient decrease

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) + \beta\alpha\nabla_{\mathbf{d}_k} f(\mathbf{x}_k)$$

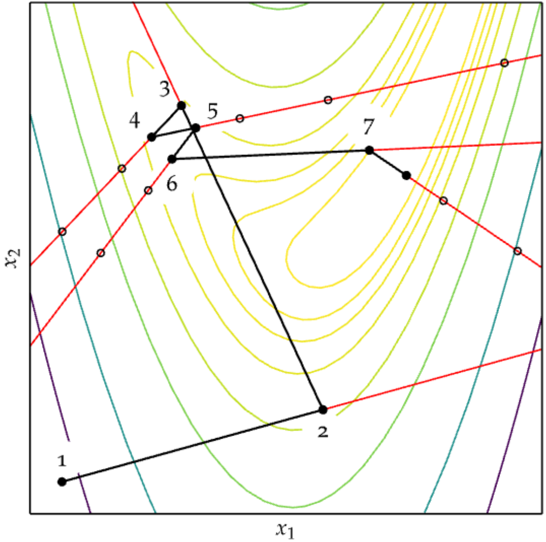


- $\beta \in [0, 1]$, usually $\beta = 1 \times 10^{-4}$
- Alone this condition is insufficient to guarantee convergence to a local minimum. It can converge prematurely.
- Backtracking line search starts with a large step and then backs off

```
def backtracking_line_search(f, grad, x, d, alpha_0=1, p=0.5, beta=1e-4):
    y, g, alpha = f(x), grad(x), alpha_0
    while ( f(x + alpha * d) > y + beta * alpha * np.dot(g, d) ) :
        alpha *= p
    return alpha
```

- Guaranteed to converge to a local minimum, but can be slow.

Approximate Line Search: Example



Approximate Line Search

Building on backtracking line search are the **Wolfe Conditions** together sufficient to guarantee convergence to a local minimum.

1. First Wolfe Condition: Sufficient Decrease (Armijo condition)

$$f(\boldsymbol{x}_{k+1}) \leq f(\boldsymbol{x}_k) + \beta\alpha\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)$$

2. Second Wolfe Condition: Curvature Condition

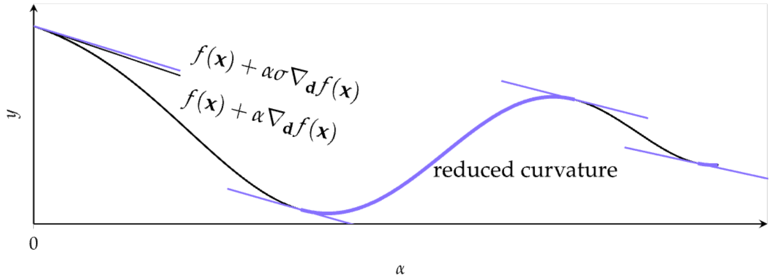
$$\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1}) \geq \sigma\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)$$

$\beta < \sigma < 1$ with
- $\sigma = 0.1$ with conjugate gradient method
- $\sigma = 0.9$ with Newton method

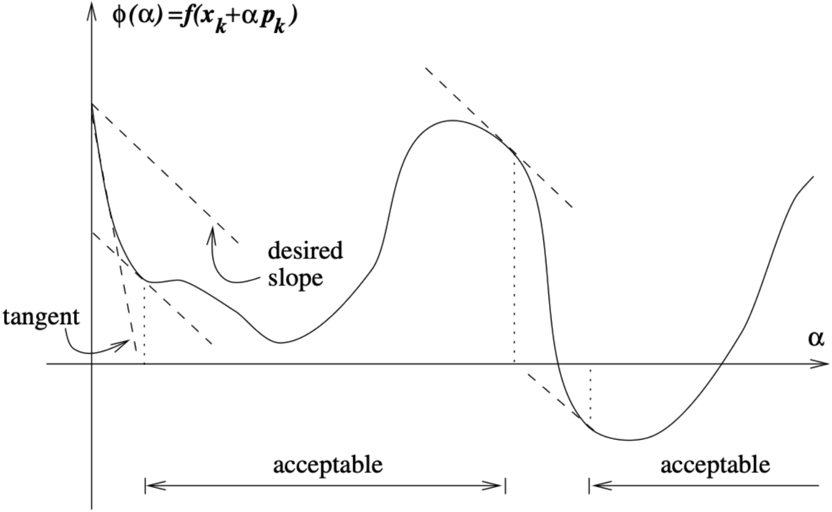
Curvature Condition

Regions satisfying the curvature condition $\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1}) \geq \sigma \nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)$

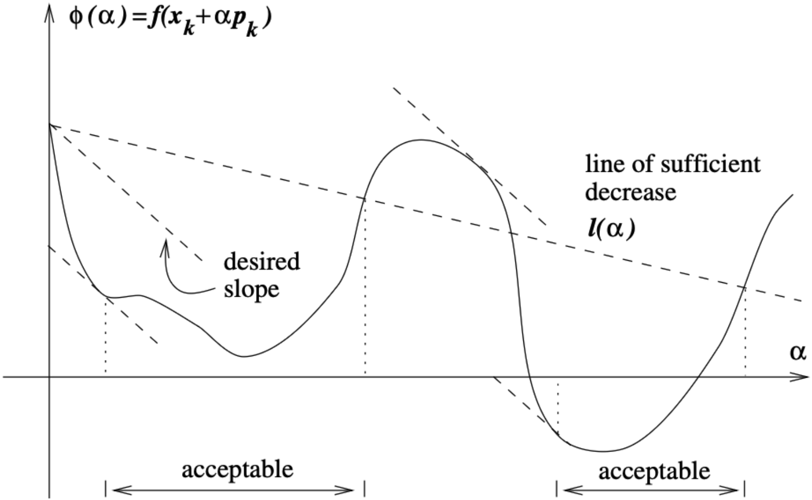
- Consider the univariate function $\phi(\alpha) = f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$.
- The left-hand-side is simply the derivative $\phi'(\alpha_k)$, so the curvature condition ensures that the slope of ϕ at α_k is greater than σ times the initial slope $\phi'(0)$.
- If the slope $\phi'(0)$ is negative, then we are looking for a step size α_k such that the slope of ϕ at that point is still negative but not too negative.
- If $\phi'(\alpha_k)$ is only slightly negative or even positive, it is a sign that we cannot expect much more decrease in f in this direction, so it makes sense to terminate the line search.



Curvature Condition



Wolfe Conditions



Approximate Line Search: Example

Consider approximate line search on $f(x_1, x_2) = x_1^2 + x_1 x_2 + x_2^2$
from $\boldsymbol{x} = [1, 2]$ in the direction $\boldsymbol{d} = [-1, -1]$, gradient at $\boldsymbol{x}$ is $\boldsymbol{g} = [4, 5]$
using a maximum step size of 10, a reduction factor of 0.5,
first Wolfe condition parameter $\beta = 1 \times 10^{-4}$, second Wolfe condition parameter $\sigma = 0.9$.

First Wolfe condition ($f(\boldsymbol{x} + \alpha \boldsymbol{d}) \leq f(\boldsymbol{x}) + \beta\alpha(\boldsymbol{g}^T \cdot \boldsymbol{d})$):

$$\begin{aligned}
\alpha = 10 : \quad & f([1,2] + 10 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} 10 [4,5]^T [-1,-1] \implies 217 \not\leq 6.991 \\
\alpha = 10 \cdot 0.5 = 5 : \quad & f([1,2] + 5 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} 5 [4,5]^T [-1,-1] \implies 37 \not\leq 6.996 \\
\alpha = 2.5 : \quad & f([1,2] + 2.5 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} 2.5 [4,5]^T [-1,-1] \implies 3.25 \leq 6.998
\end{aligned}$$

The candidate design point $\boldsymbol{x}' = \boldsymbol{x} + \alpha \boldsymbol{d} = [-1.5, -.0.5]$ is checked against the second Wolfe condition $\nabla_{\boldsymbol{d}} f(\boldsymbol{x}') \geq \sigma \nabla_{\boldsymbol{d}} f(\boldsymbol{x})$:

$$[-3.5, -2.5] \cdot [-1, -1] \geq \sigma [4, 5] \cdot [-1, -1] \implies 6 \geq -8.1$$

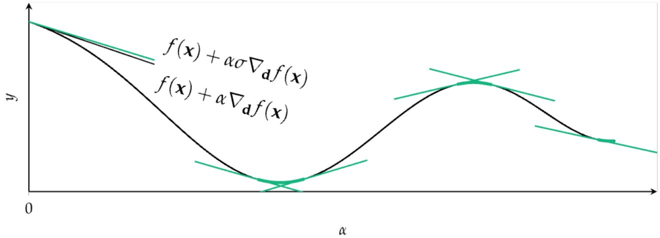
Approximate line search terminates with $\boldsymbol{x} = [-1.5, -0.5]$.

Strong Wolfe Conditions

A step length may satisfy the Wolfe conditions without being particularly close to a minimizer of $f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$.
We can modify the curvature condition to force $f(\boldsymbol{x}_{k+1})$ to exclude points that are far from stationary points of $f(\boldsymbol{x}_{k+1})$, ie, we no longer allow the gradient $\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1})$ to be too positive.

Strong Wolf conditions:

$$f(\boldsymbol{x}_{k+1}) \leq f(\boldsymbol{x}_k) + \beta\alpha\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k) \qquad \text{and} \qquad |\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1})| \leq \sigma |\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)|$$



Approximate Line Search Goal

Given:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$$

with descent direction:

$$\nabla f(\boldsymbol{x}_k)^T \boldsymbol{d}_k < 0$$

Define:

$$\phi(\alpha) = f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k), \qquad \phi'(\alpha) = \nabla f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)^T \boldsymbol{d}_k$$

Goal:

Find $\alpha > 0$ satisfying the **Strong Wolfe Conditions**.

Strong Wolfe Conditions

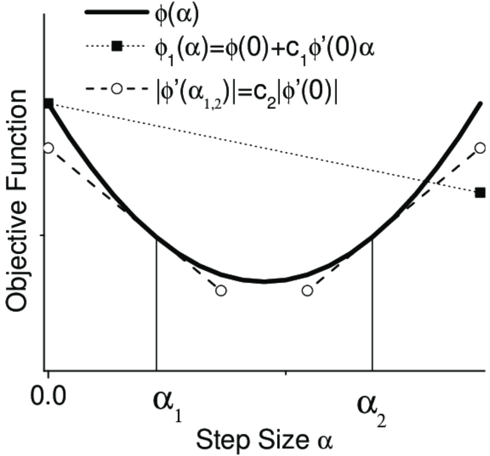
Two requirements:

Sufficient decrease (Armijo)

$$\phi(\alpha) \leq \phi(0) + \beta\alpha\phi'(0)$$

Strong curvature condition

$$|\phi'(\alpha)| \leq \sigma|\phi'(0)|$$



Ensures:

- sufficient decrease
- step near minimum

Strong Backtracking: Idea

Two phases:

1. Bracketing phase

- Start with $[\alpha_0 = 0, \alpha_1]$
- Increase step
- Find interval containing solution

2. Zoom phase

- Shrink interval
- Use bisection or interpolation
- Find Wolfe step

Idea:

Bracket minimum $\rightarrow$ Zoom to solution

Algorithm

Bracketing

- Start with $[\alpha_0 = 0, \alpha_1]$
- If Armijo fails or slope changes sign

  $\rightarrow$ Zoom

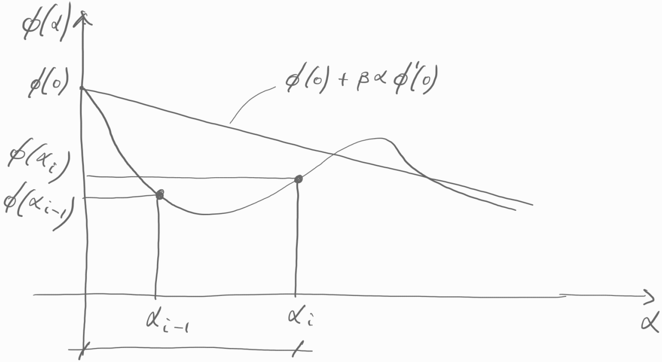
- If strong Wolfe satisfied

  $\rightarrow$ Stop

- Else increase step

Input : α_{max} maximum step allowed
Output: α^* set to a step length that satisfies the strong Wolfe condition

Initialize $k \leftarrow 0$;
Set $\alpha_0 \leftarrow 0$, choose $\alpha_{max} > 0$ and $\alpha_1 \in (0, \alpha_{max})$;
$i \leftarrow 1$;
while True **do**
  Evaluate $\phi(\alpha_i)$;
  if $\phi(\alpha_i) > \phi(0) + \beta\alpha_i\phi'(0)$ or $[\phi(\alpha_i) \geq \phi(\alpha_{i-1})$ and $i > 1]$ **then** // Armijo or next slide
    $\alpha^* \leftarrow \text{zoom}(\alpha_{i-1}, \alpha_i)$ and break;
  Evaluate $\phi'(\alpha_i)$;
  if $|\phi'(\alpha_i)| \leq -\sigma\phi'(0)$ **then**
    set $\alpha^* \leftarrow \alpha_i$ and break;
  if $\phi'(\alpha_i) \geq 0$ **then** // slope changes
    set $\alpha^* \leftarrow \text{zoom}(\alpha_i, \alpha_{i-1})$ and break;
  Choose $\alpha_{i+1} \in (\alpha_i, \alpha_{max})$; // Eg, $\alpha_{i+1} \leftarrow 2\alpha_i$
  $i \leftarrow i + 1$;


$\phi(\alpha)$
$\phi(0)$
$\phi(0) + \beta\alpha\phi'(0)$
$\phi(\alpha_i)$
$\phi(\alpha_{i-1})$
α_{i-1}
α_i
α

Algorithm

Zoom

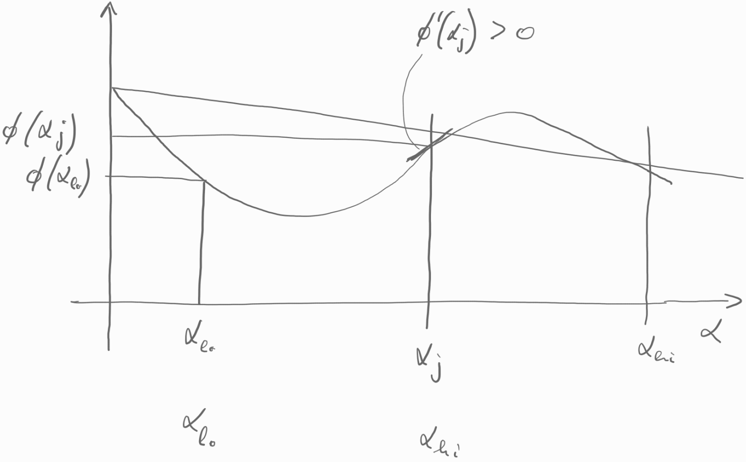
- Interpolate inside bracket
- Shrink interval
- Stop when Wolfe satisfied

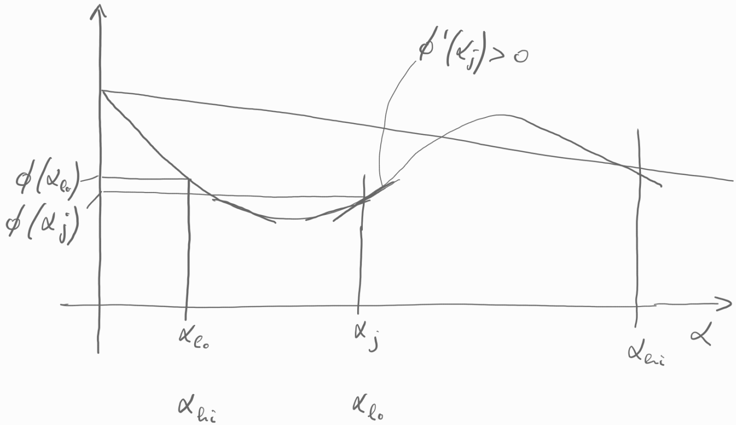
Iterate generating an $\alpha_j \in [\alpha_{lo}, \alpha_{hi}]$, and then replacing one of these endpoints by α_j maintaining the invariants:

1. $[\alpha_{lo}, \alpha_{hi}]$ satisfy the strong Wolfe conditions;
2. α_{lo} smallest function value $\phi(\alpha_{lo})$;
3. α_{hi} chosen so that $\phi'(\alpha_{lo})(\alpha_{hi} - \alpha_{lo}) < 0$.

Input : α_{lo}, α_{hi}
Output: α^* set to a step length that satisfies the strong Wolfe condition

```
while True do
    Interpolate (using quadratic, cubic or bisection) to find a trial step
     length α_j between α_lo and α_hi ;
    Evaluate φ(α_j);
    if φ(α_j) > φ(0) + βα_j φ'(0) or φ(α_j) ≥ φ(α_lo) then // next slide
        α_hi ← α_j;
    else
        Evaluate φ'(α_j);
        if |φ'(α_j)| ≤ −σφ'(0) then
            Set α* ← α_j and break;
        if φ'(α_j)(α_hi − α_lo) ≥ 0 then // violates 3., second next slide
            α_hi ← α_lo;
        α_lo ← α_j;
```


$\phi'(\alpha_j) > 0$
$\phi(\alpha_j)$
$\phi(\alpha_{lo})$
α_{lo}
α_j
α_{hi}
α
α_{lo}
α_{hi}



$\phi'(\alpha_j) > 0$
$\phi(\alpha_{lo})$
$\phi(\alpha_j)$
α_{lo}
α_j
α_{hi}
α
α_{hi}
α_{lo}

Approximate Line Search: Python Implementation

```python
def strong_backtracking(f, nabla, x: np.array, d: np.array, alpha: float=1,
  beta: float=1e-4, sigma: float=0.1) -> float:
  y0, g0, y_prev, alpha_prev = f(x), nabla(x) @ d, np.nan, 0
  alpha_lo, alpha_hi = np.nan, np.nan
```

```python
  # bracket phase
  while True:
    y = f(x + alpha*d)
    if y > y0 + beta*alpha*g0 or (not math.isnan(↪
        ↪y_prev) and y >= y_prev):
      alpha_lo, alpha_hi = alpha_prev, alpha
      break
    g = nabla(x + alpha*d) @ d
    if abs(g) <= -sigma * g0:
      return alpha
    elif g <= 0:
      alpha_lo, alpha_hi = alpha, alpha_prev
      break
    y_prev, alpha_prev, alpha = y, alpha, 2 * alpha
```

```python
  # zoom phase
  ylo = f(x + alpha_lo*d)
  while abs(alpha_hi - alpha_lo) > 1e-6:
    print(alpha_lo,alpha_hi)
    alpha = (alpha_lo + alpha_hi)/2
    y = f(x + alpha * d)
    if y > y0 + beta * alpha * g0 or y >=↪
        ↪ ylo:
      alpha_hi = alpha
    else:
      g = nabla(x + alpha * d) @ d
      if abs(g) <= -sigma * g0:
          return alpha
      elif g*(alpha_hi -alpha_lo) >= 0:
          alpha_hi = alpha_lo
      alpha_lo = alpha
```

Why Strong Backtracking?

Used in:

- Gradient descent
- Quasi-Newton: BFGS, L-BFGS
- Nonlinear Conjugate Gradient

Guarantees:

- global convergence
- stable steps
- fast convergence of quasi-Newton

Standard method in modern optimization.

Note: it is an algorithm that makes use of derivative information!

Outline

Convergence Analysis

Let $\{\boldsymbol{x}_k\}$ be a sequence of points belonging in $\mathbb{R}^n$.
We say that a sequence $\{\boldsymbol{x}_k\}$ **converges** to some point $\boldsymbol{x}$, written $\lim_{k\to\infty} \boldsymbol{x}_k = x$, if for any $\epsilon > 0$, there is an index K such that

$$\|\boldsymbol{x}_k - \boldsymbol{x}\| \leq \epsilon \qquad \text{for all } k \geq K$$

For example, the sequence $\boldsymbol{x}_k$ defined by $\boldsymbol{x}_k = (1 - 2^{-k}, 1/k^2)^T$ converges to $(1, 0)^T$.

Convergence of Line Search

Let θ_k be the angle between $\boldsymbol{d}_k$ and the steepest descent direction $-\nabla f_k$, defined by:

$$\cos\theta_k = \frac{-\nabla f_k^T \boldsymbol{d}_k}{\|\nabla f_k\| \, \|\boldsymbol{d}_k\|}$$

Theorem (Zoutendijk condition)

Consider any iteration of the form $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$, where $\boldsymbol{d}_k$ is a descent direction and α_k satisfies the Wolfe conditions. Suppose that f is bounded below in $\mathbb{R}^n$ and that f is continuously differentiable in an open set N containing the level set $\mathcal{L} = \{x : f(x) \leq f(x_0)\}$, where $\boldsymbol{x}_0$ is the starting point of the iteration. Assume also that the gradient ∇f is Lipschitz continuous on N, that is, there exists a constant $L > 0$ such that:

$$|f(x) - f(y)| \leq L|x - y|, \quad \forall x, y \in N$$

Then:

$$\sum_{k \geq 0} \cos^2\theta_k \, \|\nabla f_k\|^2 < \infty.$$

The Zoutendijk condition implies that

$$\cos^2\theta_k \|\nabla f_k\|^2 \to 0$$

We can be sure that the gradient norms $\|\nabla f_k\|$ converge to zero, provided that the search directions are never too close to orthogonality with the gradient.

It is a **global convergence** result.
Here the strongest possible result: we cannot guarantee that the method converges to a minimizer, but only that it is attracted by stationary points (only introducing the Hessian we can prove convergence to a local minimum — see next slide)

In

$$\boldsymbol{d}_k = -B_k^{-1}\nabla f(\boldsymbol{x}_k)$$

assume that the matrices B_k are **positive definite** with a uniformly bounded condition number. That is, there is a constant M such that $\|B_k\| \left\|B_k^{-1}\right\| \leq M$ for all k.

Then from the definition of θ_k:

$$\cos\theta_k \geq 1/M$$

By combining this bound with $\cos^2\theta_k \left\|\nabla f_k\right\|^2 \to 0$ we find that

$$\lim_{k\to\infty} \|\nabla f_k\| = 0$$

So, globally convergent under the positive definitness assumptions on B_k, (which is needed to ensure that $\boldsymbol{d}_k$ is a descent direction), and if the step lengths satisfy the Wolfe conditions.

Rate of Convergence

Let $\{\boldsymbol{x}_k\}$ be a sequence in $\mathbb{R}^n$ that converges to $\boldsymbol{x}^*$.
The convergence is said to be **Q-linear** (quotient-linear) if there is a constant $r \in (0, 1)$ such that

$$\boxed{\frac{\|\boldsymbol{x}_{k+1} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}_k - \boldsymbol{x}^*\|} \leq r \qquad \text{for all } k \text{ sufficiently large}}$$

ie, the distance to the solution $\boldsymbol{x}^*$ decreases at each iteration by at least a constant factor bounded away from 1 (ie, < 1).

Example:
sequence $\{1 + (0.5)^k\}$ converges Q-linearly to 1, with rate $r = 0.5$.

Rate of Convergence

The convergence is said to be **Q-superlinear** if

$$\boxed{\lim_{k\to\infty} \frac{\|\boldsymbol{x}_{k+1} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}_k - \boldsymbol{x}^*\|} = 0}$$

Example: the sequence $\{1 + k^{-k}\}$ converges superlinearly to 1.

An even more rapid convergence rate: The convergence is said to be **Q-quadratic** if

$$\boxed{\frac{\|\boldsymbol{x}_{k+1} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}_k - \boldsymbol{x}^*\|^2} \le M \qquad \text{for all } k \text{ sufficiently large}}$$

where M is a positive constant, not necessarily less than 1. Example: the sequence $\{1 + (0.5)^{2^k}\}$. The values of r and M depend not only on the algorithm but also on the properties of the particular problem. Regardless of these values a quadratically convergent sequence will always eventually converge faster than a linearly convergent sequence.

Rate of Convergence

Superlinear convergence (quadratic, cubic, quartic, etc) is regarded as fast and desirable, while sublinear convergence is usually impractical.

- Steepest descent algorithms converge only at a Q-linear rate, and when the problem is ill-conditioned the convergence constant r is close to 1.
- Quasi-Newton methods for unconstrained optimization typically converge Q-superlinearly
- Newton's method converges Q-quadratically under appropriate assumptions.

Rate of Convergence

A slightly weaker form of convergence:
overall rate of decrease in the error, rather than the decrease over each individual step of the algorithm.
We say that convergence is **R-linear** (root-linear) if there is a sequence of nonnegative scalars $\{v_k\}$ such that

$$\|\boldsymbol{x}_k - \boldsymbol{x}^*\| \leq \{v_k\} \text{ for all } k \text{, and } \{v_k\} \text{ converges Q-linearly to zero.}$$

Outline

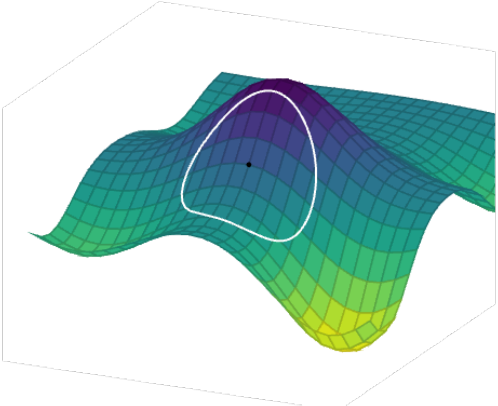
Trust Region Methods

- Descent methods can place too much trust in their first and second order information
- A **trust region** is the local area of the design space where the local model is believed to be reliable.
- Trust region methods, or restricted step methods, limit the step size to ensure local approximation error is minimized
- If the improvement matches the predicted value, the trust region is expanded; otherwise it is contracted

Trust Region Methods

- $\boldsymbol{x}'$ is new design point
- $\hat{f}(\boldsymbol{x}')$ is local function approximation, eg, second-order Taylor approximation
- δ is trust region radius

$$\begin{aligned} &\text{minimize}_{\boldsymbol{x}'} \hat{f}(\boldsymbol{x}') \\ &\text{subject to} \quad \|\boldsymbol{x} - \boldsymbol{x}'\| \leq \delta \end{aligned}$$



Constrained optimization problem.
It can be solved efficiently if $\hat{f}$ quadratic

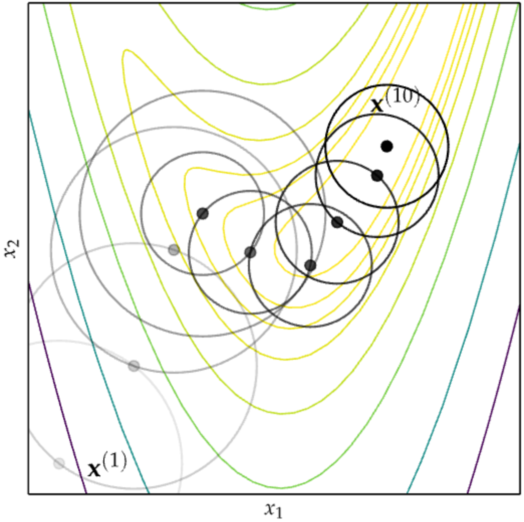
Trust Region Methods

δ can be expanded or contracted based on performance

$$\eta = \frac{\text{actual improvement}}{\text{predicted improvement}} = \frac{f(\boldsymbol{x}) - f(\boldsymbol{x}')}{f(\boldsymbol{x}) - \hat{f}(\boldsymbol{x}')}$$

If $\eta < \eta_1$ contract by a factor $\gamma_k < 1$
if $\eta > \eta_2$ expand by a factor $\gamma_k > 1$

Trust Region Methods: Example



Trust regions can be also non circular.

Termination Conditions

For all iterative descent methods and commonly used together:

- Maximum Iterations: $k > k_{\text{max}}$
- Absolute Improvement: $f(\boldsymbol{x}_k) - f(\boldsymbol{x}_{k+1}) < \epsilon_a$
- Relative Improvement: $f(\boldsymbol{x}_k) - f(\boldsymbol{x}_{k+1}) < \epsilon_r |f(\boldsymbol{x}_k)|$
- Gradient Magnitude: $\|\nabla f(\boldsymbol{x}_{k+1})\| < \epsilon_g$

Then random restart.

Summary

- Descent direction methods incrementally descend toward a local optimum.
- Univariate optimization can be applied during line search.
- Approximate line search can be used to identify appropriate descent step sizes.
- Trust region methods constrain the step to lie within a local region that expands or contracts based on predictive accuracy.
- Termination conditions for descent methods can be based on criteria such as the change in the objective function value or magnitude of the gradient.

5. First-Order Methods

Descent Direction Methods

How to select the descent direction?

- first-order methods that rely on gradient

- second-order methods that rely on Hessian information

Advantages of first order methods:

- cheap iterations: good for small and large scale optimization
- helpful because easy to warm restart

Limitations of first order methods:

- not hard to find challenging instances for them (instances with many saddle regions).
- can converge slowly.

Outline

Gradient Descent

The **steepest descent** direction at $\boldsymbol{x}_k$, at kth iteration of a **local descent iterative method**, is the one opposite to the gradient (**gradient descent**):

$$\boxed{\boldsymbol{d}_k = -\frac{\nabla f(\boldsymbol{x}_k)}{\|\nabla f(\boldsymbol{x}_k)\|}}$$

Guaranteed to lead to improvement if:

- f is smooth
- step size is sufficiently small
- $\boldsymbol{x}_k$ is not a stationary point (ie, $\nabla f(\boldsymbol{x}_k) = 0$)

Gradient Descent: Example

- Suppose we have

$$f(\boldsymbol{x}) = x_1 x_2^2$$

- The gradient is $\nabla f = [x_2^2, 2x_1 x_2]$
- $\boldsymbol{x}_k = [1, 2]$

$$\boldsymbol{d}_{k+1} = -\frac{\nabla f(\boldsymbol{x}_k)}{\|\nabla f(\boldsymbol{x}_k)\|} = \frac{[-4, -4]}{\sqrt{16+16}} = \left[-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right]$$

Implementation

```python
class DescentMethod:
    alpha: float

class GradientDescent(DescentMethod):
    def __init__(self, f, grad, x, alpha):
        self.alpha = alpha

    def step(self, f, grad, x):
        alpha, g = self.alpha, grad(x)
        return x - alpha * g
```

Gradient Descent

Theorem: The next direction is orthogonal to the current direction.

Proof:

$$\alpha_k^* = \underset{\alpha}{\operatorname{argmin}}\, f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$$

$$\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k) = \nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k) = 0 \qquad \text{because } \alpha_k^* \text{ is minimum}$$

$$\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)^T \boldsymbol{d}_k = 0 \qquad \text{because directional derivative: } \nabla_{\boldsymbol{s}} f(\boldsymbol{x}) = \nabla f(\boldsymbol{x})^T \boldsymbol{s}$$

$$\boldsymbol{d}_{k+1} = -\frac{\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)}{\left\|\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)\right\|} \qquad \text{gradient descent}$$

$$\boldsymbol{d}_{k+1} \cdot \boldsymbol{d}_k = -\frac{\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)}{\left\|\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)\right\|} \cdot \boldsymbol{d}_k = 0 \qquad \boldsymbol{d}_{k+1}^T \boldsymbol{d}_k = 0 \implies \boldsymbol{d}_{k+1} \perp \boldsymbol{d}_k$$

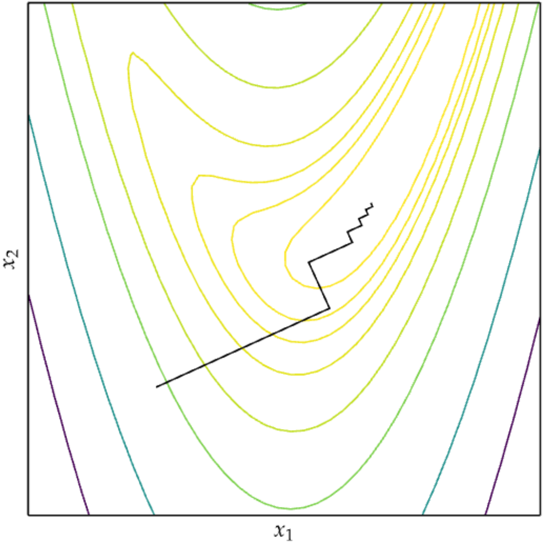
$\square$

Gradient Descent: Example

2D Rosenbrock function

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

Narrow valleys not aligned with gradient can be a problem



Outline

Conjugate Gradient

[Hestenes and Stiefel, 1950s]

For A symmetric positive definite:

$$A\boldsymbol{x} = \boldsymbol{b} \iff \underset{\boldsymbol{x}}{\text{minimize}}\, f(\boldsymbol{x}) \stackrel{def}{=} \frac{1}{2}\boldsymbol{x}^T A\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{x}$$

$$\nabla f(\boldsymbol{x}) = A\boldsymbol{x} - \boldsymbol{b} \stackrel{def}{=} \boldsymbol{r}(\boldsymbol{x})$$

Conjugate Direction

Def.: A set of nonzero vectors $\{\boldsymbol{d}_0, \boldsymbol{d}_1, \ldots, \boldsymbol{d}_\ell\}$ is said to be **conjugate** with respect to the symmetric positive definite matrix A if

$$\boldsymbol{d}_i^T A \boldsymbol{d}_j = 0, \qquad \text{for all } i \neq j$$

(the vectors are linearly independent. Generally, not orthogonal.)

Theorem: Given an arbitrary $\boldsymbol{x}_0 \in \mathbb{R}^n$ and a set of conjugate vectors $\{\boldsymbol{d}_0, \boldsymbol{d}_1, \ldots, \boldsymbol{d}_{n-1}\}$ the sequence $\{\boldsymbol{x}_k\}$ generated by

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$$

where α_k is the analytical solution of $\min_\alpha f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$ given by:

$$\alpha_k = -\frac{\boldsymbol{r}_k^T \boldsymbol{d}_k}{\boldsymbol{d}_k^T A \boldsymbol{d}_k}$$

(aka, **conjugate direction algorithm**) converges to the solution $\boldsymbol{x}^*$ of the linear system and minimization problem in at most n steps.

Proof:

$$\min_{\alpha} f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$$

We can compute the derivative with respect to α:

$$\begin{aligned}
\frac{\partial}{\partial \alpha} f(\boldsymbol{x} + \alpha \boldsymbol{d}) &= \frac{\partial}{\partial \alpha} (\boldsymbol{x} + \alpha \boldsymbol{d})^T A (\boldsymbol{x} + \alpha \boldsymbol{d}) - \boldsymbol{b}^T (\boldsymbol{x} + \alpha \boldsymbol{d}) \\
&= \boldsymbol{d}^T A (\boldsymbol{x} + \alpha \boldsymbol{d}) - \boldsymbol{d}^T \boldsymbol{b} \\
&= \boldsymbol{d}^T (A\boldsymbol{x} - \boldsymbol{b}) + \alpha \boldsymbol{d}^T A \boldsymbol{d}
\end{aligned}$$

Setting $\frac{\partial f(\boldsymbol{x} + \alpha \boldsymbol{d})}{\partial \alpha} = 0$ results in:

$$\alpha_k = -\frac{\boldsymbol{d}_k^T (A\boldsymbol{x}_k - \boldsymbol{b})}{\boldsymbol{d}_k^T A \boldsymbol{d}_k} = -\frac{\boldsymbol{d}_k^T \boldsymbol{r}(\boldsymbol{x}_k)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k} \tag{1}$$

- Since the directions $\{\boldsymbol{d}_k\}$ are linearly independent, they must span the whole space $\mathbb{R}^n$. Hence, there is a set of scalars σ_k such that:

$$\boldsymbol{x}^* - \boldsymbol{x}_0 = \sigma_0 \boldsymbol{d}_0 + \sigma_1 \boldsymbol{d}_1 + \ldots + \sigma_{n-1} \boldsymbol{d}_{n-1} \qquad \text{(the “in at most } n \text{ steps” part)}$$

- By premultiplying this expression by $\boldsymbol{d}_k^T A$ and using the conjugacy property, we obtain:

$$\sigma_k = \frac{\boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_0)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k} \tag{2}$$

- If $\boldsymbol{x}_k$ is generated by conjugate direction algorithm, then we have

$$\boldsymbol{x}_k = \boldsymbol{x}_0 + \alpha_0 \boldsymbol{d}_0 + \alpha_1 \boldsymbol{d}_1 + \ldots + \alpha_k \boldsymbol{d}_{k-1}$$

- By premultiplying this expression by $\boldsymbol{d}_k^T A$ and using the conjugacy property, we have that

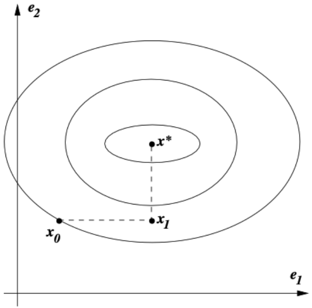
$$\boldsymbol{d}_k^T A(\boldsymbol{x}_k - \boldsymbol{x}_0) = 0$$

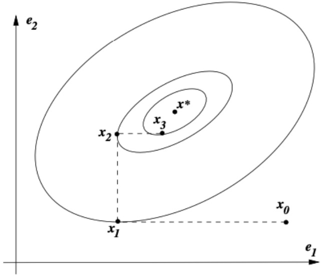
- and therefore

$$\begin{aligned}\boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_0) = \boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_k + \boldsymbol{x}_k - \boldsymbol{x}_0) = \boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_k) + \boldsymbol{d}_k^T A(\boldsymbol{x}_k - \boldsymbol{x}_0) = \\ = \boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_k) = \boldsymbol{d}_k^T (\boldsymbol{b} - A\boldsymbol{x}_k) = -\boldsymbol{d}_k^T \boldsymbol{r}_k.\end{aligned}$$

- Using this result in (2) and comparing with (1) we conclude $\alpha_k = \sigma_k$. □

Interpretation of Conjugate Directions





If the matrix A is diagonal, the contours of the function $f(\cdot)$ are ellipses whose axes are aligned with the coordinate directions. In each iteration we compute one component of the optimal solution, this takes two steps in $\mathbb{R}^2$.

If A is not diagonal, its contours are elliptical, but they are usually not aligned with the coordinate directions. Finding the optimum may take more than n iterations. Transform the problem to make A diagonal and minimize along the coordinate directions. Transformation matrix: $S = \left[\boldsymbol{d}_0, \boldsymbol{d}_1, \ldots, \boldsymbol{d}_{n-1}\right]$

Conjugate Gradient Method

- The **conjugate gradient method** is a **first order method** with the property: In generating its set of conjugate vectors, it can compute a new vector $\boldsymbol{d}_k$ by using only the previous vector $\boldsymbol{d}_{k-1}$. Hence, little storage and computation requirements.

$$\boxed{\boldsymbol{d}_k = -\boldsymbol{r}_k + \beta_k \boldsymbol{d}_{k-1}}$$

  where β_k is to be determined such that $\boldsymbol{d}_{k-1}$ and $\boldsymbol{d}_k$ are conjugate with respect to A. By premultiplying by $\boldsymbol{d}_{k-1}^T A$ and imposing that $\boldsymbol{d}_{k-1}^T A \boldsymbol{d}_k = 0$ we find that

$$\beta_k = \frac{\boldsymbol{r}_k^T A \boldsymbol{d}_{k-1}}{\boldsymbol{d}_{k-1}^T A \boldsymbol{d}_{k-1}}$$

- Larger values of β indicate that the previous descent direction contributes more strongly.
- $\boldsymbol{d}_0$ is commonly chosen to be the steepest descent direction at $\boldsymbol{x}_0$
- Advantage with respect to steepest descent: implicitly reuses previous information about the function and thus better convergence.

Algorithm CG

Basic version:

Input: f, $\boldsymbol{x}_0$
Output: x^*
Set $\boldsymbol{r}_0 \leftarrow A\boldsymbol{x}_0 - b$, $\boldsymbol{d}_0 \leftarrow \boldsymbol{r}_0$, $k \leftarrow 0$;
while $\boldsymbol{r}_k \neq \boldsymbol{0}$ **do**

$\quad \alpha_k \leftarrow -\frac{\boldsymbol{d}_k^T \boldsymbol{r}(\boldsymbol{x}_k)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k}$;
$\quad \boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$;
$\quad \boldsymbol{r}_{k+1} \leftarrow A\boldsymbol{x}_{k+1} - \boldsymbol{b}$;
$\quad \beta_{k+1} \leftarrow \frac{\boldsymbol{r}_{k+1}^T A \boldsymbol{d}_k}{\boldsymbol{d}_k^T A \boldsymbol{d}_k}$;
$\quad \boldsymbol{d}_{k+1} \leftarrow -\boldsymbol{r}_{k+1} + \beta_{k+1} \boldsymbol{d}_k$;
$\quad k \leftarrow k + 1$;

Computationally improved version:

Input: f, $\boldsymbol{x}_0$
Output: x^*
Set $\boldsymbol{r}_0 \leftarrow A\boldsymbol{x}_0 - b$, $\boldsymbol{d}_0 \leftarrow \boldsymbol{r}_0$, $k \leftarrow 0$;
while $\boldsymbol{r}_k \neq \boldsymbol{0}$ **do**

$\quad \alpha_k \leftarrow -\frac{\boldsymbol{r}(\boldsymbol{x}_k)^T \boldsymbol{r}(\boldsymbol{x}_k)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k}$;
$\quad \boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$;
$\quad \boldsymbol{r}_{k+1} \leftarrow \boldsymbol{r}_k + \alpha_k A\boldsymbol{d}_k$;
$\quad \beta_{k+1} \leftarrow \frac{\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}}{\boldsymbol{r}_k^T \boldsymbol{r}_k}$;
$\quad \boldsymbol{d}_{k+1} \leftarrow -\boldsymbol{r}_{k+1} + \beta_{k+1} \boldsymbol{d}_k$;
$\quad k \leftarrow k + 1$;

- we never need to know the vectors $\boldsymbol{x}$, $\boldsymbol{r}$, and $\boldsymbol{d}$ for more than the last two iterations.
- major computational tasks: the matrix–vector product $A\boldsymbol{d}_k$, inner products $\boldsymbol{d}_k^T A\boldsymbol{d}_k$ and $\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}$, and three vector sums

NonLinear Conjugate Gradient Methods

- The conjugate gradient method can be applied to nonquadratic functions as well.
- Smooth, continuous functions behave like quadratic functions close to a local minimum
- but! we do not know the value of A that best approximates f around $\boldsymbol{x}_k$. Instead, several choices for β_k tend to work well
- Two changes:
  - α_k is computed by solving an approximate line search
  - the residual $\boldsymbol{r}$, (it was simply the gradient of f), must be replaced by the gradient of the nonlinear objective f.

NonLinear Conjugate Gradient Methods

Fletcher-Reeves Method:

Input: f, $\boldsymbol{x}_0$
Output: x^*
Evaluate $f_0 = f(\boldsymbol{x}_0), \nabla f_0 = \nabla f(\boldsymbol{x}_0)$;
Set $\boldsymbol{d}_0 \leftarrow -\nabla f_0$, $k \leftarrow 0$;
while $\nabla f_k \neq 0$ **do**
  Compute α_k by line search and set
  $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$;
  Evaluate ∇f_{k+1};
  $\beta_{k+1}^{FR} \leftarrow \frac{\nabla f_{k+1}^T \nabla f_{k+1}}{\nabla f_k^T \nabla f_k}$;
  $\boldsymbol{d}_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{FR} \boldsymbol{d}_k$;
  $k \leftarrow k+1$;

Polak-Ribière:

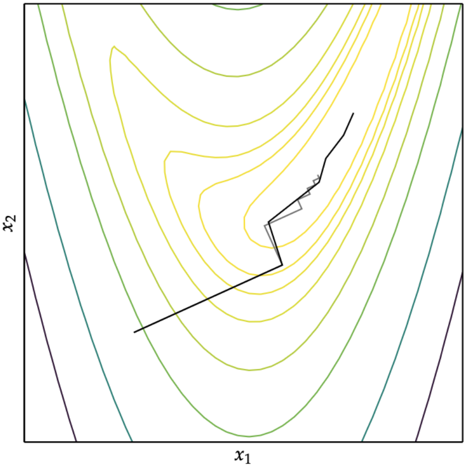
Input: f, $\boldsymbol{x}_0$
Output: x^*
Evaluate $f_0 = f(\boldsymbol{x}_0), \nabla f_0 = \nabla f(\boldsymbol{x}_0)$;
Set $\boldsymbol{d}_0 \leftarrow -\nabla f_0$, $k \leftarrow 0$;
while $\nabla f_k \neq 0$ **do**
  Compute α_k by line search and set
  $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$;
  Evaluate ∇f_{k+1};
  $\beta_{k+1}^{PR} \leftarrow \frac{\nabla f_{k+1}^T (\nabla f_{k+1} - \nabla f_k)}{\nabla f_k^T \nabla f_k}$;
  $\boldsymbol{d}_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{PR} \boldsymbol{d}_k$;
  $k \leftarrow k+1$;

PR with:

$$\beta_{k+1}^{+} = \max\{\beta_{k+1}^{PR}, 0\}$$

becomes PR$^+$ and guaranteed to converge (satisfies first Wolfe conditions).

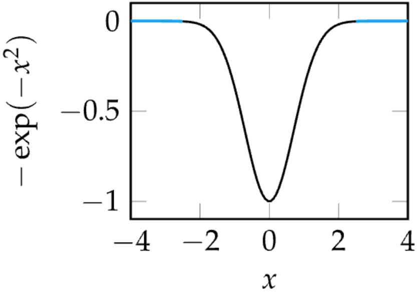
The conjugate gradient method with the Polak-Ribière update. Gradient descent is shown in gray.



Outline

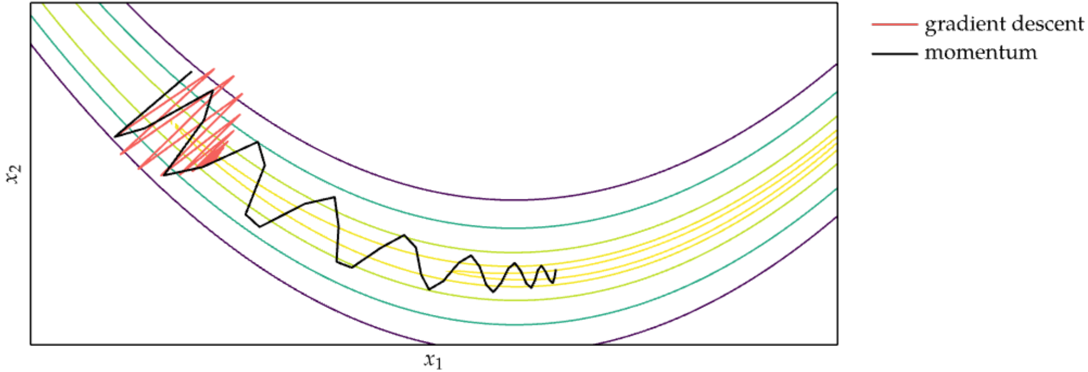
Accelerated Descents

- Addresses common convergence issues
- Some functions have regions with very small gradients (flat surface) where gradient descent gets stuck



Momentum

Rosenbrock function with $b = 100$



Momentum overcomes these issues by replicating the effect of physical momentum

Momentum

Momentum update equations:

$$\boxed{\begin{aligned} \mathbf{v}_{k+1} &= \beta \mathbf{v}_k - \alpha \nabla f(\mathbf{x}_k) \\ \mathbf{x}_{k+1} &= \mathbf{x}_k + \mathbf{v}_{k+1} \end{aligned}}$$

```python
import numpy as np

class Momentum(DescentMethod):
  alpha: float # learning rate
  beta: float # momentum decay
  v: np.array # momentum

  def __init__(self, alpha, beta, f, grad, x):
    self.alpha = alpha
    self.beta = beta
    self.v = np.zeros_like(x)

  def step(self, grad, x):
    self.v = self.beta * self.v - self.alpha * grad(x
        )
    return x + self.v
```

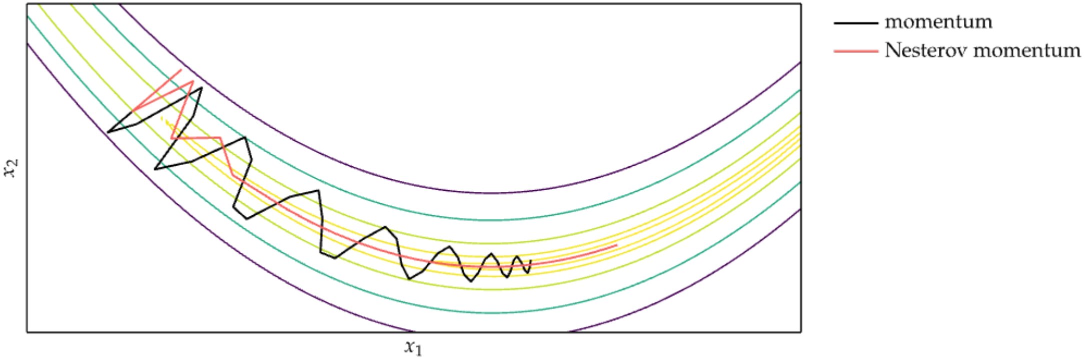
Nesterov Momentum Gradient (NAG)

Issue of momentum: steps do not slow down enough at the bottom of a valley, overshoot.

Nesterov Momentum update equations:

$$\boldsymbol{v}_{k+1} = \beta \boldsymbol{v}_k - \alpha \nabla f(\boldsymbol{x}_k + \beta \boldsymbol{v}_k)$$
$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{v}_{k+1}$$

it calculates the gradient at a "look-ahead" position



Adagrad

- Instead of using the same learning rate for all components of $\boldsymbol{x}$, **Adaptive Subgradient method** (Adagrad) adapts the learning rate for each component of $\boldsymbol{x}$. For each component of $\boldsymbol{x}$, the update equation is

$$x_{i,k+1} = x_{i,k} - \frac{\alpha}{\epsilon + \sqrt{s_{i,k}}} \nabla f_i(\boldsymbol{x}_k)$$

  where

$$s_{i,k} = \sum_{j=1}^{k} \left(\nabla f_i(\boldsymbol{x}_j)\right)^2$$

$$\epsilon \approx 1 \times 10^{-8}, \alpha = 0.01$$

- components of $\boldsymbol{s}$ are strictly nondecreasing, hence learning rate decreases over time

RMSProp

- Extends Adagrad to avoid monotonically decreasing learning rate by maintaining a decaying average of squared gradients

$$\hat{s}_{k+1} = \gamma \hat{s}_k + (1-\gamma)\left(\nabla f(\boldsymbol{x}_k) \odot \nabla f(\boldsymbol{x}_k)\right), \qquad \gamma \in [0,1], \qquad \odot \text{ element-wise product}$$

(accumulated squared gradients) and using it in the
Update Equation

$$\begin{aligned} x_{i,k+1} &= x_{i,k} - \frac{\alpha}{\epsilon + \sqrt{\hat{s}_{i,k}}} \nabla f_i(\boldsymbol{x}_k) \\ &\approx x_{i,k} - \frac{\alpha}{\epsilon + RMS(\nabla f_i(\boldsymbol{x}_k))} \nabla f_i(\boldsymbol{x}_k) \end{aligned}$$

root mean square: For n values $\{x_1, x_2, \ldots, x_n\}$

$$x_{\mathrm{RMS}} = \sqrt{\frac{1}{n}\left({x_1}^2 + {x_2}^2 + \cdots + {x_n}^2\right)}.$$

AdaDelta

Also extends Adagrad to avoid monotonically decreasing learning rate
Modifies RMSProp to eliminate learning rate parameter entirely

$$x_{i,k+1} = x_{i,k} - \frac{RMS(\Delta x_i)}{\epsilon + RMS(\nabla f_i(\boldsymbol{x}))} \nabla f_i(\boldsymbol{x}_k)$$

Δx_i sum of squared updates
$\nabla f_i(\boldsymbol{x})$ sum of squared gradients

Adam

- The **adaptive moment estimation method** (Adam), adapts the learning rate to each parameter.
- stores both an exponentially decaying gradient like momentum and an exponentially decaying squared gradient like RMSProp and Adadelta
- At each iteration, a sequence of values are computed

$$\begin{aligned}
\text{Biased decaying momentum} \quad & \mathbf{v}_{k+1} = \beta \mathbf{v}_k - \alpha \nabla f(\mathbf{x}_k) \\
\text{Biased decaying squared gradient} \quad & \mathbf{s}_{k+1} = \gamma \mathbf{s}_k + (1-\gamma)\left(\nabla f(\mathbf{x}_k) \odot \nabla f(\mathbf{x}_k)\right) \\
\text{Corrected decaying momentum} \quad & \hat{\mathbf{v}}_{k+1} = \mathbf{v}_{k+1} / (1 - \gamma_{v,k}) \\
\text{Corrected decaying squared gradient} \quad & \hat{\mathbf{s}}_{k+1} = \mathbf{s}_{k+1} / (1 - \gamma_{s,k}) \\
\text{Next iterate} \quad & \mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\alpha}{\left(\epsilon + \sqrt{\hat{\mathbf{s}}_{k+1}}\right)} \hat{\mathbf{v}}_{k+1}
\end{aligned}$$

- Defaults: $\alpha = 0.001, \gamma_v = 0.9, \gamma_s = 0.999, \epsilon = 1 \times 10^{-8}$

Adamax

Same as Adam, but based on the max-norm L_∞.

$$\begin{aligned} \boldsymbol{s}_{k+1} &= \gamma^\infty \boldsymbol{s}_k + (1-\gamma^\infty)\left(\|\nabla f(\boldsymbol{x}_k)\|_\infty\right) \\ &= \max\left(\gamma \boldsymbol{s}_k, \|\nabla f(\boldsymbol{x}_k)\|_\infty\right) \end{aligned}$$

Nadam

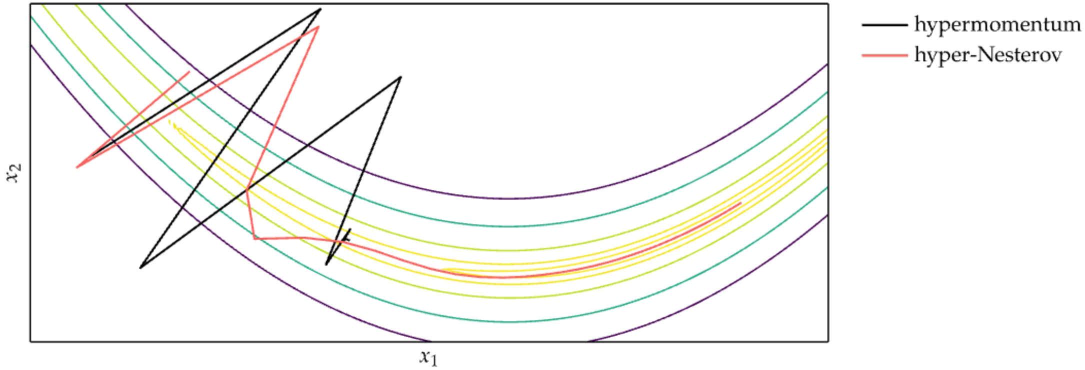
Nadam

- Nesterov-accelerated Adaptive Moment Estimation
- Adam is basically RMSProp with momentum
- We have seen that Nesterov is often more efficient
- Welcome to Nadam: Adam which uses the Nesterov momentum.

Hypergradient Descent

- Learning rate determines how sensitive the method is to the gradient signal.
- Many accelerated descent methods are highly sensitive to hyperparameters such as learning rate.
- Applying gradient descent to a hyperparameter of an underlying descent method is called hypergradient descent
- Requires computing the partial derivative of the objective function with respect to the hyperparameter

Hypergradient Descent



Summary

- Gradient descent follows the direction of steepest descent.
- The conjugate gradient method can automatically adjust to local valleys.
- Descent methods with momentum build up progress in favorable directions.
- A wide variety of accelerated descent methods use special techniques to speed up descent.
- Hypergradient descent applies gradient descent to the learning rate of an underlying descent method.

6. Second-Order Methods

Descent Direction Methods

How to select the descent direction?

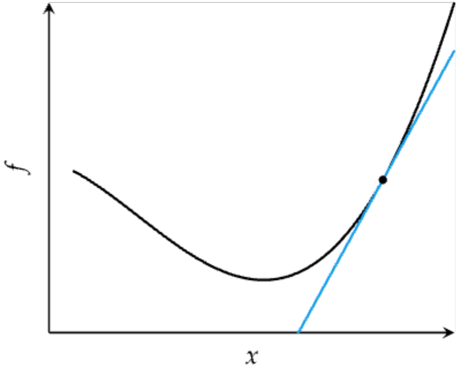
- first-order methods that rely on gradient
- second-order methods that rely on Hessian information

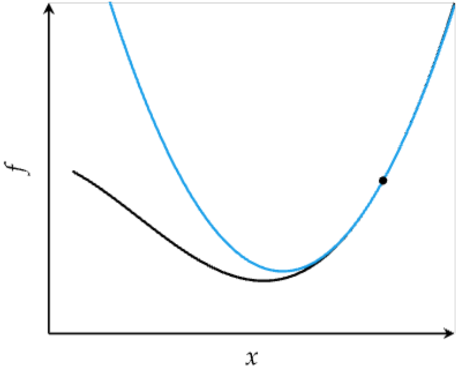
Advantages of second order methods in descent algorithms:

- way of accelerating the iteration [Davidon mid 1950s]
- additional information that can help improve the local model for informing the selection of
  - directions and
  - step lengths

Second-Order Methods

- Locally approximate function as quadratic
- Comparison of first-order and second order approximations





Outline

Newton's Method – Univariate

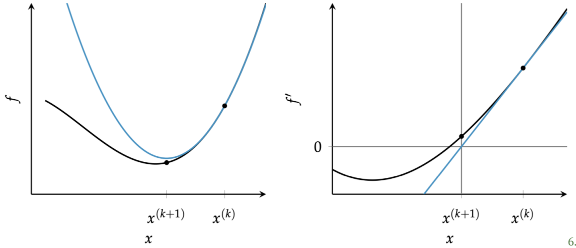
- Approximate a function using second-order Taylor series expansion
- analytically obtain the location where a quadratic approximation has a zero gradient.
- use that location as the next iteration to approach a local minimum.
- Univariate function

$$q(x) = f(x_k) + (x - x_k)f'(x_k) + \frac{(x - x_k)^2}{2} f''(x_k)$$

$\equiv$ finding roots of derivative function

$$\frac{\mathrm{d}q(x)}{\mathrm{d}x} = f'(x_k) + (x - x_k)f''(x_k) = 0$$

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$



Newton's Method - Multivariate

- Multivariate function

$$f(\boldsymbol{x}) \approx q(\boldsymbol{x}) = f(\boldsymbol{x}_k) + \nabla f(\boldsymbol{x}_k)^T(\boldsymbol{x} - \boldsymbol{x}_k) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}_k)^T H_k(\boldsymbol{x} - \boldsymbol{x}_k)$$

- H is the Hessian matrix
- Evaluate the gradient and set it to zero:

$$\nabla \boldsymbol{q}(x) = \nabla f(\boldsymbol{x}_k) + H(\boldsymbol{x}_k)(\boldsymbol{x} - \boldsymbol{x}_k) = 0$$

- Multivariate update rule

$$\boxed{\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - H_k^{-1}\nabla f(\boldsymbol{x}_k)}$$

- (If f is quadratic and its Hessian is positive definite, then the update converges to the global minimum in one step.)

Algorithm

Input: ∇f, H, $\boldsymbol{x}_0$, ϵ, k_{max}
Output: $\boldsymbol{x}^*$
Set $k = 0, \Delta = 1, \boldsymbol{x} = \boldsymbol{x}_0$;
while $\|\Delta\| > \epsilon$ and $k \leq k_{max}$ **do**
    $\Delta = H(\boldsymbol{x})^{-1}\nabla f(\boldsymbol{x})$;
    $\boldsymbol{x} = \boldsymbol{x} - \Delta$;
    $k = k + 1$;

It can be modified to only give a descent direction $\boldsymbol{d} = -H(\boldsymbol{x})^{-1}\nabla f(\boldsymbol{x})$ and leave the step size to be determined with line search.

Newton's method – Example

Minimize **Booth's function**:

$$f(x) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$$

- $\boldsymbol{x}_0 = [9, 8]$
- The gradient of Booth's function is:

$$\nabla f(x) = [10x_1 + 8x_2 - 34, 8x_1 + 10x_2 - 38]$$

- The Hessian of Booth's function is:

$$H(x) = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$$

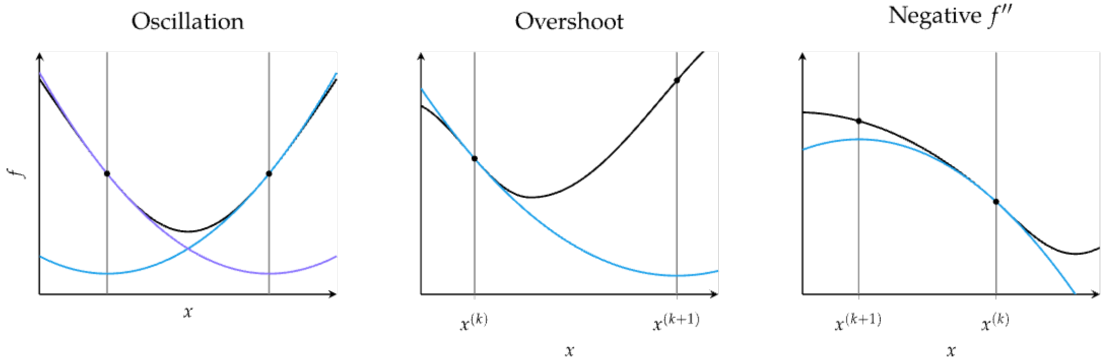
- The first iteration of Newton's method yields:

$$
\begin{aligned}
\boldsymbol{x}_1 &= \boldsymbol{x}_0 - H(\boldsymbol{x}_0)^{-1}\nabla f(\boldsymbol{x}_0) \\
&= \begin{bmatrix} 9 \\ 8 \end{bmatrix} - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1} \cdot \begin{bmatrix} 10 \cdot 9 + 8 \cdot 8 - 34 \\ 8 \cdot 9 + 10 \cdot 8 - 38 \end{bmatrix} = \begin{bmatrix} 9 \\ 8 \end{bmatrix} - \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}^{-1} \cdot \begin{bmatrix} 120 \\ 114 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}
\end{aligned}
$$

- Second iteration: The gradient at $\boldsymbol{x}_1$ is zero, so we have converged after a single iteration. The Hessian is positive definite everywhere, so $\boldsymbol{x}_1$ is the global minimum.

Newton's Method

Common causes of error in Newton's method



Strong Convexity

Recall that for a symmetric matrix A, $A \succeq 0$ means that A is positive semidefinite, i.e.

$$\boldsymbol{x}^\top A\boldsymbol{x} \geq 0 \quad \forall \boldsymbol{x} \in \mathbb{R}^n.$$

Hence, for a Q symmetric matrix, and I the identity matrix, the condition

$$Q \succeq cI \qquad \text{means:} \qquad Q - cI \succeq 0.$$

In scalar form:

$$\boldsymbol{x}^T(Q - cI)\boldsymbol{x} = \boldsymbol{x}^T Q\boldsymbol{x} - c\|\boldsymbol{x}\|^2 \geq 0.$$

$$\boldsymbol{x}^\top Q\boldsymbol{x} \geq c\|\boldsymbol{x}\|^2.$$

this was explained wrongly in class
and it has been corrected here

Strong Convexity

- Intuition: the second order derivative is positive, ie, $f''(x) \geq c > 0$.
- For high dimension: the Hessian matrix is positive definite.

Definition (Strongly Convex Function (Lower Curvature Bound))

The function $f : \mathbb{R}^d \to \mathbb{R}$ twice-differentiable function (ie, $f \in C^2(\mathbb{R}^n)$) is c-**strongly convex** if there exists a constant $c > 0$ such that for all $\boldsymbol{x}$ and $\boldsymbol{y}$ in $\mathbb{R}^d$:

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}) + \frac{c}{2}\|\boldsymbol{y} - \boldsymbol{x}\|^2.$$

or, equivalently, if there exists $c > 0$:

$$\nabla^2 f(\boldsymbol{x}) \succeq c\boldsymbol{I} \qquad \text{for all } \boldsymbol{x} \in \mathbb{R}^d$$

When $c = 0$, we recover the basic inequality characterizing convexity; for $c > 0$ we obtain a better lower bound on $f(\boldsymbol{x})$ than follows from convexity alone.

Convex $\rightarrow$ no flat "bending downward"; Strongly convex $\rightarrow$ strictly curved upward everywhere

Strong Convexity

Implications:

- Uniqueness of Minimizer:
  f has **exactly one** minimizer, denoted as $\boldsymbol{x}^* \in \mathbb{R}^d$ with $f^* \overset{def}{=} f(\boldsymbol{x}^*)$.

- $f(x) - f^* \leq \frac{1}{2c}\|\nabla f(x)\|^2$ (strong inequality with $\boldsymbol{y} = \boldsymbol{x}^*$ and $\nabla f(\boldsymbol{y}) = 0$)
  quadratic growth: function grows at least like a quadratic bowl centered at $\boldsymbol{x}^*$

Strong Convexity

Interpretation

- Compared to ordinary convexity:

  $$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}),$$

  strong convexity adds a quadratic curvature term.
  It enforces that the function grows at least quadratically away from any point.
  Equivalently:

  $$f(\boldsymbol{x}) - \frac{c}{2}\|\boldsymbol{x}\|^2 \text{ is convex.}$$

- Monotonicity of Gradient
  Strong convexity is equivalent to strong monotonicity of the gradient:

  $$(\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y}))^\top (\boldsymbol{x} - \boldsymbol{y}) \geq c\|\boldsymbol{x} - \boldsymbol{y}\|^2 \quad \forall \boldsymbol{x}, \boldsymbol{y}.$$

  This is extremely important in convergence proofs.

Strong Convexity

Eigenvalue Interpretation:

Because Q is symmetric, it has an orthonormal eigenbasis.
Let its eigenvalues be $\lambda_1, \ldots, \lambda_n$.
Then:

$$Q \succeq cI \iff \lambda_i \geq c \quad \forall i.$$

In other words:

$$\lambda_{\min}(Q) \geq c.$$

So the smallest curvature direction of Q is at least c.

If

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 100 \end{bmatrix},$$

then eigenvalues are 1 and 100. So:
So:

- $Q \succeq I$
- $Q \not\succeq 2I$ (since $1 < 2$)

The smallest curvature direction determines the strong convexity constant.

Strong Convexity

Geometric Meaning:

Consider the quadratic function

$$f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top Q\boldsymbol{x}.$$

Its curvature in direction $\boldsymbol{v}$ is $\boldsymbol{v}^\top Q\boldsymbol{v}$.
The condition $Q \succeq cI$ means:

$$\boldsymbol{v}^\top Q\boldsymbol{v} \geq c\|\boldsymbol{v}\|^2 \quad \forall\boldsymbol{v}.$$

So:
The function curves upward at least as much as a bowl with curvature c in every direction.

Smoothness Constant

- Definition in Optimization. It is NOT the smoothness in Mathematics (C^∞)
- Lipschitzness controls the changes in function value, while smoothness controls the changes in gradients.

Definition (Lipschitz Gradient (Upper Curvature Bound))

We say $f(\boldsymbol{x})$ is **L-smooth** when

$$f(\boldsymbol{y}) \leq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}) + \frac{L}{2}\|\boldsymbol{y} - \boldsymbol{x}\|^2, \quad L > 0$$

Some alternative definitions

1. $L\|x - x_0\|_2^2/2 - f(x)$ is convex (upper bounded by a quadratic function)
2. $-LI \preceq \nabla^2 f(x) \preceq LI$ (f has Lipschitz Hessian)
3. $\langle \nabla f(\boldsymbol{y}) - \nabla f(\boldsymbol{x}), \boldsymbol{y} - \boldsymbol{x} \rangle \leq L\|\boldsymbol{y} - \boldsymbol{x}\|^2$ (weaker when not require convexity)

Lipschitz Gradient

Definition (Lipschitz Gradient (Upper Curvature Bound))

f has an L-Lipschitz gradient if

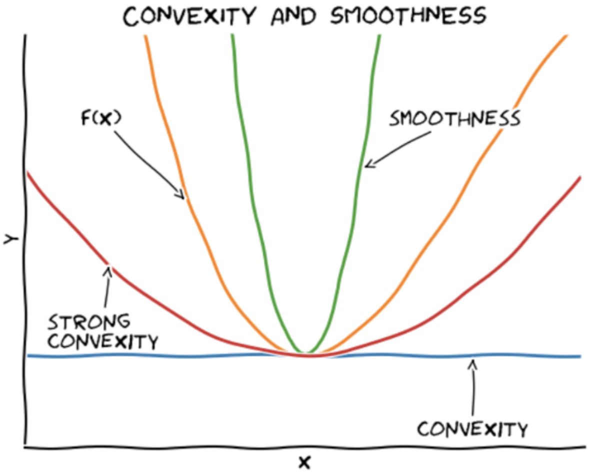
$$\|\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y})\| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|.$$

Equivalent (if $f \in C^2(\mathbb{R}^n)$):

$$\nabla^2 f(\boldsymbol{x}) \preceq LI \qquad \text{for all } \boldsymbol{x} \in \mathbb{R}^n$$

This gives a uniform upper bound on curvature.

Illustration of Strong Convexity and Smoothness



[illustration by Zhanyu Wang, 2021]

Combined Condition:
If both hold:

$$cI \preceq \nabla^2 f(\boldsymbol{x}) \preceq LI \qquad \text{for all } \boldsymbol{x} \in \mathbb{R}^n$$

then all eigenvalues of the Hessian satisfy:

$$c \leq \lambda_i(\nabla^2 f(\boldsymbol{x})) \leq L.$$

So:
Strong convexity $\rightarrow$ prevents flatness
Lipschitz gradient $\rightarrow$ prevents excessive curvature

Stable Step Size

We can derive the stable step size for gradient descent for quadratic problems.
Consider the quadratic problem

$$f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top Q\boldsymbol{x} - \boldsymbol{b}^\top \boldsymbol{x}, \qquad \text{where } Q = Q^\top \succ 0.$$

The gradient is $\nabla f(x) = Qx - \boldsymbol{b}$ and gradient descent with step size α:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \alpha(Q\boldsymbol{x}_k - \boldsymbol{b}).$$

We can derive that the step size that stabilizes the iteration is

$$\boxed{0 < \alpha < \frac{2}{\lambda_{\max}(Q)}}$$

The largest eigenvalue corresponds to the direction with the largest curvature: $L = \lambda_{\max}(Q)$. Thus:

$$\boxed{0 < \alpha < \frac{2}{L}.}$$

If the step size is too large relative to that curvature,
the method overshoots and oscillates.

Convergence rate of gradient descent for quadratic problems:

$$\|x_k - x_*\| \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^k \|x_0 - x_*\|$$

where

$$\kappa = \frac{\lambda_{\max}(Q)}{\lambda_{\min}(Q)} = \frac{L}{c}$$

is the optimization condition number of Q.

Stable Step Size - Line Search

- If relevant parameters describing the regularity and curvature of the objective function (Lipschitz constant, smoothness constant, strong convexity parameter) are known they can be used to tune the algorithm's own parameters.

- These parameters are often unknown and difficult to estimate, and Line search is a powerful technique heavily used in practice.

Newton's Method - Multivariate

Theorem

If f is a strongly convex function with Lipschitz Hessian (aka, has $\nabla^2 f(\boldsymbol{x})$ that is Lipschitz continuous), then provided that x_0 is close enough to $x_* = \arg\min f(x)$, the sequence $x_0, x_1, x_2, \ldots$ generated by Newton's method will converge to the (necessarily unique) minimizer x_* of f quadratically fast. That is,

$$\|x_{k+1} - x_*\| \leq \frac{1}{2}\|x_k - x_*\|^2, \qquad \forall k \geq 0.$$

Lipschitz Hessian is a third order information on the variation of curvature.
Strong convexity says: The bowl never flattens. Lipschitz Hessian says: The bowl shape cannot change too abruptly.

Convergence of Newton's Method

Theorem (Convergence of Netwons Method)

Suppose that f is twice differentiable and that the Hessian $\nabla^2 f(\boldsymbol{x})$ is Lipschitz continuous with constant L in a neighborhood of a strict local minimizer $\boldsymbol{x}^*$. Consider the iteration $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{p}_k$, where $\boldsymbol{p}_k = -\nabla^2(\boldsymbol{x}_k)^{-1}\nabla f(\boldsymbol{x}_k)$. Then:

1. *if the starting point $\boldsymbol{x}_0$ is sufficiently close to $\boldsymbol{x}^*$, the sequence of iterates converges to $\boldsymbol{x}^*$;*
2. *the rate of convergence of $\{\boldsymbol{x}_k\}$ is quadratic; and*
3. *the sequence of gradient norms $\{\|\nabla f_k\|\}$ converges quadratically to zero.*

Quadratic convergence means that, when the difference is small, the difference between the minimum and the iterate is approximately squared with every iteration.

Newton's Method

Equivalently, from the text book:

- A quadratic rate of convergence holds for Newton's method starting from x_0 within an interval $I = [x^* - \delta, x^* + \delta]$, for a root x^*, if

  1. $f''(x) \neq 0$ for all points in I,
  2. $f'''(x)$ is continuous on I, and
  3. $\frac{1}{2}\left|\frac{f'''(x_0)}{f''(x_0)}\right| < L \left|\frac{f'''(x^*)}{f''(x^*)}\right|$ for some $L < \infty$
     sufficient closeness condition, ensuring that the function is sufficiently approximated by the Taylor expansion and no overshoot.

- Note that the Hessian matrix $\nabla^2 f(\boldsymbol{x}_k)$ may not always be positive definite, and the Netwon step: $\boldsymbol{d}_k = -\nabla^2(\boldsymbol{x}_k)^{-1}\nabla f(\boldsymbol{x}_k)$ not always be a descent direction. (There are approaches for obtaining a globally convergent iteration based on the Newton step: a line search approach, in which the Hessian $\nabla^2 f(\boldsymbol{x}_k)$ is modified, if necessary, to make it positive definite)

Outline

Secant Method – Univariate

- For univariate functions, if the second derivative is unknown, it can be approximated using the secant method

$$f''(x_k) \approx \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}}$$

- Update equation

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} f'(x_k)$$

- It requires an additional initial design point and suffers from the same problems as Newton's method and may take more iterations to converge due to approximating the second derivative.

Outline

Quasi-Netwon Methods – Multivariate

- Automatic differentiation tools may not be applicable in many situations, and it may be much more costly to work with second derivatives in automatic differentiation software than with the gradient.

- Quasi-Newton methods, like steepest descent, require only the gradient of the objective function to be supplied at each iterate.

- By measuring the changes in gradients, they construct a model of the objective function that is good enough to produce superlinear convergence.

- The improvement over steepest descent is dramatic, especially on difficult problems.

Quasi-Netwon Methods – Multivariate

Use an approximation $Q_k \approx H^{-1}(\boldsymbol{x}_k)$

Input: $\boldsymbol{x}_0$, convergence tolerance $\epsilon > 0$, Q_0 (typically the $n \times n$ identity matrix)
Output: x^*
Set $k \leftarrow 0$;
while $\|\nabla f(\boldsymbol{x}_k)\| > \epsilon$ **do**
  Compute search direction $\boldsymbol{d}(x_k) = -Q_k \nabla f(\boldsymbol{x}_k)$;
  Set $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{d}(\boldsymbol{x}_k)$ where α_k is computed from a line search procedure to satisfy the Wolfe conditions;
  Define $\boldsymbol{\delta}_{k+1} \stackrel{def}{=} \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$ and $\gamma_{k+1} \stackrel{def}{=} \nabla f(\boldsymbol{x}_{k+1}) - \nabla f(\boldsymbol{x}_k)$;
  Compute Q_{k+1};
  $k \leftarrow k + 1$;

- Davidon-Fletcher-Powell (DFP) method
- Broyden-Fletcher-Goldfarb-Shanno (BFGS) method
- Limited-memory BFGS (L-BFGS) method

Davidon-Fletcher-Powell (DFP) method

$$Q_{k+1} = Q_k - \frac{Q_k \gamma_k \gamma_k^T Q_k}{\gamma_k^T Q_k \gamma_k} + \frac{\delta_k \delta_k^T}{\delta_k^T \gamma_k}$$

where all terms on the right hand side are evaluated at the same iteration k.

The update for Q in the DFP method has three properties:

- Q remains symmetric and positive definite.
- If $f(x) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} + \mathbf{b}^T\mathbf{x} + c$, then $Q = A^{-1}$. Thus the DFP has the same convergence properties as the conjugate gradient method.
- For high-dimensional problems, storing and updating Q can be significant compared to other methods like the conjugate gradient method.

Broyden-Fletcher-Goldfarb-Shanno (BFGS) method

$$Q_{k+1} = Q_k - \left(\frac{\delta_k \gamma_k^T Q_k + Q_k \gamma_k \delta_k^T}{\delta_k^T \gamma_k} \right) + \left(1 + \frac{\gamma_k^T Q_k \gamma_k}{\delta_k^T \gamma_k} \right) \frac{\delta_k \delta_k^T}{\delta_k^T \gamma_k}$$

BFGS better than DFP with approximate line search but still uses an $n \times n$ dense matrix.

Theorem: Suppose that f is twice continuously differentiable and that the iterates generated by the BFGS algorithm converge to a minimizer $\boldsymbol{x}^*$ at which the Hessian matrix G is Lipschitz continuous Suppose also that the sequence $\|\boldsymbol{x}_k - \boldsymbol{x}^*\|$ converges to zero rapidly enough that $\sum_{k=1}^{\infty} \|\boldsymbol{x}_k - \boldsymbol{x}^*\| < \infty$. Then $\boldsymbol{x}_k$ converges to $\boldsymbol{x}^*$ at a superlinear rate (ie, faster than linear).

Limited-memory BFGS (L-BFGS) method

For **large-scale unconstrained optimization**
It stores the last m values for δ and γ rather than the full inverse Hessian ($i = 1$ oldest, $i = m$ last).
Compute $\boldsymbol{d}$ at $\boldsymbol{x}$ as $\boldsymbol{d} = -\boldsymbol{z}_m$ using:

$$\boldsymbol{q}_m = \nabla f(\boldsymbol{x}_k) \qquad \boldsymbol{q}_i = \boldsymbol{q}_{i+1} - \frac{\boldsymbol{\delta}_{i+1}^T \boldsymbol{q}_{i+1}}{\boldsymbol{\gamma}_{i+1}^T \boldsymbol{\delta}_{i+1}} \boldsymbol{\gamma}_{i+1}, \quad i = m-1, \ldots, 1$$

$$\boldsymbol{z}_0 = \frac{\boldsymbol{\delta}_m \odot \boldsymbol{\delta}_m \odot \boldsymbol{q}_m}{\boldsymbol{\gamma}_m^T \boldsymbol{\gamma}_m} \qquad \boldsymbol{z}_i = \boldsymbol{z}_{i-1} + \boldsymbol{\delta}_{i-1} \left(\frac{\boldsymbol{\delta}_{i-1}^T \boldsymbol{q}_{i-1}}{\boldsymbol{\gamma}_{i-1}^T \boldsymbol{\delta}_{i-1}} - \frac{\boldsymbol{\gamma}_{i-1}^T \boldsymbol{z}_{i-1}}{\boldsymbol{\gamma}_{i-1}^T \boldsymbol{\gamma}_{i-1}} \right), \quad i = 1, \ldots, m$$

For minimization, the inverse Hessian Q must remain positive definite.
The initial Hessian is often set to the diagonal of

$$Q_0 = \frac{\boldsymbol{\gamma}_0 \boldsymbol{\delta}_0^T}{\boldsymbol{\gamma}_0^T \boldsymbol{\gamma}_0}$$

Computing the diagonal for the above expression and substituting the result into $\boldsymbol{z}_0 = Q_0 \boldsymbol{q}_0$ results in the equation for $\boldsymbol{z}_0$.

L-BFGS

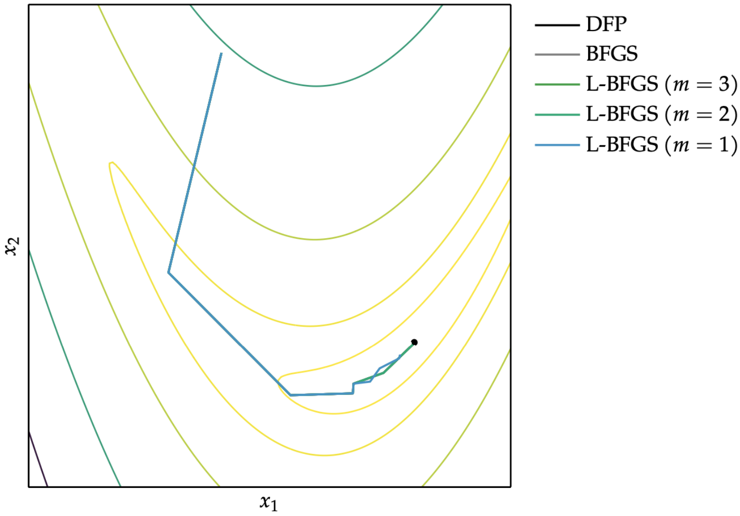
(L-BFGS two-loop recursion)

Input:
Output: $Q_k \nabla f_k = \boldsymbol{z}$
Set $\boldsymbol{q} \leftarrow \nabla f_k$;
for $i = k-1, k-2, \dots, k-m$ **do**
  $\alpha_i \leftarrow \frac{\delta_i^T \boldsymbol{q}}{\boldsymbol{\gamma}_i \cdot \boldsymbol{\delta}_i^T}$;
  $\boldsymbol{q} \leftarrow \boldsymbol{q} - \alpha_i \cdot \boldsymbol{\gamma}_i$;

$\boldsymbol{z} \leftarrow Q_0 \boldsymbol{q}$;
for $i = k-m, k-m+1, \dots, k-1$
 do
  $\beta \leftarrow \frac{\boldsymbol{\gamma}_i^T \cdot \boldsymbol{r}}{\boldsymbol{\gamma}_i \cdot \boldsymbol{\delta}_i^T}$;
  $\boldsymbol{z} \leftarrow \boldsymbol{z} + \delta_i(\boldsymbol{z}_i - \beta)$;

Input:
Output: x^*
Choose starting point $\boldsymbol{x}_0$, integer $m > 0$;
$k \leftarrow 0$;
while not convergence **do**
  Set Q_0;
  Compute $\boldsymbol{d}_k \leftarrow -Q_k \nabla f_k$ from Algorithm on the left;
  Compute $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$, where α_k is chosen to satisfy the Wolfe conditions;
  if $k > m$ **then**
   Discard the vector pair $\{\boldsymbol{\delta}_{k-m}, \boldsymbol{\gamma}_{k-m}\}$ from storage;
  Compute and save $\boldsymbol{\delta}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, $\boldsymbol{\gamma}_k = \nabla f_{k+1} - \nabla f_k$;
  $k \leftarrow k + 1$;

BFGS Methods - Comparison



Summary

- Incorporating second-order information in descent methods often speeds convergence.
- Newton's method is a root-finding method that leverages second-order information to quickly descend to a local minimum.
- The secant method and quasi-Newton methods approximate Newton's method when the second-order information is not directly available.
- In Python, methods implemented in the module scipy
  `https://docs.scipy.org/doc/scipy/tutorial/optimize.html`

7. Direct Methods

Outline

Black-Box Model

$$\begin{aligned} \min \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{x} \in \mathcal{X} \end{aligned}$$

In the black-box model we assume that we have unlimited computational resources, the set of constraint $\mathcal{X}$ is known, and the objective function $f : \mathcal{X} \to \mathbb{R}$ is unknown but can be accessed through queries to **oracles**:

- A zeroth order oracle takes as input a point $\boldsymbol{x} \in \mathcal{X}$ and outputs the value of f at x.
- A first order oracle takes as input a point $\boldsymbol{x} \in \mathcal{X}$ and outputs a subgradient of f at x.

In the situation where also the set of constraint $\mathcal{X}$ is unknown, it can only be accessed through a **separation oracle**: given $\boldsymbol{x} \in \mathbb{R}^n$, it outputs either that $\boldsymbol{x}$ is in $\mathcal{X}$, or if $\boldsymbol{x} \notin \mathcal{X}$. Then it may output a separating hyperplane between $\boldsymbol{x}$ and $\mathcal{X}$.

Subgradient and Generalized Gradient

The subgradient generalizes the notion of gradient to nondifferentiable continuous, functions.

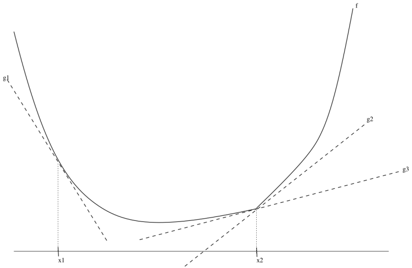
Definition (Subgradient)

We say a vector $g \in \mathbb{R}^n$ is a subgradient of $f : \mathbb{R}^n \to \mathbb{R}$ at $\boldsymbol{x} \in \operatorname{dom} f$ if for all $z \in \operatorname{dom} f$,

$$f(\boldsymbol{z}) \geq f(\boldsymbol{x}) + \boldsymbol{g}^\top (\boldsymbol{z} - \boldsymbol{x}).$$

Equivalently, g is a subgradient of f at x if $f(x) + g^T(z - x)$ is an hyperplane that supports $\operatorname{epi} f$ at $(x, f(x))$.

- If f is convex and differentiable, then its gradient at $\boldsymbol{x}$ is a subgradient of f at $\boldsymbol{x}$.
- There can be more than one subgradient of a function f at a point $\boldsymbol{x}$.
- (We will not use subgradients in this course)



Derivative-Free Methods

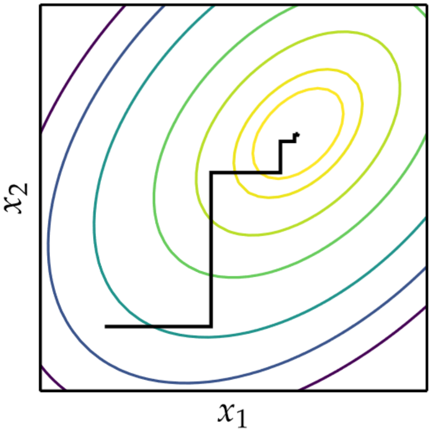
- Also called **direct search methods**, zeroth order black box, or pattern search
- Direct method search using function evaluations only

Cyclic Coordinate Search

- Also known as coordinate descent, or taxicab search
- Performs line search in alternating coordinate directions

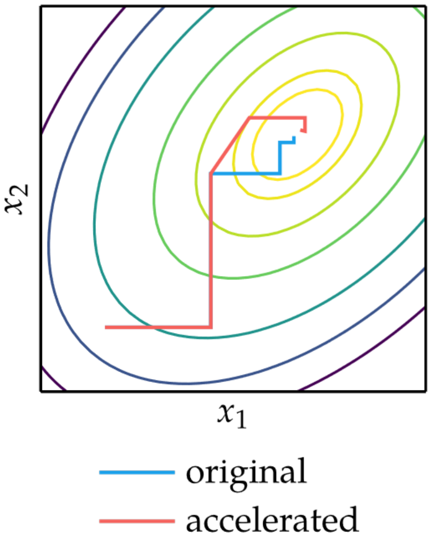
$$x_{1,1} = \text{argmin}_{x_1} f(x_1, x_{2,0}, x_{3,0}, \ldots, x_{n,0})$$

$$x_{2,1} = \text{argmin}_{x_2} f(x_{1,1}, x_2, x_{3,1}, \ldots, x_{n,1})$$



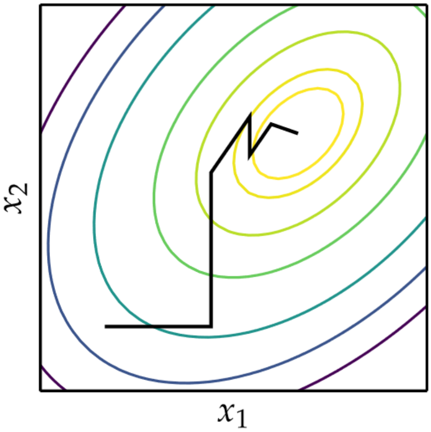
Cyclic Coordinate Search

- Can be augmented to accelerate convergence
- For every full cycle starting with optimizing $\boldsymbol{x}_1$ along $[1, 0, \ldots, 0]$ and ending with $\boldsymbol{x}_{n+1}$ after optimizing along $[0, 0, \ldots, 1]$, an additional line search is conducted along the direction $\boldsymbol{x}_{n+1} - \boldsymbol{x}_1$.



Powell's Method

- Similar to Cyclic Coordinate Search, but can search in non-orthogonal directions
- Drops the oldest search direction in favor of the overall direction of progress
- It can lead the search directions to become linearly dependent and the search directions can no longer cover the full design space, and the method may not be able to find the minimum



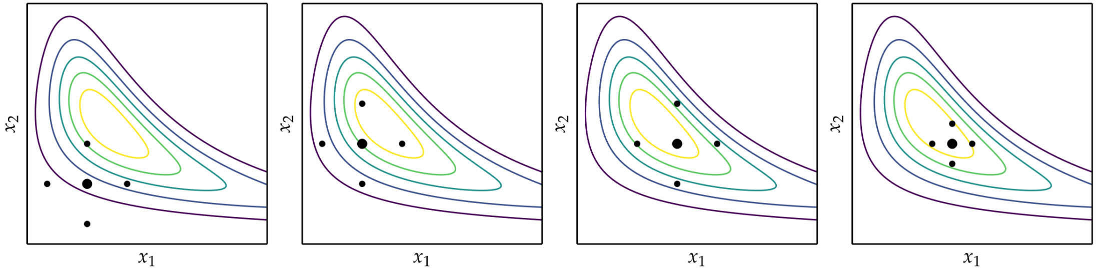
Powell's Method

Input: $f, \boldsymbol{x}_0$
Output: $\boldsymbol{x}^*$
search directions $\boldsymbol{u}_1 = \boldsymbol{e}_1, \ldots, \boldsymbol{u}_n = \boldsymbol{e}_i$;
while not converged **do**
    for i in $\{1, \ldots, n\}$ **do**
        $\boldsymbol{x}_{i+1} \leftarrow \texttt{line\_search}(f, \boldsymbol{x}_i, \boldsymbol{u}_i)$;
    for i in $\{1, \ldots, n-1\}$ **do**
        $\boldsymbol{u}_i \leftarrow \boldsymbol{u}_{i+1}$;
    $\boldsymbol{u}_n \leftarrow \boldsymbol{x}_{n+1} - \boldsymbol{x}_1$;

- search directions can become linearly dependent and no longer cover the full design space.
- peridocally reset the diections to the canonical basis.

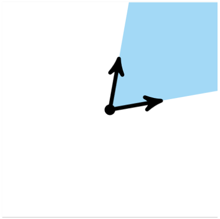
Hooke-Jeeves

- Evaluate $f(\mathbf{x})$ and $f(\mathbf{x} \pm \alpha \mathbf{e}_i)$ for a given step size α in every coordinate direction from an anchoring point $\mathbf{x}$.
- It accepts any improvement it may find.
- If no improvements are found, it decreases the step size.
- The process repeats until the step size is sufficiently small.
- $2n$ evaluations for an n-dimensional problem

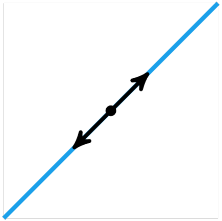


Generalized Pattern Search

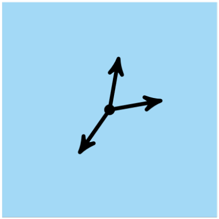
- Generalization of Hooke-Jeeves method that can use as few as $n + 1$ search directions instead of $2n$
- A pattern P can be constructed from a set of m directions $\mathcal{D}$ about an anchoring point $\boldsymbol{x}$ with a step size α according to: $P = \{\mathbf{x} + \alpha\boldsymbol{d} \text{ for each } \boldsymbol{d} \in \mathcal{D}\}$
- Searches in set of directions that **positively spans** (nonnegative linear combination) search space. (if D has full row rank and if $D\mathbf{x} = -D\mathbf{1}$ with $\boldsymbol{x} \geq 0$)



only positively spans the cone

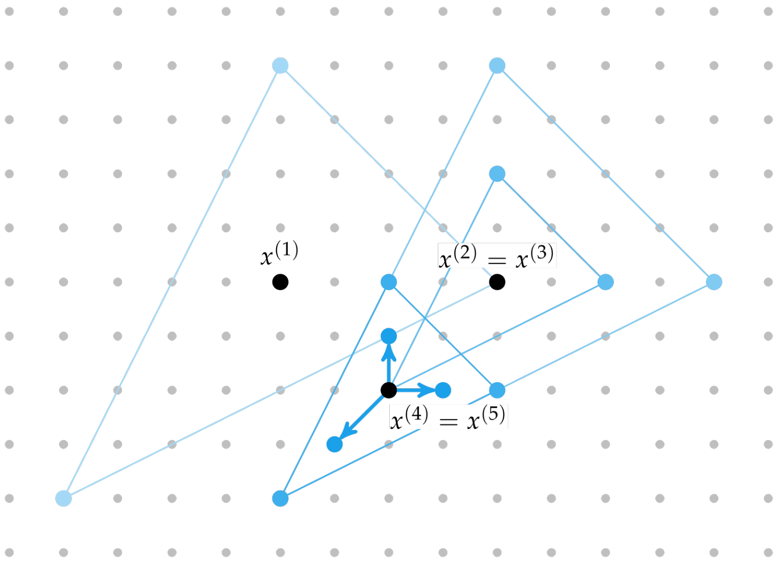


only positively spans 1d space



positively spans $\mathbb{R}^2$

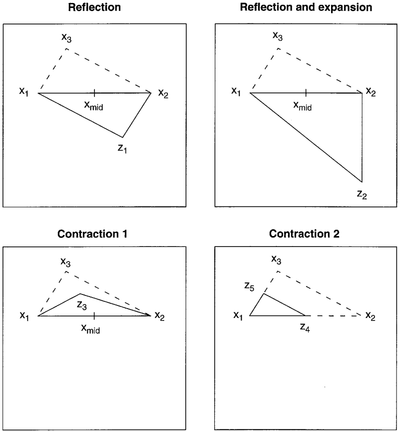
Generalized Pattern Search



Outline

Nelder-Mead Simplex Method

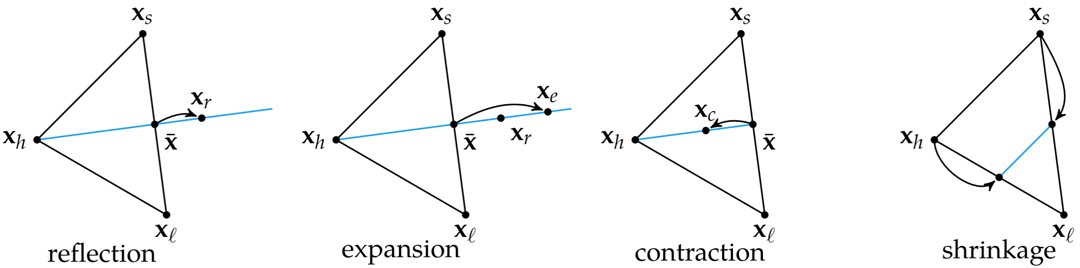
Uses simple algorithm to traverse search space using set of points defining a simplex [Nelder and Mead, 1965]:

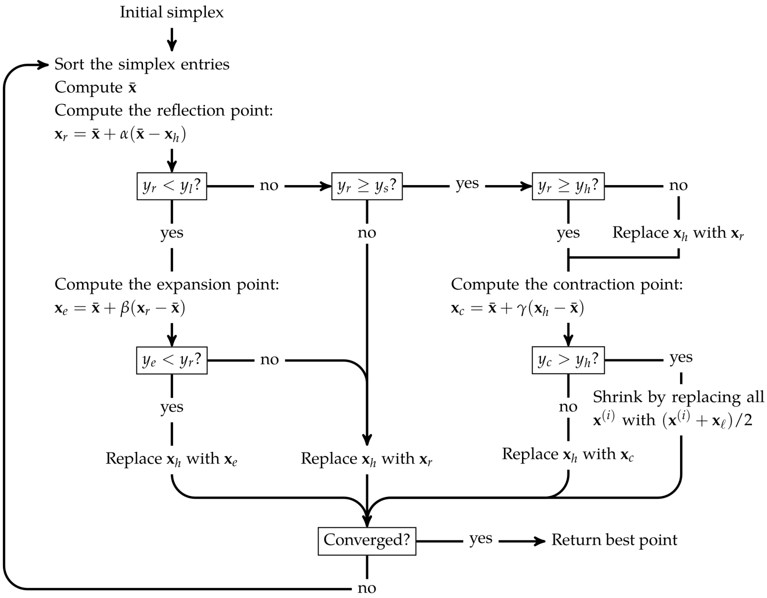


In each iteration:

- we seek to replace the vertex with the worst function value with another point with a better value.
- The new point is obtained by **reflecting, expanding**, or **contracting** the simplex along the line joining the worst vertex with the centroid of the remaining vertices.
- If we cannot find a better point in this manner, we retain only the vertex with the best function value, and we **shrink** the simplex by moving all other vertices toward this value.

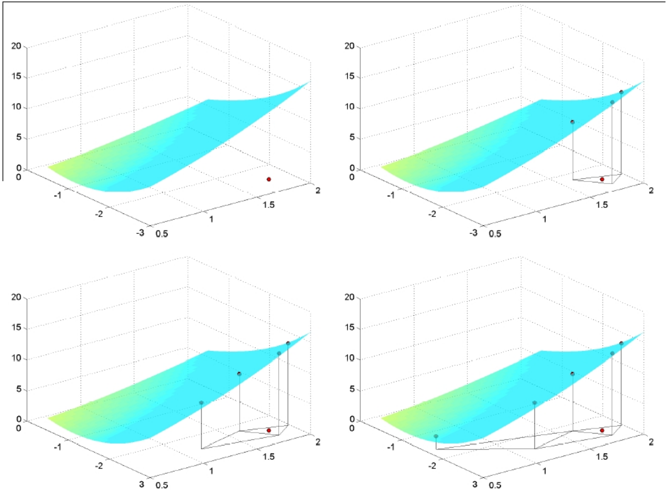
Nelder-Mead (cont.)




Initial simplex
Sort the simplex entries
Compute $\bar{\mathbf{x}}$
Compute the reflection point:
$\mathbf{x}_r = \bar{\mathbf{x}} + \alpha(\bar{\mathbf{x}} - \mathbf{x}_h)$
$y_r < y_l?$
no
$y_r \geq y_s?$
yes
$y_r \geq y_h?$
no
yes
no
yes
Replace $\mathbf{x}_h$ with $\mathbf{x}_r$
Compute the expansion point:
$\mathbf{x}_e = \bar{\mathbf{x}} + \beta(\mathbf{x}_r - \bar{\mathbf{x}})$
Compute the contraction point:
$\mathbf{x}_c = \bar{\mathbf{x}} + \gamma(\mathbf{x}_h - \bar{\mathbf{x}})$
$y_e < y_r?$
no
$y_c > y_h?$
yes
yes
no
Shrink by replacing all
$\mathbf{x}^{(i)}$ with $(\mathbf{x}^{(i)} + \mathbf{x}_\ell)/2$
Replace $\mathbf{x}_h$ with $\mathbf{x}_e$
Replace $\mathbf{x}_h$ with $\mathbf{x}_r$
Replace $\mathbf{x}_h$ with $\mathbf{x}_c$
Converged?
yes
Return best point
no

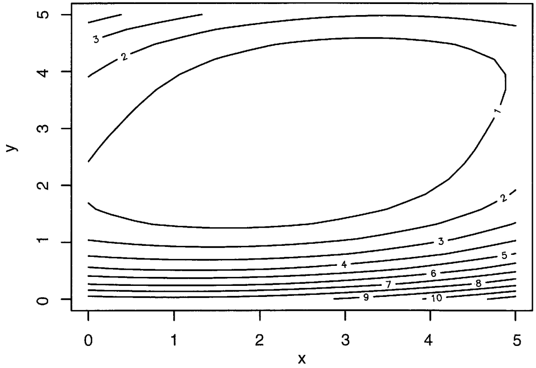
Nelder-Mead

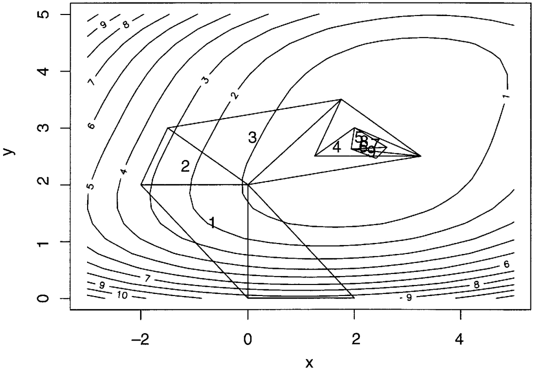
Simplex based method [Spendley et al. (1962)]

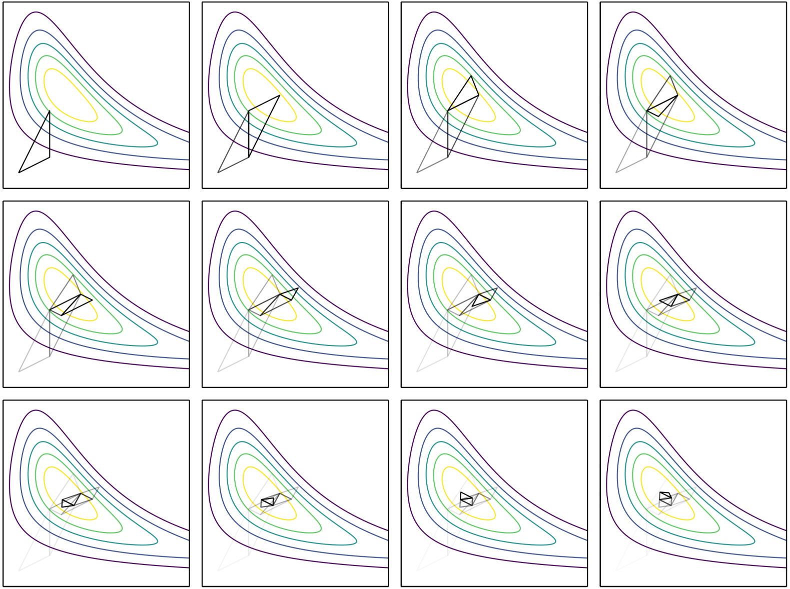


Nelder-Mead (cont.)

Example:





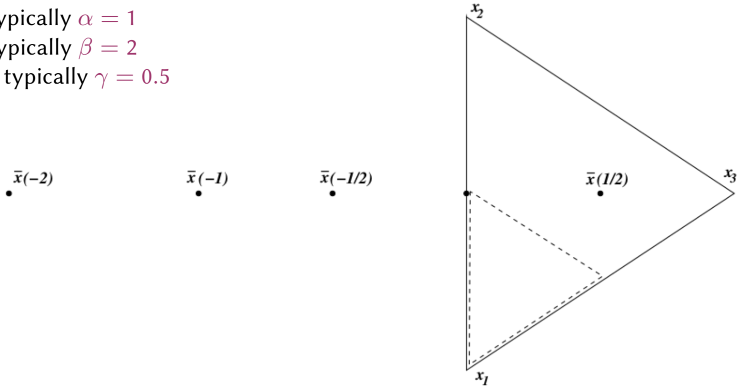


Nelder-Mead (cont.)

Modelling the operations: [Nocedal and Wright, 2006]

- $\boldsymbol{x}_1, \dots, \boldsymbol{x}_{n+1}$ vertices of a simplex ordered such that $f(\boldsymbol{x}_1) \le f(\boldsymbol{x}_2) \le \dots \le f(\boldsymbol{x}_{n+1})$
- $\bar{\boldsymbol{x}} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}_i$ centroid of the points excluding the worst $\boldsymbol{x}_{n+1}$
- $\bar{\boldsymbol{x}}(t) = \bar{\boldsymbol{x}} + t(\boldsymbol{x}_{n+1} - \bar{\boldsymbol{x}})$ points along the line joining $\boldsymbol{x}_{n+1}$ and $\bar{\boldsymbol{x}}$

Reflection: evaluating $\bar{\boldsymbol{x}}(t)$ at $t = -\alpha$, typically $\alpha = 1$
Expansion: evaluating $\bar{\boldsymbol{x}}(t)$ at $t = -\beta$, typically $\beta = 2$
Contraction: evaluating $\bar{\boldsymbol{x}}(t)$ at $t = \pm\gamma$, typically $\gamma = 0.5$

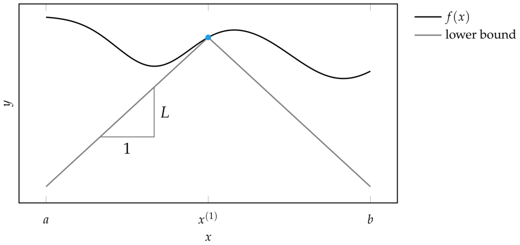


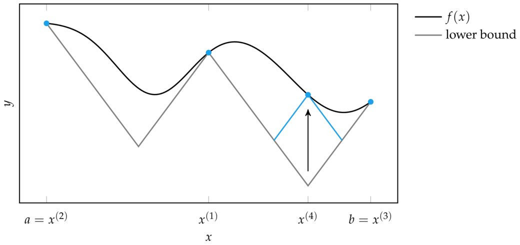
Input: f, $\mathbf{x}_1, \ldots, \mathbf{x}_{n+1}$ vertices of a simplex ordered such that $f(\mathbf{x}_1) \leq f(\mathbf{x}_2) \leq \ldots \leq f(\mathbf{x}_{n+1})$
Output: $\mathbf{x}^*$
Let $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$ be the centroid of the points excluding $\mathbf{x}_{n+1}$;
Let $\bar{\mathbf{x}}(t) = \bar{\mathbf{x}} + t(\mathbf{x}_{n+1} - \bar{\mathbf{x}})$ points along the line joining $\mathbf{x}_{n+1}$ and $\bar{\mathbf{x}}$;
Compute the **reflection point** $\mathbf{x}_R = \bar{\mathbf{x}}(-1)$ and evaluate $f_R = f(\mathbf{x}(R))$;
while termination condition not met **do**
- **if** $f(\mathbf{x}_1) \leq f_R < f(\mathbf{x}_n)$ **then**
  - $\mathbf{x}_{n+1} \leftarrow \mathbf{x}_R$ and continue;
- **else if** $f_R < f(\mathbf{x}_1)$ **then**
  - Compute the **expansion point** $\mathbf{x}_E = \bar{\mathbf{x}}(-2)$ and evaluate $f_E = f(\mathbf{x}_E)$;
  - **if** $f_E < f_R$ **then**
    - $\mathbf{x}_{n+1} \leftarrow \mathbf{x}_E$ and continue;
  - **else**
    - $\mathbf{x}_{n+1} \leftarrow \mathbf{x}_R$ and continue;
- **else if** $f_R \geq f(\mathbf{x}_n)$ **then**
  - **if** $f(\mathbf{x}_n) \leq f_R < f(\mathbf{x}_{n+1})$ **then**
    - Compute **outside contraction point** $\mathbf{x}_C = \bar{\mathbf{x}}(-1/2)$ and evaluate $f_C = f(\mathbf{x}_C)$;
    - **if** $f_C < f_R$ **then** $\mathbf{x}_{n+1} \leftarrow \mathbf{x}_C$ and continue ;
  - **else**
    - Compute **inside contraction point** $\mathbf{x}_C = \bar{\mathbf{x}}(1/2)$ and evaluate $f_C = f(\mathbf{x}_C)$;
    - **if** $f_C < f(\mathbf{x}_{n+1})$ **then** $\mathbf{x}_{n+1} \leftarrow \mathbf{x}_C$ and continue ;
- **shrink**: $\mathbf{x}_i \leftarrow (1/2)(\mathbf{x}_1 + \mathbf{x}_i)$ for $i = 2, 3, \ldots, n+1$;

Outline

DIRECT – Divided Rectangles

- Also called DIRECT for Divided RECTangles
- Recall from Shubert-Piyavskii, a Lipshitz constant is used to provide a lower bound on the function, and a function evaluation is made where this bound is lowest





DIRECT – Divided Rectangles

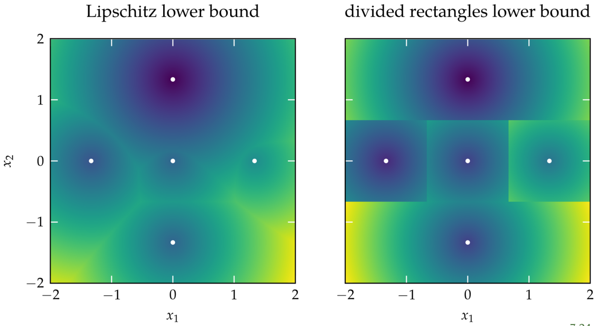
- The notion of Lipschitz continuity can be extended to multiple dimensions.
  If f is Lipschitz continuous over a domain X with Lipschitz constant $\ell > 0$, then for a given design $\boldsymbol{x}_0$ and $y = f(\boldsymbol{x}_0)$, the circular cone

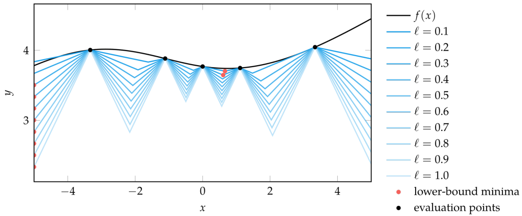
$$f(\boldsymbol{x}_0) - \ell \|\boldsymbol{x} - \boldsymbol{x}_0\|_2$$

  forms a lower bound of f

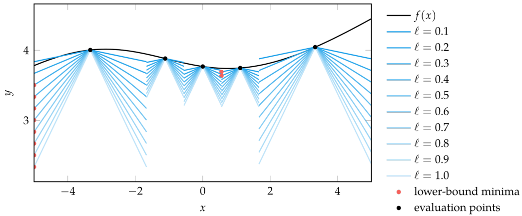
- Given m function evaluations with design points $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$, we can construct a superposition of these lower bounds by taking their maximum:

$$\max_i f(\boldsymbol{x}_i) - \ell \|\boldsymbol{x} - \boldsymbol{x}_i\|_2$$

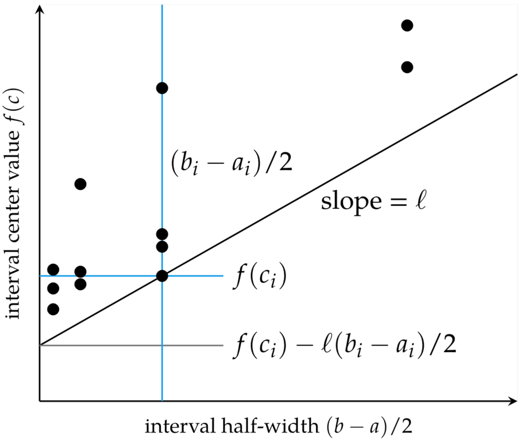




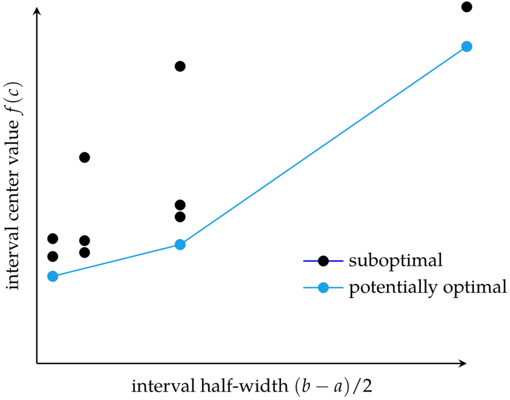
4
3
y
−4
−2
0
2
4
x
f(x)
ℓ = 0.1
ℓ = 0.2
ℓ = 0.3
ℓ = 0.4
ℓ = 0.5
ℓ = 0.6
ℓ = 0.7
ℓ = 0.8
ℓ = 0.9
ℓ = 1.0
lower-bound minima
evaluation points



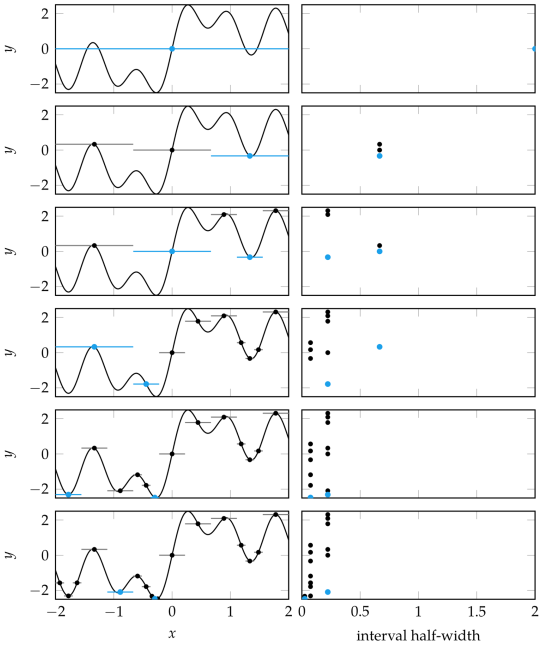
4
3
y
−4
−2
0
2
4
x
f(x)
ℓ = 0.1
ℓ = 0.2
ℓ = 0.3
ℓ = 0.4
ℓ = 0.5
ℓ = 0.6
ℓ = 0.7
ℓ = 0.8
ℓ = 0.9
ℓ = 1.0
lower-bound minima
evaluation points



interval center value $f(c)$
$(b_i - a_i)/2$
slope = ℓ
$f(c_i)$
$f(c_i) - \ell(b_i - a_i)/2$
interval half-width $(b - a)/2$



interval center value $f(c)$
suboptimal
potentially optimal
interval half-width $(b - a)/2$

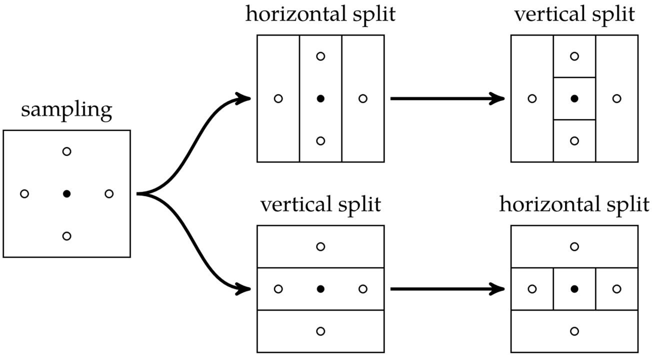


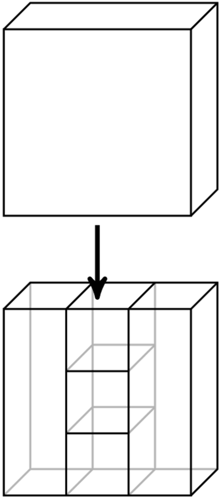
y
x
interval half-width

Multivariate DIRECT

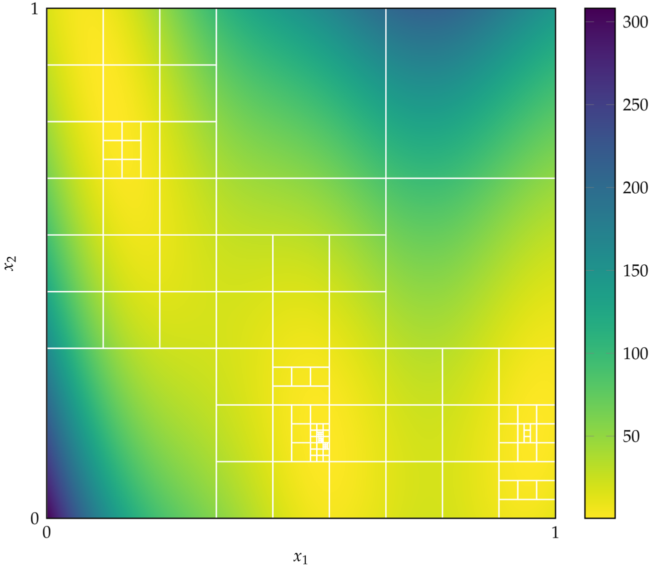
- intervals $\rightarrow$ hyper-rectangles
- normalizes the search space to be the unit hypercube
- divide the rectangles into thirds along the axis directions
- larger rectangles for the points with lower function evaluations
- larger rectangles are prioritized for additional splitting
- when splitting a region without equal side lengths, only the longest dimensions are split

Multivariate DIRECT





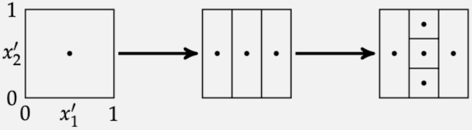
Multivariate DIRECT



Consider using DIRECT to optimize the flower function (appendix B.4) over $x_1 \in [-1, 3]$, $x_2 \in [-2, 1]$. The function is first normalized to the unit hypercube such that we optimize $x_1', x_2' \in [0, 1]$:

$$f(x_1', x_2') = \text{flower}(4x_1' - 1, 3x_2' - 2)$$

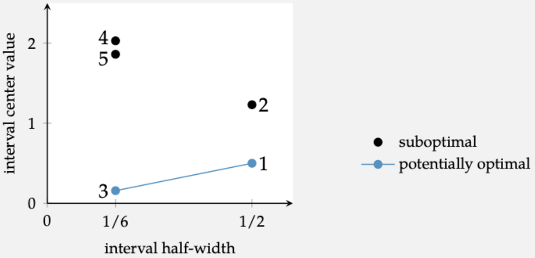
The objective function is sampled at $[0.5, 0.5]$ to obtain 0.158. We have a single interval with center $[0.5, 0.5]$ and side lengths $[1, 1]$. The interval is divided twice, first into thirds in x_1' and then the center interval is divided into thirds in x_2'.



the interval width can only take on powers of one-third, hence the interval half-width is $\left\|\frac{a-b}{2}\right\|_2 = \left\|\frac{3^{-h}}{2}\right\|_2$ where h is the depth of the rectangle

We now have five intervals:

interval	center	side lengths	vertex distance	center value
1	$[1/6, 3/6]$	$[1/3, 1]$	0.527	0.500
2	$[5/6, 3/6]$	$[1/3, 1]$	0.527	1.231
3	$[3/6, 3/6]$	$[1/3, 1/3]$	0.236	0.158
4	$[3/6, 1/6]$	$[1/3, 1/3]$	0.236	2.029
5	$[3/6, 5/6]$	$[1/3, 1/3]$	0.236	1.861



We next split on the two intervals centered at $[1/6, 3/6]$ and $[3/6, 3/6]$.

Summary

- Direct methods rely solely on the objective function and do not use derivative information.
- Cyclic coordinate search optimizes one coordinate direction at a time.
- Powell's method adapts the set of search directions based on the direction of progress.
- Hooke-Jeeves searches in each coordinate direction from the current point using a step size that is adapted over time.
- Generalized pattern search is similar to Hooke-Jeeves, but it uses fewer search directions that positively span the design space.
- The Nelder-Mead simplex method uses a simplex to search the design space, adaptively expanding and contracting the size of the simplex in response to evaluations of the objective function.
- The divided rectangles algorithm extends the Shubert-Piyavskii approach to multiple dimensions and does not require specifying a valid Lipschitz constant.

8. Stochastic Methods

Outline

- Employ randomness strategically to help explore design space
- Randomness can help escape local minima
- Increases chance of searching near the global minimum
- Typically rely on pseudo-random number generators to ensure repeatability
- Control over randomness and the exploration vs exploitation trade off.

Noisy Descent

- Saddle points, where the gradient is very close to zero, can cause descent methods to select step sizes that are too small to be useful
- add Gaussian noise at each descent step

$$\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha \nabla f(\boldsymbol{x}_k) + \boldsymbol{\epsilon}_k \qquad \boldsymbol{\epsilon}_k \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$$

- $\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$ multivariate normal distribution (see next slide) with covariance matrix $\boldsymbol{\Sigma} = \sigma_k^2 I$.
- $\sigma_k = 1/k$, so that the noise decreases over time and the method converges to a local minimum

Multivariate normal distribution
Probability density function

- 1-dimensional:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) .$$

- 2-dimensional:

$$f(x,y) = \frac{1}{2\pi\sigma_X\sigma_Y\sqrt{1-\rho^2}} \exp\left(-\frac{1}{2\left[1-\rho^2\right]}\left[\left(\frac{x-\mu_X}{\sigma_X}\right)^2 - 2\rho\left(\frac{x-\mu_X}{\sigma_X}\right)\left(\frac{y-\mu_Y}{\sigma_Y}\right) + \left(\frac{y-\mu_Y}{\sigma_Y}\right)^2\right]\right)$$

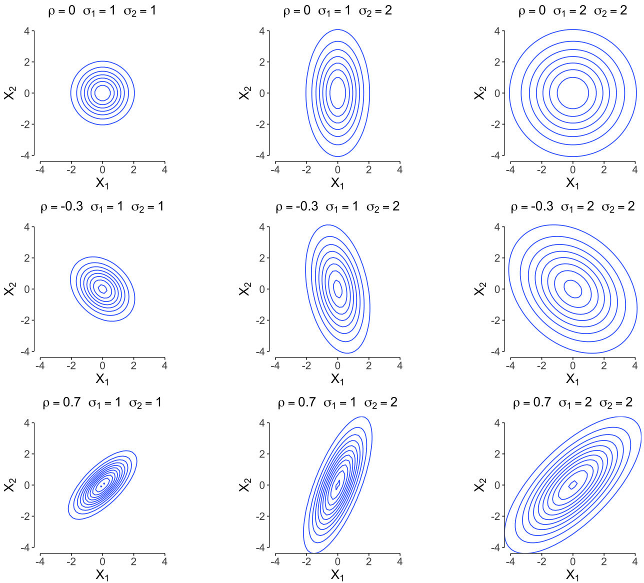
where ρ is the correlation between X and Y and where $\sigma_X > 0$ and $\sigma_Y > 0$. In this case,

$$\boldsymbol{\mu} = \begin{pmatrix} \mu_X \\ \mu_Y \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_X^2 & \rho\sigma_X\sigma_Y \\ \rho\sigma_X\sigma_Y & \sigma_Y^2 \end{pmatrix}.$$

- d dimensional:

$$f_{\mathbf{X}}(x_1, \ldots, x_d) = \frac{\exp\left(-\frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)^{\mathrm{T}} \boldsymbol{\Sigma}^{-1}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)\right)}{\sqrt{(2\pi)^k|\boldsymbol{\Sigma}|}}$$

with symmetric covariance matrix $\boldsymbol{\Sigma}$ positive definite.

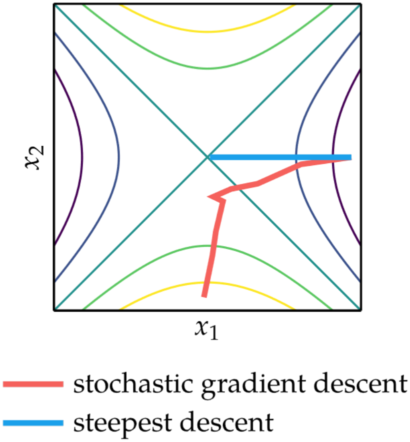

$\rho = 0 \quad \sigma_1 = 1 \quad \sigma_2 = 1$
$\rho = 0 \quad \sigma_1 = 1 \quad \sigma_2 = 2$
$\rho = 0 \quad \sigma_1 = 2 \quad \sigma_2 = 2$
$\rho = -0.3 \quad \sigma_1 = 1 \quad \sigma_2 = 1$
$\rho = -0.3 \quad \sigma_1 = 1 \quad \sigma_2 = 2$
$\rho = -0.3 \quad \sigma_1 = 2 \quad \sigma_2 = 2$
$\rho = 0.7 \quad \sigma_1 = 1 \quad \sigma_2 = 1$
$\rho = 0.7 \quad \sigma_1 = 1 \quad \sigma_2 = 2$
$\rho = 0.7 \quad \sigma_1 = 2 \quad \sigma_2 = 2$
X_1
X_2
-4 -2 0 2 4

Stochastic Gradient Descent

- evaluates gradients using randomly chosen subsets of the training data (**batches**)
- significantly less expensive computationally than calculating the true gradient at every iteration and yields same effect as noisy gradient approximation
- helping traverse past saddle points Convergence guarantees for stochastic gradient descent require that the positive step sizes be chosen such that:

$$\sum_{k=1}^{\infty} \alpha_k = \infty \qquad \sum_{k=1}^{\infty} \alpha_k^2 < \infty$$

- ensure that the step sizes decrease and allow the method to converge, but not too quickly so as to become stuck away from a local minimum

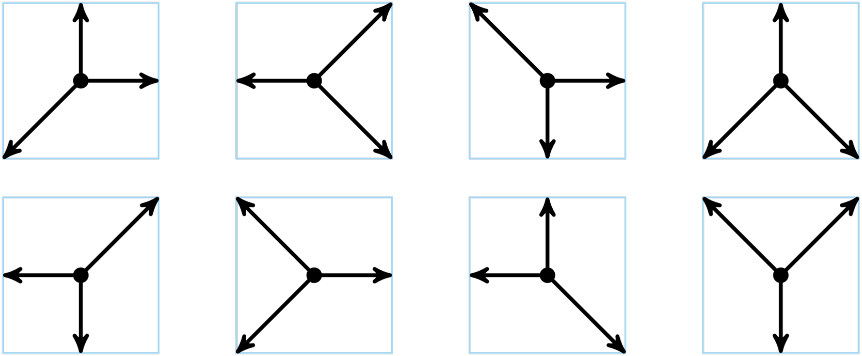


x_1
x_2
stochastic gradient descent
steepest descent

Mesh Adaptive Direct Search

- Similar to generalized pattern search but uses random positive spanning directions

- Example: set of positive spanning sets constructed from nonzero directions $d_1, d_2 \in \{-1, 0, 1\}$.



- Construct lower triangular matrix L sampling from:

$$\{-1/\sqrt{\alpha_k}+1, -1/\sqrt{\alpha_k}+2, \ldots, 1/\sqrt{\alpha_k}-1\}$$

- permute rows and columns of L randomly to obtain a matrix D whose columns correspond to n directions that linearly span $\mathbb{R}^n$. The maximum magnitude among these directions is $1/\sqrt{\alpha_k}$
- add one additional direction $\boldsymbol{d}_{n+1} = -\sum_{i=1}^{n} \boldsymbol{d}_i$ or add n additional directions $\boldsymbol{d}_{n+j} = -\boldsymbol{d}_j$
- $$\alpha_{k+1} \leftarrow \begin{cases} \alpha_k/4 & \text{if no improvement was found in this iteration} \\ \min(1, 4\alpha_k) & \text{otherwise} \end{cases}$$
- If $f(\boldsymbol{x}_k = \boldsymbol{x}_{k-1} + \alpha\boldsymbol{d}) < f(\boldsymbol{x}_{k-1})$, then the queried point is $\boldsymbol{x}_{k-1} + 4\alpha\boldsymbol{d} = \boldsymbol{x}_k + 3\alpha\boldsymbol{d}$

Simulated Annealing

- often used on functions with many local minima due to its ability to escape local minima.

- a candidate transition from $\boldsymbol{x}$ to $\boldsymbol{x}'$ is sampled from a transition distribution T, eg, multivariate Gaussian

$$\boldsymbol{x}' = \boldsymbol{x} + \epsilon \qquad \epsilon \sim T$$

- **Metropolis acceptance criterion:**

$$\boxed{p(\boldsymbol{x}, \boldsymbol{x}') = \begin{cases} 1 & \text{if } \Delta \leq 0 \\ e^{-\frac{\Delta}{t_k}} & \text{if } \Delta > 0 \end{cases}}$$

where $\Delta = f(\boldsymbol{x}') - f(\boldsymbol{x})$ is the change in the objective function value and t_k is the **temperature** at iteration k.

Annealing Plan

- a logarithmic annealing schedule

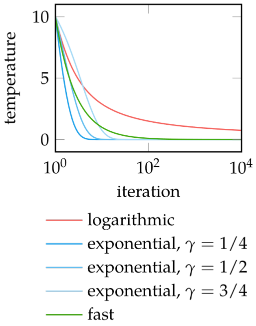
$$t_k = t_0 \frac{\ln(2)}{\ln(k+1)}$$

  guaranteed to asymptotically reach the global optimum under certain conditions, but it can be slow in practice.
- exponential annealing schedule, more common, uses a simple decay factor:

$$t_{k+1} = \gamma t_k$$

- fast annealing

$$t_k = \frac{t_0}{k}$$



Simulated Annealing

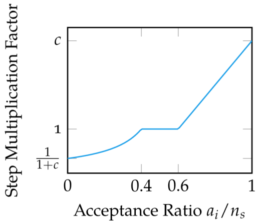
- Corana et al 1987 introduced variable step-size $\boldsymbol{v}$ (separate directional components)
- cycle of random moves, one in each direction

  $$\boldsymbol{x}' = \boldsymbol{x} + r v_i \boldsymbol{e}_i$$

  where r is randomly sampled from $\{-1, 1\}$
- after n_s cycles, step size is adjusted according to

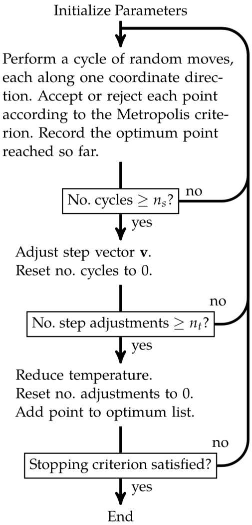
  $$v_i = \begin{cases} v_i \left(1 + c_i \frac{a_i/n_s - 0.6}{0.4}\right) & \text{if } a_i > 0.6 n_s \\ v_i \left(1 + c_i \frac{0.4 - a_i/n_s}{0.4}\right)^{-1} & \text{if } a_i < 0.4 n_s \\ v_i & \text{otherwise} \end{cases}$$

  $\boldsymbol{a}$: accepted steps in each direction; c_i: typically 2.



regulates the ratio of accepted-to-rejected points to about 50%.

Simulated Annealing



- Temperature reduction occurs every n_t step adjustments, which is every $n_s \cdot n_t$ cycles
- termination when the temperature sinked low and no improvement expected or when no movement more than ϵ in last n_ϵ iterations

Cross-Entropy Method

- Maintains explicit probability distribution over design space often called a **proposal distribution**
- Requires choosing a family of parameterized distributions
- At each iteration, a set of design points are **conditionally independently** sampled from the proposal distribution; these are evaluated and ranked
- The best-performing subset of samples, called **elite samples**, are retained
- The proposal distribution parameters are then **updated** based on the elite samples, and the next iteration begins

Cross-Entropy Method

- Cross-entropy is a measure of divergence between two probability distributions p and q (related to Kullback-Leibler divergence)
- Here we measure cross-entropy in a case where one distribution (the one of optimal solutions) is unknown.
- A model is created and then its cross-entropy is measured on the elite set to assess how accurate the model is in predicting this set.
- Let q be the true distribution of the optimal solutions, and p the distribution of solutions as predicted by the model. Since the true distribution is unknown, cross-entropy cannot be directly calculated. Instead, an estimate of cross-entropy is:

$$H(T, p) = -\sum_{i=1}^{N} \frac{1}{N} \log_2 p(\boldsymbol{x}_i)$$

  where N is the size of the elite set, and $p(\boldsymbol{x})$ is the probability of solution $\boldsymbol{x}$ estimated from the training set T.

Cross-Entropy Method

cross-entropy $\equiv$ Maximum likelihood estimation

- A widely used frequentist estimator is maximum likelihood, in which $\boldsymbol{\theta}$ is set to the value that maximizes the **likelihood function** $p(\mathbf{x} \mid \boldsymbol{\theta})$.
- This corresponds to choosing the value of $\boldsymbol{\theta}$ for which the probability of the observed data set is maximized.
- In the machine learning literature, the **negative log of the likelihood function** is called an error function. Because the negative logarithm is a monotonically decreasing function, maximizing the likelihood is equivalent to minimizing the error.
- Suppose our data set consists of N data points $\mathbf{x} = \{x_1, \dots, x_N\}$:

$$\mathcal{L}(\mathbf{x} \mid \boldsymbol{\theta}) = p(\mathbf{x} \mid \boldsymbol{\theta}) = \prod_{i=1}^{N} p(x_i \mid \boldsymbol{\theta}) \qquad \text{likelihood}$$

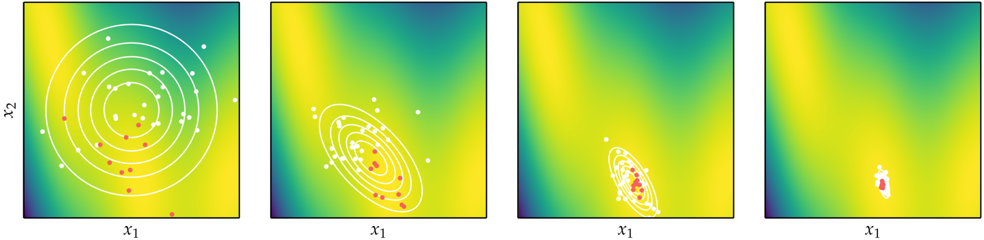
$$\mathcal{E}(\mathbf{x} \mid \theta) = -\log \mathcal{L}(\mathbf{x}) \qquad \text{error function}$$

Cross-Entropy Method

- $$\min_{\boldsymbol{\theta}} \mathcal{E}(\boldsymbol{x} \mid \theta) = \min_{\boldsymbol{\theta}} (-\log \mathcal{L}(\boldsymbol{x})) = -\max_{\boldsymbol{\theta}} \log \mathcal{L}(\boldsymbol{x}) \qquad \text{maximum log-likelihood}$$

$$\min \left(-\sum_{i=1}^{N} \log p(x_i \mid \boldsymbol{\theta}) \right)$$

- hence minimizing the negative of the log-likelihoood is equivalent to minimizing the entropy



Solution of Max Likelihood

Multivariate Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance Σ has a closed solution for the maximum likelihood estimation of the parameters:

$$\boldsymbol{\mu}_{k+1} = \frac{1}{m_e} \sum_{i=1}^{m_e} \mathbf{x}_i$$

$$\Sigma_{k+1} = \frac{1}{m_e} \sum_{i=1}^{m_e} (\mathbf{x}_i - \boldsymbol{\mu}_{k+1})(\mathbf{x}_i - \boldsymbol{\mu}_{k+1})^T$$

m_e is the number of elite samples, and $\mathbf{x}_i$ are the elite samples. The mean is updated to be the average of the elite samples, and the covariance is updated to be the average of the outer products of the centered elite samples.

Disadvantage of the Gaussian distribution: it is unimodal

Natural Evolution Strategies

- Similar to cross-entropy method, except instead of parameterizing distribution based on elite samples, it is optimized using **gradient descent**

- The aim is to minimize the expectation

$$E_{\boldsymbol{x}\sim p(\cdot|\boldsymbol{\theta})}[f(\boldsymbol{x})] = \int_{\mathbb{R}} f(\boldsymbol{x})p(\boldsymbol{x} \mid \boldsymbol{\theta})d\boldsymbol{x}$$

$$\begin{aligned}
\nabla_{\theta} \mathbb{E}_{\mathbf{x}\sim p(\cdot|\theta)}[f(\mathbf{x})] &= \int \nabla_{\theta} p(\mathbf{x} \mid \theta) f(\mathbf{x})\, d\mathbf{x} \\
&= \int \frac{p(\mathbf{x} \mid \theta)}{p(\mathbf{x} \mid \theta)} \nabla_{\theta} p(\mathbf{x} \mid \theta) f(\mathbf{x})\, d\mathbf{x} \\
&= \int p(\mathbf{x} \mid \theta) \nabla_{\theta} \log p(\mathbf{x} \mid \theta) f(\mathbf{x})\, d\mathbf{x} \\
&= \mathbb{E}_{\mathbf{x}\sim p(\cdot|\theta)}[f(\mathbf{x}) \nabla_{\theta} \log p(\mathbf{x} \mid \theta)] \\
&\approx \frac{1}{m} \sum_{i=1}^{m} f(\mathbf{x}^{(i)}) \nabla_{\theta} \log p(\mathbf{x}^{(i)} \mid \theta)
\end{aligned}$$

Natural Evolution Strategies

The distribution parameter gradient is estimated from the set of function evaluations

Input: $f, \boldsymbol{\theta}, k_{MAX}, N = 100, \alpha = 0.01$
Output: $\boldsymbol{\theta}$
for k in $1, \ldots, k_{MAX}$ **do**
  Let $X = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ be a conditionally independent sample of size N from $p(\boldsymbol{\theta})$;
  $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \alpha \frac{1}{N} \sum_{i=1}^{N} f(\boldsymbol{x}_i) \nabla_{\boldsymbol{\theta}} \log p(\boldsymbol{x}_i, \boldsymbol{\theta}_k)$;

Natural Evolution Strategies

The multivariate normal distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$ is a popular distribution family due to having analytic solutions. The likelihood in d dimensions has the form

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = (2\pi)^{-\frac{d}{2}} |\boldsymbol{\Sigma}|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)$$

where $|\boldsymbol{\Sigma}|$ is the determinant of $\boldsymbol{\Sigma}$. The log likelihood is

$$\log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\boldsymbol{\Sigma}| - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$$

The parameters can be updated using their log likelihood gradients:

$$\nabla_{(\boldsymbol{\mu})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$$

$$\nabla_{(\boldsymbol{\Sigma})} \log p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{2} \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} - \frac{1}{2} \boldsymbol{\Sigma}^{-1}$$

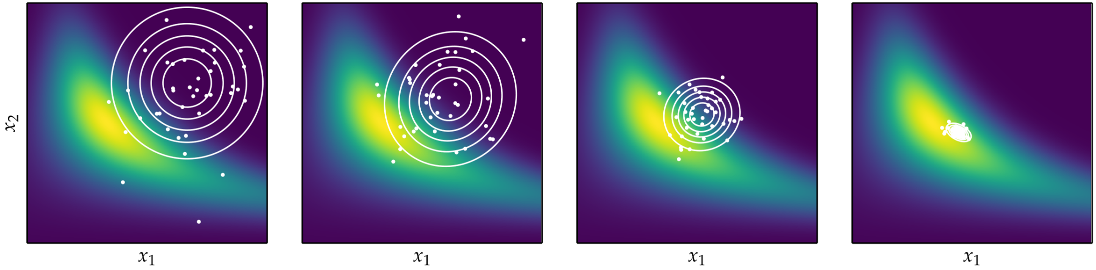
The term $\nabla_{(\boldsymbol{\Sigma})}$ contains the partial derivative of each entry of $\boldsymbol{\Sigma}$ with respect to the log likelihood.

Natural Evolution Strategies

Directly updating $\mathbf{\Sigma}$ may not result in a positive definite matrix, as is required for covariance matrices. One solution is to represent $\mathbf{\Sigma}$ as a product $\mathbf{A}^\top\mathbf{A}$, which guarantees that $\mathbf{\Sigma}$ remains positive semidefinite, and then update $\mathbf{A}$ instead. Replacing $\mathbf{\Sigma}$ by $\mathbf{A}^\top\mathbf{A}$ and taking the gradient with respect to $\mathbf{A}$ yields:

$$\nabla_{(\mathbf{A})} \log p(\mathbf{x} \mid \mu, \mathbf{A}) = \mathbf{A}\left[\nabla_{(\mathbf{\Sigma})} \log p(\mathbf{x} \mid \mu, \mathbf{\Sigma}) + \nabla_{(\mathbf{\Sigma})} \log p(\mathbf{x} \mid \mu, \mathbf{\Sigma})^\top\right]$$

Natural Evolution Strategies



Covariance Matrix Adaptation Evolutionary Strategy (CMA-ES)

- Same approach as natural evolution strategy and cross entropy method, but the proposal distribution is a multivariate Gaussian parameterized by a covariance matrix.
- At every iteration, m designs are sampled from the multivariate Gaussian:

  $$\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \Sigma)$$

  parameters: mean vector $\boldsymbol{\mu}$, a covariance matrix Σ, and an additional step-size scalar σ.
- The covariance matrix only increases or decreases in a single direction with every iteration, whereas σ is adapted to control the overall spread of the distribution.
- Design points are sorted $f(x^{(1)}) \leq f(x^{(2)}) \leq \ldots \leq f(x^{(m)})$.
- A new mean vector $\boldsymbol{\mu}_{k+1}$ is formed using a weighted average of the first m_e-elite sampled designs:

  $$\boldsymbol{\mu}(k+1) \leftarrow \sum_{i=1}^{m_e} w_i x^{(i)}$$

CMA-ES

- the first m_e elite weights sum to 1, and all the weights approximately sum to 0 and are ordered largest to smallest

- positive and negative weigths, more aggressive shift

- The step size σ is updated using a cumulative vector $\boldsymbol{p}_1$ that tracks steps over time:
  Comparing the length of $\boldsymbol{p}_1$ to its expected length under random selection provides the mechanism by which σ is increased or decreased.

- covariance matrix is updated using a cumulative vector $\boldsymbol{p}_2$ and adjusted weights;
  the update consists of three components: the previous covariance matrix Σ_k, a rank-one update, and a rank-$\boldsymbol{\mu}$ update
  Rank-one updates using the cumulation vector allow for correlations between consecutive steps to be exploited

- covariance estimated around original mean $\boldsymbol{\mu}_k$ (cross-entropy did it around new mean $\boldsymbol{\mu}_{k+1}$)

Summary

- Stochastic methods employ random numbers during the optimization process
- Simulated annealing uses a temperature that controls random exploration and which is reduced over time to converge on a local minimum
- The cross-entropy method and evolution strategies maintain proposal distributions from which they sample in order to inform updates
- Natural evolution strategies uses gradient descent with respect to the log likelihood to update its proposal distribution
- Covariance matrix adaptation is a robust and sample-efficient optimizer that maintains a multivariate Gaussian proposal distribution with a full covariance matrix

Outline

Terminology in COCO Benchmarking

- **function** a parametrized mapping $f : \mathbb{R}^n \to \mathbb{R}$, Functions are parametrized such that different instances of the "same" function are available, e.g. translated or shifted versions.

- **quality indicator measure** The indicator assigns to any solution a real value. Our simplest quality indicator measure is the minimum of all so-far-observed f-values. Hence, by convention and abuse of naming, the 'quality' indicator is minimized.

- **problem** a specific function instance on which a solver is run. In the context of performance assessment, a target value is added to define a problem. A problem is considered as solved when the quality indicator value reaches this target, which is always given as a deviation from a (supposedly) optimal value.

- **runtime** the number of function evaluations on a problem until a prescribed target value is hit.

- **suite** a collection of problems in different dimensions. It typically consists of 1000 to 5000 problems (number of dimensions $\times$ number of functions $\times$ number of instances)

Benchmarking in the COCO Platform

- Functions divided in suites.
- Functions, f_i, within suites are distinguished by their identifier $i = 1, 2, \ldots$.
- parametrized by the (input) dimension, n, and
- instance number, j. (j as an index to a continuous parameter vector setting, eg, search space translations and rotations).

$$f_i^j \equiv f[n, i, j] : \mathbb{R}^n \to \mathbb{R} \qquad \boldsymbol{x} \mapsto f_i^j(\boldsymbol{x}) = f[n, i, j](\boldsymbol{x}).$$

- Varying n or j leads to a variation of the same function i of a given suite.
- Fixing n and j of function f_i defines an **optimization problem instance** $(n, i, j) \equiv (f_i, n, j)$ that can be presented to the solver.

Why?

Varying the instance parameter j represents a natural randomization for experiments in order to:

- generate repetitions on a single function for deterministic solvers, making deterministic and non-deterministic solvers directly comparable (both are benchmarked with the same experimental setup)
- average away irrelevant aspects of the function definition
- alleviate the problem of overfitting, and
- prevent exploitation of artificial function properties

BBOB Functions

Real-Parameter Black-Box Optimization Benchmarking

- All benchmark functions are scalable with the dimension.
- Most functions have no specific value of their optimal solution (they are randomly shifted in $\boldsymbol{x}$-space).
- All functions have an artificially chosen optimal function value (they are randomly shifted in f-space).

Target Values and Runtime

- Runtime of a solver on a problem is the hitting time condition.
- define a non-increasing quality indicator measure and prescribe a set of **target values**, t.
- target values are compared with the best so-far-seen f-value.
- For a single run, the solver run is **successful** on the problem instance (f_i, n, j) when the best-so-far f-value reaches the target value t.
- COCO collects hundreds of different target values from each single run.
- targets $t(i, j)$ depend on the problem instance in a way to make problems comparable
- typically, target values are set to known or estimated optimal solution plus an added **precision**
- **runtime** is the number of f-evaluations needed to solve the problem $(f_i, n, j, t(i, j))$.
- only runtimes to comparable target values can be aggregated among problem instances.

Simulated Restarts

- If a solver does not hit the target t in a given single run, the run is considered to be unsuccessful.
- The runtime of this single run remains undefined but is bounded from below by the number of evaluations conducted during the run $\tau \in [T, \infty]$
- T depends on the termination condition encountered. It can be the budget of evaluations.
- For hard problem instances COCO uses **budget-based target values**:
  For any given budget, COCO selects from the finite set of recorded target values the easiest (i.e., largest) target for which the expected runtime of all solvers (ERT) exceeds the budget.
- With unsuccessful runs: draw further runs from the set of tried problem instances, uniformly at random with replacement, until find an instance, j, for which $(f_i, n, j, t(i, j))$ is solved.
  the runtime is then the sum of the overall time spend and associated to the initially unsolved problem instance.
  ```
  print: '|'if problem.final_target_hit, ':'if restarted and '.'otherwise↪
  ↪.
  ```

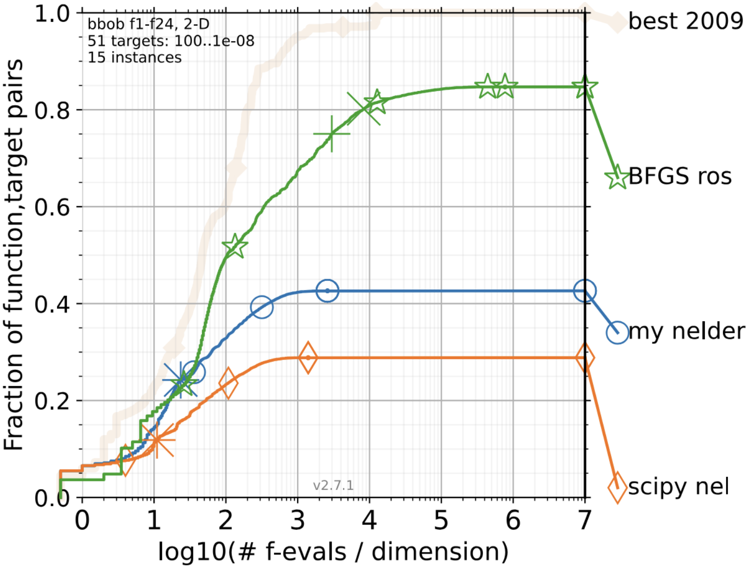
Aggregation

- Aggregation is to compute a statistical summary over a set or subset of problem instances over which we assume a uniform distribution

- If we can distinguish between problems easily, for example, according to their input dimension, we can use the information to select the solver, hence not worth aggregating data

- Empirical cumulative distribution functions of runtimes (**runtime ECDFs**)
  - Absolute distributions vs Performance profiles (ECDFs of runtimes relative to the respective best solver)
  - aggregate runtimes from several targets per function (!?)

- arithmetic average, as an estimator of the expected runtime. The estimated expected runtime of the restarted solver, ERT, is often plotted against dimension to indicate scaling with dimension. alternatives: average of log-runtimes $\equiv$ geometric average or shifted geometric mean


bbob f1-f24, 2-D
51 targets: 100..1e-08
15 instances
best 2009
BFGS ros
my nelder
scipy nel
v2.7.1
Fraction of function,target pairs
log10(# f-evals / dimension)
1.0
0.8
0.6
0.4
0.2
0.0
0
1
2
3
4
5
6
7

Reference

Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar & Dimo Brockhoff (2021) COCO: a platform for comparing continuous optimizers in a black-box setting, Optimization Methods and Software, 36:1, 114-144, DOI: 10.1080/10556788.2020.1808977

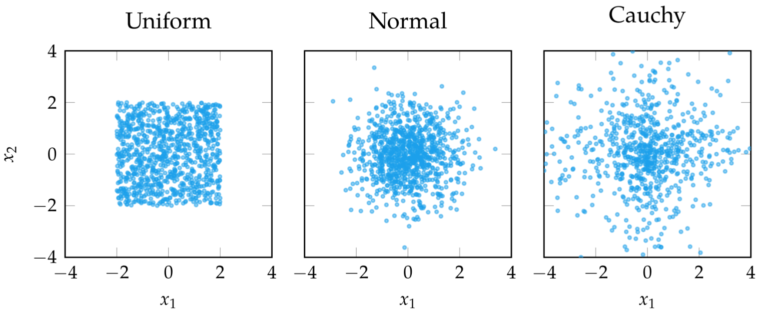
9. Population-based Methods

Population Methods

- Instead of optimizing a single design point, population methods optimize a collection of **individuals**
- A large number of individuals prevents algorithm from being stuck in a local minimum
- Useful information can be shared between individuals
- Stochastic in nature
- Easy to parallelize

Initialization

- Population methods begin with an initial population
- Common initializations are uniform, normal distribution, and Cauchy distribution
- But also space filling designs (later)



Genetic Algorithms

- Inspired by biological evolution where the fittest individuals pass their genetic information to the next generation
- Individuals are interpreted as **chromosomes**
- The fittest individuals are determined by **selection**
- The next generation is formed by selecting the fittest individuals and performing **crossover** and **mutation**

Genetic Algorithm Template

Input: f
Output: a "good" solution $\boldsymbol{x}$
determine initial population `sp`;
while termination criterion is not satisfied **do**
    generate set `spr` of new candidate solutions by **recombination**;
    generate set `spm` of new candidate solutions from `spr` and `sp` by **mutation**;
    select new population `sp` from candidate solutions in `sp`, `spr`, and `spm`;

Genetic Algorithms: Chromosomes

- Simplest representation is the binary string chromosome

  but the process of decoding a binary string and producing a design point is not always straightforward.

- Chromosomes are more commonly represented as real-valued chromosomes which are simply real-valued vectors

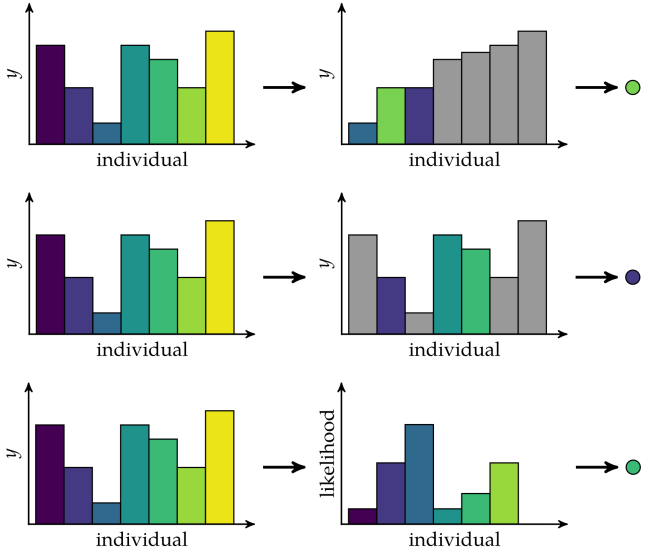
- Typically initialized randomly

Genetic Algorithms: Selection

- Determining which individuals pass their genetic information on to the next generation
  choosing chromosomes to use as parents for the next generation
- **Truncation selection**: truncate the lowest performers
- **Tournament selection**: selects fittest out of k randomly chosen individuals
- **Roulette Wheel selection**: individuals are chosen with probability proportional to their fitness

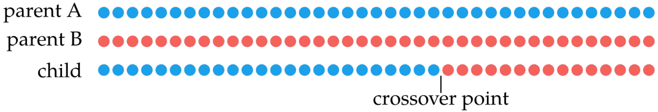
Genetic Algorithms: Selection

Truncation Selection
Tournament Selection
Roulette Wheel Selection

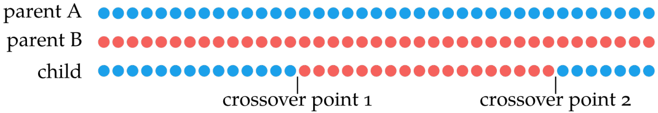


Genetic Algorithms: Crossover

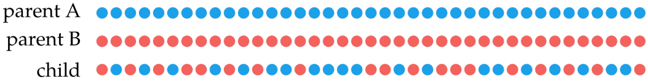
- Combines the chromosomes of the parents to form children
- Single-point crossover: swap occurs after single crossover point



- Two-point crossover: two crossover points



- Uniform crossover: each bit has 50% chance of crossover



- real values are linearly interpolated between the parents' values $\boldsymbol{x}_a$ and $\boldsymbol{x}_b$: $\boldsymbol{x} \leftarrow (1-\lambda)\boldsymbol{x}_a + \lambda \boldsymbol{x}_b$

Genetic Algorithms: Mutation

- Mutation supports exploration of new areas of design space
- Each bit or real-valued element has a probability of being flipped or modified by noise
- The probability of an element mutating is called **mutation rate**

Selection of the New Population

Determines population for next cycle (**generation**) of the algorithm by selecting individual candidate solutions from

- current population +
- new candidate solutions from recombination and mutation

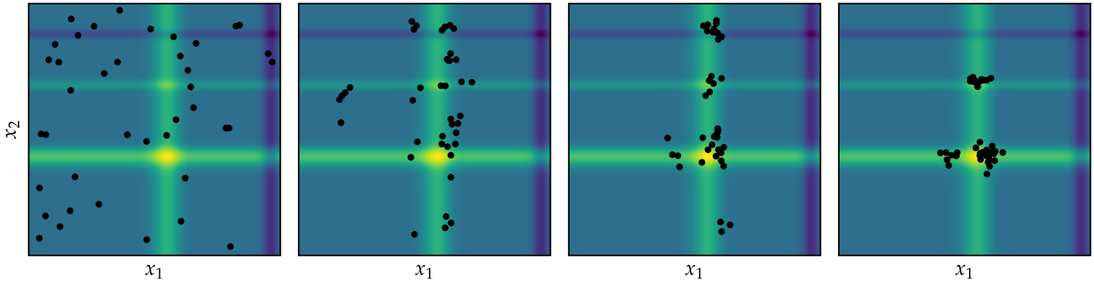
Goal: Obtain population of high-quality solutions while maintaining population diversity.
Survival of the fittest and maintenance of diversity (duplicates avoided)

Typical strategies:

- Generational Replacement (λ, μ): $\lambda \leftarrow \mu$
- Elitist strategy $(\lambda + \mu)$ the best candidates are always selected
- Steady state (most common) only a small number of least fit individuals is replaced

Genetic Algorithms

Genetic algorithm with truncation selection, single point crossover, and Gaussian mutation applied to Michalewicz function



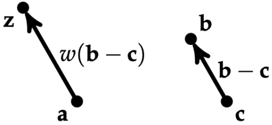
Differential Evolution

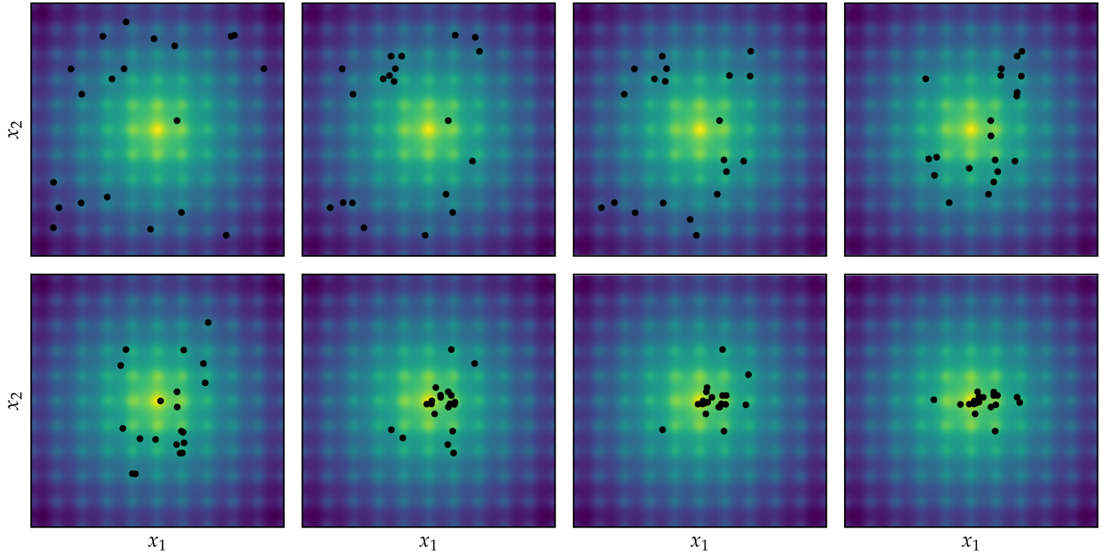
Improves each individual $\boldsymbol{x}$ by recombining other individuals according to a simple formula

1. Choose three random, distinct individuals $\boldsymbol{a}$, $\boldsymbol{b}$, and $\boldsymbol{c}$
2. Construct interim design $\boldsymbol{z} = \boldsymbol{a} + w(\boldsymbol{b} - \boldsymbol{c})$
3. Choose a random dimension to optimize in
4. Construct candidate $\boldsymbol{x}'$ via binary crossover of $\boldsymbol{x}'$ and $\boldsymbol{z}$

$$x'_i = \begin{cases} z_i & \text{if } i = j \text{ or with probability } p \\ x_i & \text{otherwise} \end{cases}$$

5. Insert better design between $\boldsymbol{x}$ and $\boldsymbol{x}'$ into next generation





x_2
x_2
x_1
x_1
x_1
x_1

Particle Swarm Optimization

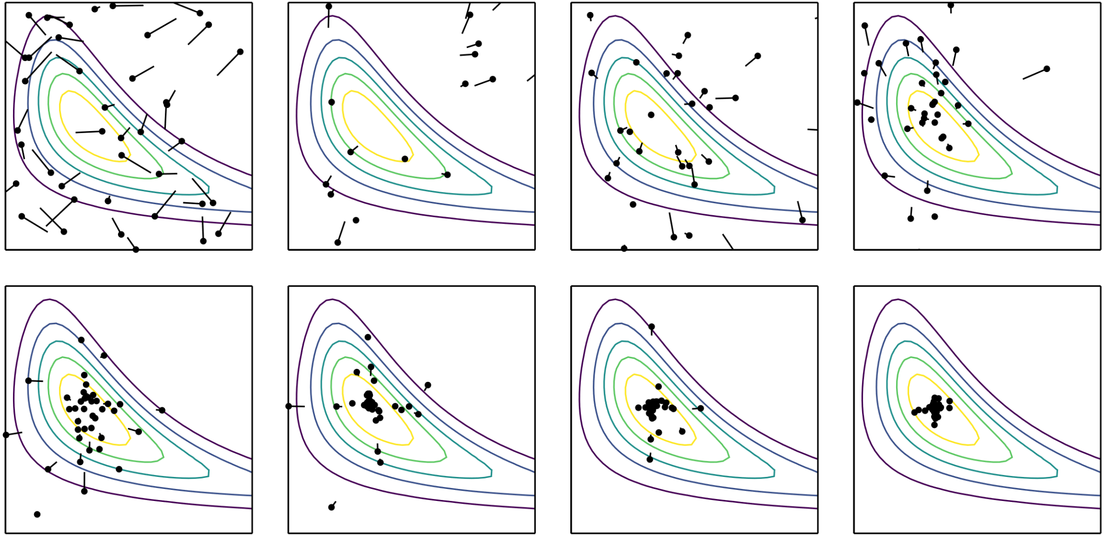
- Each individual, or particle, tracks the following
  - Current position
  - Current velocity
  - Best position seen so far by the particle
  - Best position seen so far by any particle
- At each iteration, these factors produce force and momentum effects to determine the movement of a particle p from the population:

$$x_i^p \leftarrow x_i^p + v_i^p$$

$$v_i^p \leftarrow w v_i^p + c_1 r_1 \left(x_i^{p,best} - x_i^p\right) + c_2 r_2 \left(x_i^{best} - x_i^p\right)$$

$\mathbf{x}_{p,best}$ is the best location among those seen by the particle; $\mathbf{x}_{best}$ is the best location found so far over all particles; w, c_1, and c_2 are parameters; and r_1 and r_2 are random numbers drawn from $U(0,1)$

Particle Swarm Optimization



...

The Evolutionary Computation Bestiary
`https://fcampelo.github.io/EC-Bestiary/`

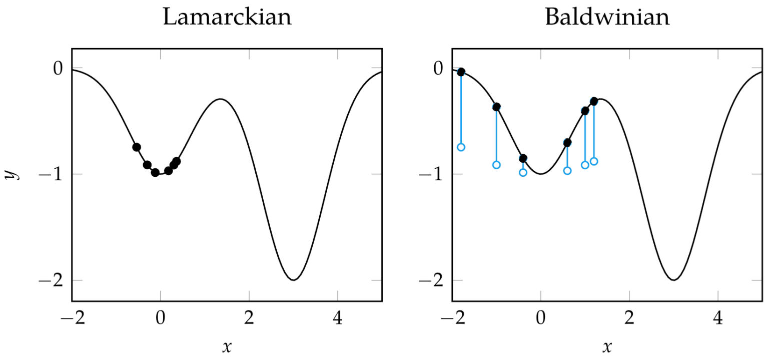
Hybrid Methods

- So far, we have discussed Darwinian Evolution: random variation + natural selection
  Some giraffes are born with longer necks. They access more food. They reproduce more. Long-neck genes spread.
- Generally, these methods are good at finding the best regions in design space, but do not perform as well as descent methods near the minimizer
- Hybrid methods try to leverage the strength of both methods
- Two hybrid approaches
  - Lamarckian evolution: traits that an organism acquires during its lifetime are directly inherited by offspring.
    Giraffes stretch their necks to reach higher leaves. Their necks become longer during life. Their offspring inherit the longer neck.
  - Baldwinian evolution: Learning does not directly change genes, but learning influences which genes are selected. Learning guides evolution indirectly.
    Giraffes stretch their necks to reach higher leaves. Their necks become longer during life. They are more likely to survive and reproduce. Their offspring inherit the genes that make it easier to stretch their necks.

Hybrid Methods

- Lamarckian evolution
  Performs regular descent method update on each individual

- Baldwinian evolution
  Uses value of descent method update to augment the objective value of each design point



Hybird Genetic Algorithm Template

Input: f
Output: a "good" solution$\boldsymbol{x}$
determine initial population `sp`;
perform local search on `sp`
while termination criterion is not satisfied **do**
  generate set `spr` of new candidate solutions by **recombination**;
  perform local search on `spr`
  generate set `spm` of new candidate solutions from `spr` and `sp` by **mutation**;
  perform local search on `spm`
  select new population `sp` from candidate solutions in `sp`, `spr`, and `spm`;

Summary

- Population methods use a collection of individuals in the design space to guide progression toward an optimum

- Genetic algorithms leverage selection, crossover, and mutations to produce better subsequent generations

- Differential evolution, particle swarm optimization, (the firefly algorithm, and cuckoo search) include rules and mechanisms for attracting design points to the best individuals in the population while maintaining suitable state space exploration

- Population methods can be extended with local search approaches to improve convergence

10. Sampling Plans

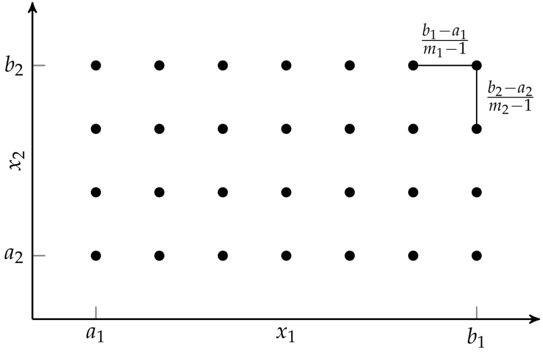
Sampling Plans

- In all nonlinear non convex optimization, to generate good initial design points
- With computationally costly functions, to create an initial set of design points from where to build a **surrogate models** to optimize in place of the original function
- In hyperparameter tuning

Full Factorial Design

- Factors and levels, terms from the field of Experimental Design in Statistics
- Uniform and evenly spaced samples across domain
- Simple, easy to implement, and covers domain
- Optimization over the points known as **grid search**
- Sample count grows exponentially with dimension: n^m
- Can be coarse and miss local features

$a_i \leq x_i \leq b_i$ for each component i.
grid with m_i samples in the ith dimension



Random Sampling

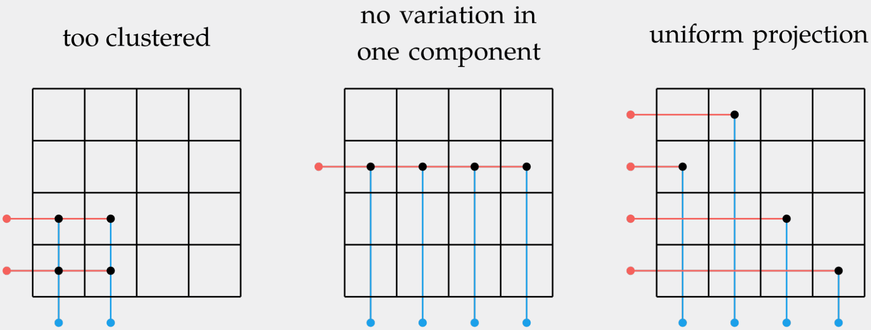
- Uses pseudorandom number generator to define samples according to our desired distribution
- If variable bounds are known, a common choice is independent **uniform distributions** across domains of possible variable values
  $[a_1, b_1] \times \ldots \times [a_n, b_n]$
- Ideally, if enough points are sampled and the right distribution is chosen, the design space will be covered

Uniform Projection Plans

- A **uniform projection plan** is a sampling plan over a discrete grid where the distribution over each dimension is uniform.

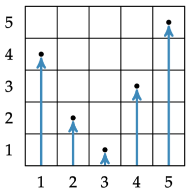
Example

In 2D, $m \times m$ sampling grid (as in full factorial), but, instead of taking all m^2 samples, we want to sample only m positions.



Uniform Projection Plans

Example (Random m-permutations)



$p = 4\ 2\ 1\ 3\ 5$

Example (Latin square)

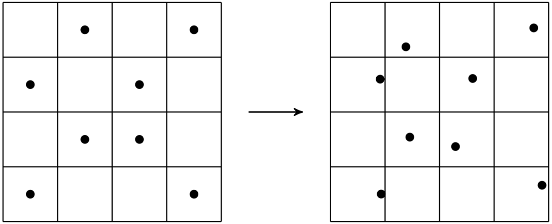
Latin squares are $m \times m$ grids where each row contains each integer 1 through m and each column contains each integer 1 through m. N rooks on a chess board without threatening each other

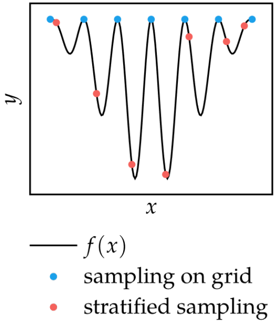
4	1	3	2
1	4	2	3
3	2	1	4
2	3	4	1

Latin-hypercubes are a generalization to any number of dimensions (note that the points

Stratified Sampling

- Each point is sampled uniformly at random within each grid cell instead of the center
- Cells decided by Full Factorial or Uniform Projection Plans
- Can capture details that regularly-spaced samples might miss

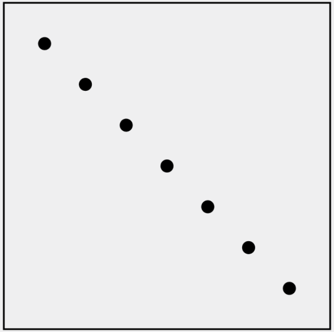




Space Filling Metrics

- A sampling plan may cover a search space fully, but still leave large areas unexplored

Example (Uniform Projection Plan)



- **space-filling metrics** quantify this aspect measuring the degree to which a sampling plan $X \subseteq \mathcal{X}$ fills the design space

Space-Filling Metrics: Discrepancy

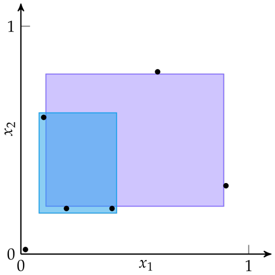
- **Discrepancy**: measure of ability of the sampling plan X to fill a hyper-rectangular design space
- It is given by hyper-rectangular subset $\mathcal{H}$ with the maximum difference between the fraction of samples in $\mathcal{H}$ and the volume of $\mathcal{H}$'s.

$$d(X) = \underset{\mathcal{H}}{\text{supremum}} \left| \frac{\#(X \cap \mathcal{H})}{\#X} - \lambda(\mathcal{H}) \right|$$

$\lambda(\mathcal{H})$ is the n-dimensional volume of $\mathcal{H}$, ie, the product of the side lengths of $\mathcal{H}$

We wish to have a plan X with low discrepancy

Often very difficult to compute directly



d for the purple rectangle is $>$ than d for the blue rectangle

Space-Filling Metrics: Pairwise Distances

- Method of measuring relative space-filling performance of two m-point sampling plans

- Better spread-out plans will have larger pairwise distances:

  1. compute all pairwise distances between all points within each sampling plan

  2. sort the pairwise distances of each set in ascending order

  3. the plan with the first pairwise distance exceeding the other is considered more space-filling

- Suggests simple algorithm:
  1. produce a set of randomly distributed sampling plans,
  2. pick the one with greatest pairwise distances

- Possible also for uniform projection plans, by mutating them with swaps and simulated annealing.

Space-Filling Metrics: Morris-Mitchell Criterion

- Alternative to previously suggested algorithm that simplifies the optimization problem

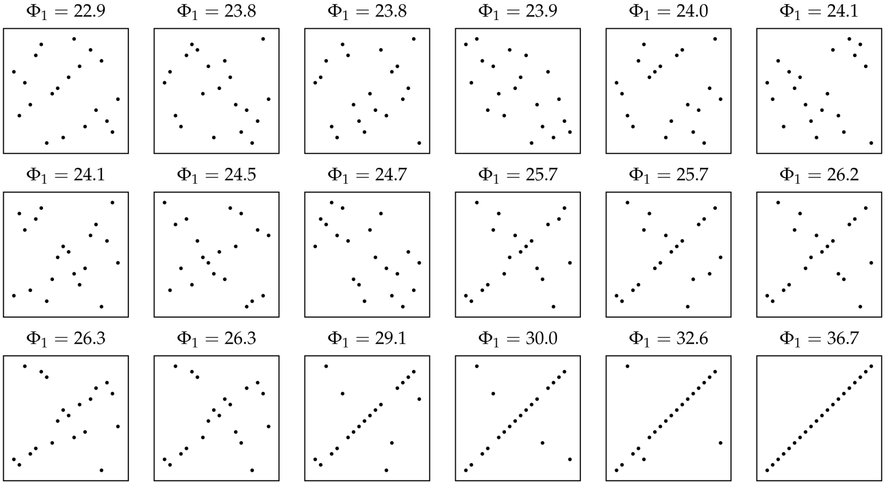
$$\underset{X}{\text{minimize}} \underset{q \in \{1,2,3,10,20,50,100\}}{\text{maximize}} \Phi_q(X)$$

$$\Phi_q(X) = \left(\sum_i d_i^{-q} \right)^{\frac{1}{q}}$$

where d_i is the ith pairwise distance between points in X and $q > 0$ is a tunable parameter. Larger values of q give higher penalties to large distances.

Space-Filling Metrics: Morris-Mitchell Criterion

Uniform projection plans sorted from best to worst according to Φ_1



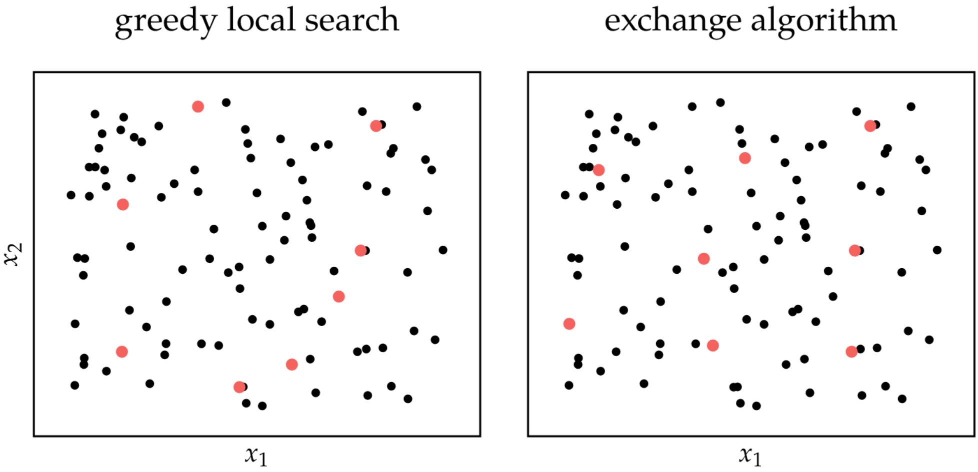
Space-Filling Subsets

- Often, the set of possible sample points is constrained to be a subset of available choices
- A space-filling metric for a subset S within a finite set X is the maximum distance between a point in X and the closest point in S, using a norm to measure distance

$$d_{\max}(X, S) = \underset{\boldsymbol{x} \in X}{\text{maximize}} \; \underset{\boldsymbol{s} \in S}{\text{minimize}} \; \|\boldsymbol{s} - \boldsymbol{x}\|_q$$

- A space-filling subset minimizes this metric
- Often computationally intractable, but heuristics like (repeated) greedy construction and exchange-search often produce acceptable results

Space-Filling Subsets

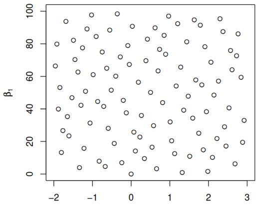


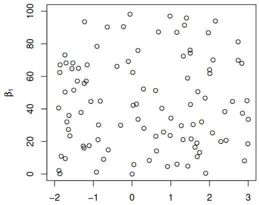
Quasi-Random Sequences

- Also called **low-discrepancy sequences**, **quasi-random sequences** are deterministic sequences that systematically fill a space such that their integral over the space converges as fast as possible

- Used for fast convergence in Monte Carlo integration, which approximates an integral by sampling points in a domain
  Monte Carlo integration: error convergence of $O(1/\sqrt{m})$; quasi-Monte Carlo: $O(1/m)$

- Quasi-random sequences are typically constructed for the unit n-dimensional hypercube, $[0, 1]^n$. Any multidimensional function with bounds on each variable can be transformed into such a hypercube.





Quasi-Random Sequences

- Additive Recurrence: Recursively adds irrational numbers
- Halton Sequence: sequence of fractions generated with coprime numbers
- Sobol Sequence: recursive XOR operation with carefully chosen numbers

Quasi-Random Sequences: Additive Recurrence

- Recursively adds irrational numbers

$$x_{k+1} = x_k + c \pmod 1$$

c irrational

$$c = 1 - \varphi = \frac{\sqrt{5} - 1}{2} \approx 0.618034$$

φ is golden ratio

- We can construct a space-filling set over n dimensions using an additive recurrence sequence for each coordinate, each with its own value of c.

- square roots of the primes are known to be irrational, and can thus be used to obtain different sequences for each coordinate:

$$c_1 = \sqrt{2}, c_2 = \sqrt{3}, c_3 = \sqrt{5}, c_4 = \sqrt{7}, c_5 = \sqrt{11}, \ldots$$

Quasi-Random Sequences: Halton Sequence

- single-dimensional version, called van der Corput sequences, generates sequences where the unit interval is divided into powers of base b. For example, $b = 2$

  $$X = \left\{ \frac{1}{2}, \frac{1}{4}, \frac{3}{4}, \frac{1}{8}, \frac{5}{8}, \frac{3}{8}, \frac{7}{8}, \frac{1}{16}, \ldots \right\}$$

  whereas $b = 5$

  $$X = \left\{ \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, \frac{1}{25}, \frac{6}{25}, \frac{11}{25}, \ldots \right\}$$

- Multi-dimensional space-filling sequences use one van der Corput sequence for each dimension, each with its own base b. The bases, however, must be **coprime** in order to be uncorrelated.

- Two integers are **coprime** if the only positive integer that divides them both is 1, eg, 8 and 9.

- Correlation can be avoided by the leaped Halton method, which takes every pth point, where p is a prime different from all coordinate bases.

Quasi-Random Sequences: Sobol Sequence

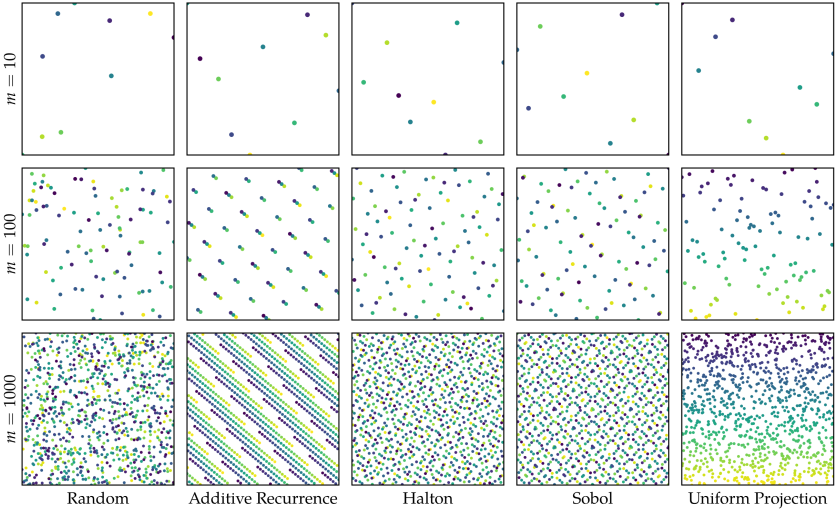
- Recursive XOR operation with carefully chosen numbers.
- XOR ($\oplus$) returns true if and only if both inputs are different
- For n-dimensional hypercube $I^n = [0,1]^n$, the ith point of the sequence $\boldsymbol{x}_i$ for dimension j is calculated as:

  $$x_{i,j} = x_{i-1,j} \oplus v_{k,j}$$

  $v_{k,j}$ is the jth dimension of the kth direction number.
- **direction numbers** $v_{k,j} = (0.v_{k,j,1}v_{k,j,2}\ldots)_2$ where $v_{k,j,m}$ denotes the mth digit after the binary point.
- Tables of direction numbers with different properties have been proposed.
- Initialization: unit initialisation: ℓth left most bit set to one $v_{k,j,\ell} = 1$ for all k and j and all others to be zero

Quasi-Random Sequences

space-filling sampling plans in two dimensions. Samples are colored according to the order in which they are sampled. The uniform projection plan was generated randomly and is not optimized.



Summary

- Sampling plans are used to cover search spaces with a limited number of points
- Full factorial sampling, which involves sampling at the vertices of a uniformly discretized grid, requires a number of points exponential in the number of dimensions
- Uniform projection plans, which project uniformly over each dimension, can be efficiently generated and can be optimized to be space-filling
- Greedy construction and the exchange local search algorithm can be used to find a subset of points that maximally fill a space
- Quasi-random sequences are deterministic procedures by which space-filling sampling plans can be generated

11. Machine Learning Applications

Introduction

Large-scale machine learning represents an important application area of optimization.

It is also a setting where traditional optimization techniques typically falter, due to the large number of parameters to be optimized, the large number of training examples, and the highly nonlinear functions often used.

Two case studies:

- Logistic regression or support vector machines
  convex optimization problems

- deep neural networks
  highly nonlinear and nonconvex problems

Text Classification via Convex Optimization

Task: determining whether a text document is one that discusses politics.

- set of examples $\{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_n, y_n)\}$, where for each $i \in \{1, \ldots, n\}$
  $\boldsymbol{x}_i$ represents the features of a text document (e.g., the words it includes)
  y_i is a label indicating whether the document belongs ($y_i = 1$) or not ($y_i = -1$) to a particular class.

- h prediction function

- performance measure: count how often the program prediction $h(\boldsymbol{x}_i)$ differs from the correct prediction y_i.

- minimize the so-called **empirical risk** of misclassification

$$R_n(h) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}[h(\boldsymbol{x}_i) \neq y_i], \qquad \text{where} \quad \mathbb{I}[A] = \begin{cases} 1 & \text{if } A \text{ is true,} \\ 0 & \text{otherwise} \end{cases}$$

Text Classification via Convex Optimization

Choosing between prediction functions belonging to a given class by comparing them using cross-validation procedures that involve splitting the examples into three disjoint subsets:

- a training set, optimizing the choice of a small subset of viable candidates h by minimizing R_n
- a validation set, generalized performance of each of these remaining candidates is then estimated using the validation set, the best performing of which is chosen as the selected function.
- a testing set, only used to estimate the generalized performance of this selected function

Formalization

- feature vector $\boldsymbol{x} \in \mathbb{R}^d$ whose components are associated with a prescribed set of vocabulary words (bag of words approach); $\|\boldsymbol{x}\| = 1$
- $h(\boldsymbol{x}; \boldsymbol{w}, \tau) = \boldsymbol{w}^T \boldsymbol{x} - \tau$, $\boldsymbol{w} \in \mathbb{R}^d$ and $\tau \in \mathbb{R}$ linear discriminator
- performance measure: count how many times $\mathrm{sign}(h(\boldsymbol{x}; \boldsymbol{w}, \tau))$ mispredicts. Discontinuous problem.
- alternatively, define a continuous **loss function** ℓ that measures a cost for predicting h when the true label is y;
  e.g., one may choose a **log-loss function** of the form $\ell(h, y) = \log(1 + \exp(-hy))$.
- minimize the empirical risk

$$\min_{(\boldsymbol{w},\tau)\in\mathbb{R}^d\times\mathbb{R}} \frac{1}{n}\sum_{i=1}^{n} \ell(h(\boldsymbol{x}_i; \boldsymbol{w}, \tau), y_i) + \frac{\lambda}{2}\|\boldsymbol{w}\|_2^2 \tag{3}$$

  solve for various λ and choose on the validation set

Deep Neural Networks

- Deep Neural Networks: represent hypotheses as computation graphs with tunable weights and compute the gradient of the loss function with respect to those weights in order to fit the training data.
- Forward accumulation: compute prediction
- Backward accumulation: compute gradient
- `https://playground.tensorflow.org/`

Perceptual Tasks via Deep Neural Networks

- Prediction function h whose value is computed by applying successive transformations to a given input vector $\boldsymbol{x}_i \in \mathbb{R}^{d_0}$.

- These transformations are made in layers. A canonical fully connected layer performs the computation

$$\boldsymbol{x}_i^{(j)} = s(W_j \boldsymbol{x}_i^{(j-1)} + b_j) \in \mathbb{R}^{d_j}$$

- where $\boldsymbol{x}_i^{(0)} = \boldsymbol{x}_i$, the matrix $W_j \in \mathbb{R}^{d_j \times d_{j-1}}$ and vector $b_j \in \mathbb{R}^{d_j}$ contain the jth layer parameters, and s is a component-wise nonlinear activation function

- $s(x) = 1/(1 + \exp(-x))$ and the hinge function $s(x) = \max(0, x)$ (often called a rectified linear unit (ReLU) in this context)

- $\boldsymbol{x}_i^{(J)}$ leads to the prediction function value $h(\boldsymbol{x}_i; \boldsymbol{w})$, $\boldsymbol{w} = \{(W_1, b_1), \ldots, (W_J, b_J)\}$.

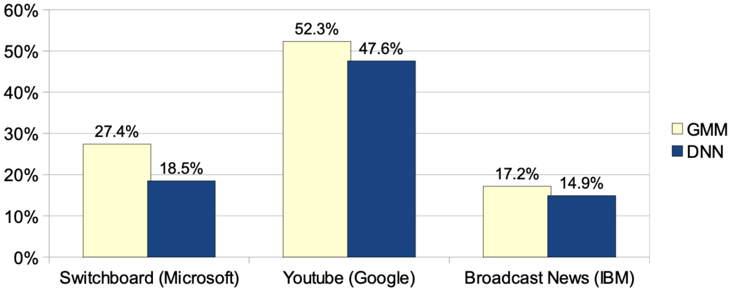
- leads to a highliy non-linear and non-convex problem of the form:

$$\min_{\boldsymbol{w} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^{n} \ell(h(\boldsymbol{x}_i; \boldsymbol{w}), y_i)$$

- The gradient with respect to $\boldsymbol{w}$ is made of simple expressions that can be computed by passing information back through the network from the output units.
- the gradient computations for any feedforward computation graph have the same structure as the underlying computation graph.
- gradients can be computed by the chain rule and the algorithmic method of automatic differentiation
- back-propagation in deep learning is simply an application of **reverse mode differentiation**, which applies the chain rule “from the outside in”

Speech recognition

- A contemporary fully connected neural network for speech recognition typically has five to seven layers. This amounts to tens of millions of parameters to be optimized,
- the training may require up to thousands of hours of speech data (representing hundreds of millions of training examples) and weeks of computation on a supercomputer
- Accuracy of 2010-2012 models (GMM: Gaussian Mixture Models)



Convolutional neural networks

Convolutional neural networks (CNNs) have proved to be very effective for computer vision and signal processing tasks

ImageNet Large Scale Visual Recognition Competition (ILSVRC) with five convolutional layers and three fully connected layers

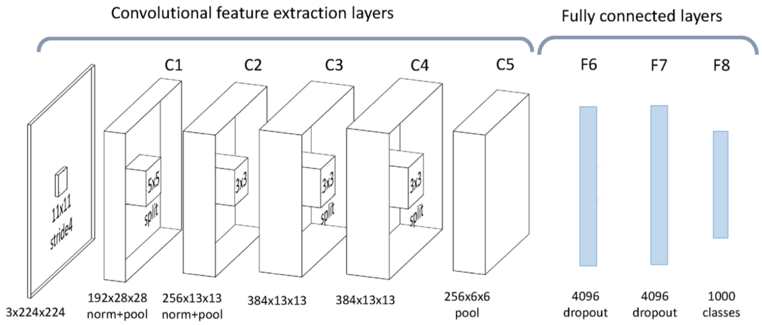


Image Recognition

- input $\boldsymbol{x}_i^{(j-1)}$ is intepreted as a multichannel image of 224×224 pixels.
- convolutional layers, wherein the parameter matrix W_j is a circulant matrix
- product $W_j \boldsymbol{x}_i^{(j-1)}$ computes the convolution of the image by a trainable filter
- activation functions are piecewise linear functions and can perform more complex operations that may be interpreted as image rectification, contrast normalization, or subsampling.
- output scores represent the odds that the image belongs to each of 1,000 categories.
- 60 million parameters
- training on a few million labelled images takes a few days on a dual GPU workstation.

Fundamentals

- Joint probability distribution function $P(x, y)$ that simultaneously represents the distribution $P(x)$ of inputs as well as the conditional probability $P(y \mid x)$ of the label y being appropriate for an input x.

- One should seek to find h that yields a small expected risk of misclassification over all possible inputs, i.e., an h that minimizes

  $$R(h) = P[h(\mathbf{x}) \neq y] = E[\mathbb{I}[h(\mathbf{x}) \neq y]],$$

  which is **variational** since we are optimizing over a set of functions (the h), and is **stochastic** since the objective function involves an expectation.

- without explicit knowledge of P the only tractable option is to construct a **surrogate** problem that relies solely on the examples $(\mathbf{x}_i, y_i)_{i=1}^n$: minimize the empirical risk

Tasks:

- how to choose the parameterized family of prediction functions $\mathcal{H}$ and

- how to determine (and find) the particular prediction function $h \in \mathcal{H}$ that is optimal.

Choice of Prediction Function

1. $\mathcal{H}$ should contain prediction functions that are able to achieve a low empirical risk over the training set, so as to avoid bias or underfitting the data. (rich family of functions or by using a priori knowledge to select a well-targeted family)

2. the gap between expected risk and empirical risk, namely, $R(h) - R_n(h)$, should be small over all $h \in \mathcal{H}$. (increases with rich family of functions)

3. $\mathcal{H}$ should be effciently solvable in the corresponding optimization problem (the richer the family of functions and/or the larger training set the more complex the problem becomes)

Choice of Prediction Function

Uniform laws of large numbers and the Hoeffding inequality gurantee that with probability at least $1 - \eta$

$$\sup_{h \in \mathcal{H}} |R(h) - R_n(h)| \leq \mathcal{O}\left(\sqrt{\frac{1}{2n}\log\left(\frac{2}{\eta}\right) + \frac{d_{\mathcal{H}}}{n}\log\left(\frac{n}{d_{\mathcal{H}}}\right)}\right)$$

- $d_{\mathcal{H}}$ Vapnik-Chervonenkis (VC) dimension (measure of capacity of separating points)
- not the same as numer of parameters

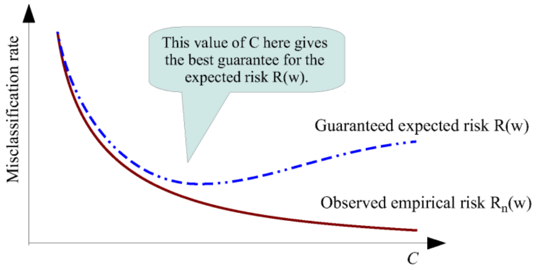
- for a fixed $d_{\mathcal{H}}$, uniform convergence is obtained by increasing the number of training points n.
- for a fixed n, the gap can widen for larger $d_{\mathcal{H}}$.

In practice it is typically easier to estimate with cross-validation experiments.

Structural Risk Minimization

- Rather than choosing a generic family of prediction functions one chooses a structure, i.e., a collection of nested function families.

- structure can be formed as a collection of subsets of a given family $\mathcal{H}$: given a preference function Ω, choose various values of a hyperparameter C, according to each of which one obtains the subset $\mathcal{H}_C \stackrel{def}{=} \{h \in \mathcal{H} : \omega(h) \leq C\}$. ($C$ is, eg, degree of a polynomial model function, dimension of an inner layer of a DNN)



$$\begin{aligned} & \min R_n(h) \\ \text{subject to} \quad & \Omega(h) \leq C \end{aligned}$$

Regularized empirical risk

$$R_n(h) + \lambda\Omega(h)$$

Structural Risk Minimization

Another approach:

- employ an algorithm for minimizing R_n, but terminate the algorithm early, i.e., before an actual minimizer of R_n is found.
  The hyperparameter is played by the training time allowed

- often essential due to computational budget limitations.

Formal Optimization Problem Statements

- We do not consider a variational optimization problem over $\mathcal{H}$,
- instead we assume that the prediction function h has a fixed form and is parameterized by a real vector $\boldsymbol{w} \in \mathbb{R}^d$
- for some given $h(\cdot;\cdot) : \mathbb{R}^{d_x} \times \mathbb{R}^d \to \mathbb{R}^{d_y}$, we consider the family of prediction functions $\mathcal{H} \stackrel{def}{=} \{h(\cdot; \boldsymbol{w}) : \boldsymbol{w} \in \mathbb{R}^d\}$
- aim to find $h \in \mathcal{H}$ that minimizes a given **loss function** $\ell : \mathbb{R}^{d_x} \times \mathbb{R}^d \to \mathbb{R}^{d_y}$, $\ell(h(x; \boldsymbol{w}), y)$
- Ideally, the expected loss is defined over **any** input-output pair. Assuming probability distribution $P(\boldsymbol{x}, y)$ represents the true input-output relationship:

$$R(w) = \int_{\mathbb{R}^{d_x} \times \mathbb{R}^{d_y}} \ell(h(\boldsymbol{x}; \boldsymbol{w}), y) dP(\boldsymbol{x}, y) = E[\ell(h(\boldsymbol{x}; \boldsymbol{w}), y)] \qquad \text{Expected Risk}$$

Formal Optimization Problem Statements

- In practice, one seeks the solution of a problem that involves an estimate of the expected risk R.

- In supervised learning, we have access (either all-at-once or incrementally) to a set of $n \in \mathbb{N}$ independently drawn input-output samples $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n \subseteq \mathbb{R}^{d_x} \times \mathbb{R}^{d_y}$, with which we define the **empirical risk function** $R_n : \mathbb{R}^d \to \mathbb{R}$ by

$$R_n(\boldsymbol{w}) \stackrel{def}{=} \frac{1}{n} \sum_{i=1}^{n} \ell(h(\boldsymbol{x}_i; \boldsymbol{w}), y_i) \qquad \text{Empirical Risk}$$

(note that before we used misclassification error while now ℓ.)

Simplified Notation

Let ξ be a random seed or the realization of a single (or a set of) sample $(\boldsymbol{x}, y)$.
For a given $(\boldsymbol{w}, \xi)$ let $f(\boldsymbol{w}; \xi)$ be the composition of the loss function ℓ and the prediction function h
Then:

$$R(\boldsymbol{w}) = \mathbb{E}_{\xi}[f(\boldsymbol{w}; \xi)] \qquad \text{Expected Risk}$$

Let $\{\xi_{[i]}\}_{i=1}^{n}$ be realizations of ξ corresponding to $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$ and $f_i(\boldsymbol{w}) \stackrel{def}{=} f(\boldsymbol{w}; \xi_{[i]})$
Then:

$$R_n(\boldsymbol{w}) \stackrel{def}{=} \frac{1}{n} \sum_{i=1}^{n} f_i(\boldsymbol{w}) \qquad \text{Empirical Risk}$$

Convex Functions

The class of convex functions satisfies the following stability properties:

Proposition

If $\{f_j\}_{j\in\{1,\dots,m\}}$ are convex and $\{\alpha_j\}_{j\in\{1,\dots,m\}}$ are nonnegative, then $\sum_{j=1}^{m} \alpha_j f_j$ and $\max_{j\in\{1,\dots,m\}} f_j$ are convex.

Proposition

If $f : \mathbb{R}^d \to \mathbb{R}$ is convex and $A : \mathbb{R}^d \to \mathbb{R}^d$ is affine then $f \circ A : \mathbb{R}^d \to \mathbb{R}$ is convex.

Proposition

If $f : \mathbb{R}^{d_1+d_2} \to \mathbb{R}$ is convex, so is the function $x_1 \mapsto \inf_{x_2\in\mathbb{R}^{d_2}} f(x_1, x_2)$ on $\mathbb{R}^{d_1}$.

- Functions of the form of $R_n(\boldsymbol{w})$ in the previous slide and equation (3) are convex if the loss ℓ is convex in the first variable, $h(\boldsymbol{x}; \theta, \tau)$ is linear in $\boldsymbol{w}, \tau$, and their domain is convex. Eg, Linear models (in their parameters).
- Neural networks are nonlinear models (in their parameters) and thus lead to non-convex problems.

Stochastic vs Batch Optimization Methods

Reduction to minimizing R_n, with $\boldsymbol{w}_0 \in \mathbb{R}^d$ given. Deterministic problem.
Stochastic Approach: Stochastic Gradient (Robbins and Monro, 1951)

$$\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha_k \nabla f_{i_k}(\boldsymbol{w}_k)$$

i_k is chosen randomly from $\{1, \ldots, n\}$, $\alpha_k > 0$.

- very cheap iteration only on one sample.
- $\{\boldsymbol{w}_k\}$ is a stochastic process determined by the random sequence $\{i_k\}$.
- the direction might not always be a descent but if it is a descent direction in **expectation**, then the sequence $\{\boldsymbol{w}_k\}$ can be guided toward a minimizer of R_n.

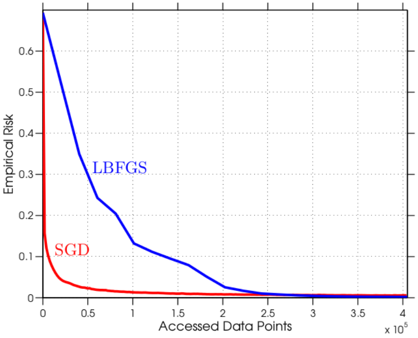
Batch Approach: batch gradient, steepest descent, full gradient method:

$$\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha_k \nabla R_n(\boldsymbol{w}_k) = \boldsymbol{w}_k - \frac{\alpha_k}{n} \sum_{i=1}^{n} \nabla f_i(\boldsymbol{w}_k)$$

- more expensive
- can use all deterministic gradient-based optimization methods
- the sum structure opens up to parallelization

Stochastic Gradient

- In case of redundancy using all the sample data in every iteration is inefficient
- Comparison of the performance of a batch L-BFGS method on number of evaluations of a sample gradient $\nabla f_{i_k}(\boldsymbol{w}_k)$.
- Each set of n consecutive accesses is called an **epoch**.
- The batch method performs only one step per epoch while SG performs n steps per epoch.



the fast initial improvement achieved by SG, followed by a drastic slowdown after 1 or 2 epochs, is common in practice

SG more sensitive to α_k and starting point

if more epochs, batch may become better

Theoretical Motivations

- A batch approach can minimize R_n at a fast rate;
  if R_n is strongly convex and we apply a batch gradient method, then there exists a constant $\rho \in (0, 1)$ such that, for all $k \in \mathbb{N}$, the training error follows **(R-)linear convergence** ($\equiv$ geometric convergence in ML)

$$R_n(\boldsymbol{w}_k) - R_n^* \leq \mathcal{O}(\rho^k),$$

- in the worst case, the total number of iterations in which the training error can be above a given $\epsilon > 0$ is proportional to $\log(1/\epsilon)$.

- with a per-iteration cost proportional to n (due to the need to compute $\nabla R_n(\boldsymbol{w}_k)$ for all $k \in \mathbb{N}$), the total work required to obtain ϵ-optimality for a batch gradient method is proportional to $n \log(1/\epsilon)$.

Theoretical Motivations

- The rate of convergence of a basic stochastic method is slower than for a batch gradient; if R_n is strictly convex and each i_k is drawn uniformly from $\{1, \ldots, n\}$, then for all $k \in \mathbb{N}$, SG satisfies the **sublinear convergence property**

$$\mathbb{E}[R_n(\boldsymbol{w}_k) - R_n^*] = \mathcal{O}(1/k).$$

- However, neither the per-iteration cost nor the right-hand side $\mathcal{O}(1/k)$ depends on the sample set size n

- the total work required to obtain ϵ-optimality for SG is proportional to $1/\epsilon$, which is larger than $n \log(1/\epsilon)$ for moderate values of n and ϵ, but SG can be more efficient than batch gradient for large n.

Theoretical Motivations

- In a **stochastic optimization setting**, in the analysis of SG we substitute $\nabla f_{i_k}(\boldsymbol{w}_k)$ by $\nabla f(\boldsymbol{w}_k; \xi_k)$ with each ξ_k drawn independently according to the distribution P
- then SG yields for the **expected risk** the same sublinear convergence rate but on the expected risk:

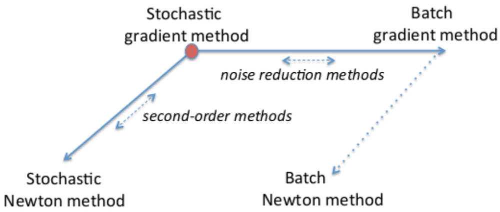
  $$\mathbb{E}[R(\boldsymbol{w}_k) - R^*] = \mathcal{O}(1/k).$$

  (a batch approach is not even viable without the ability to compute ∇R.)
- Interpretation: If $n \gg k$ up to iteration k minimizing R_n same as minimizing R

Theoretical Motivations: Summary

- Theoretical arguments in favor of stochastic over batch approaches in optimization methods for large-scale ML.
- However, if a larger number of epochs is considered, then one would see the batch approach eventually overtake the stochastic method and yield a lower training error.
- Moreover, the SG iteration is difficult to parallelize and requires excessive communication between nodes in a distributed computing setting
- Research on combining the best properties of batch and stochastic algorithms.

Beyond SG: Noise Reduction and Second-Order Methods



- on horizontal axis methods that try to improve rate of convergence
- on vertical axis, methods that try to overcome non-linearity and ill-conditioning

Mini-batch Approach small subset of samples, call it $\mathcal{S}_k \subseteq \{1, \dots, n\}$, chosen randomly in each iteration:

$$\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \frac{\alpha_k}{|\mathcal{S}_k|} \sum_{i \in \mathcal{S}} \nabla f_i(\boldsymbol{w}_k)$$

- due to the reduced variance of the stochastic gradient estimates, the method is easier to tune in terms of choosing the stepsizes $\{\alpha_k\}$.
- dynamic sample size and gradient aggregation methods, both of which aim to improve the rate of convergence from sublinear to linear

Formalization

$$F(\boldsymbol{w}) = \begin{cases} R_n(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{w}) & \text{Empirical Risk} \\ R(\boldsymbol{w}) = \mathbb{E}_\xi[f(\boldsymbol{w};\xi)] & \text{Expected Risk} \end{cases}$$

sampling uniformly with replacement from training set $\rightsquigarrow R_n$
sampling with $P(\xi)$ with replacement from training set $\rightsquigarrow R$.

Procedure SG(...);
Choose an initial iterate $\boldsymbol{w}_0$;
for $k = 0, 1, \ldots$ **do**
  Generate a realization of the random variable ξ_k;
  Compute a stochastic vector $g(\boldsymbol{w}_k, \xi_k)$;
  Choose a stepsize $\alpha_k > 0$;
  Set the new iterate as $\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha_k g(\boldsymbol{w}_k, \xi_k)$;

Formalization

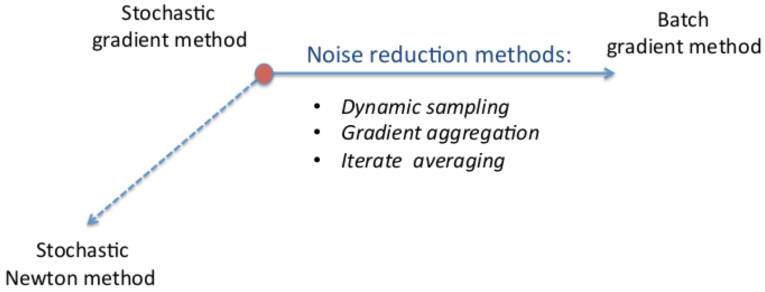
ξ_k may represent a single sample or a mini-batch
g may represent a stochastic gradient (biased estimator of $\nabla F(\boldsymbol{w}_k)$ or a stochastic Netwon or quasi-Newton direction).

$$g(\boldsymbol{w}_k, \xi_k) = \begin{cases} \nabla f(\boldsymbol{w}_k; \xi_k) \\ \frac{1}{n_k} \sum_{i=1}^{n_k} \nabla f(\boldsymbol{w}_k; \xi_{k,i}) \\ H_k \frac{1}{n_k} \sum_{i=1}^{n_k} \nabla f(\boldsymbol{w}_k; \xi_{k,i}) \end{cases}$$

H_k a symmetric positive definite scaling matrix (we called it B_k^{-1} earlier in this course)
α_k fixed stepsize or diminishing stepsizes
$\boldsymbol{w}_k$ can have influence on the sample selection (active learning)

Noise Reduction Methods

- SG as the ideal optimization approach for large-scale applications.
- SG suffers from the adverse effect of **noisy gradient estimates**.
  - when fixed stepsizes are used it prevents SG from converging to the solution
  - when a diminishing stepsize sequence $\{\alpha_k\}$ is employed it leads to a slow, sublinear rate of convergence.
- Remedies:



Overview

Achieve linear rate of convergence to the optimal value using a fixed stepsize by constructing a more reliable step with smaller variance than an SG step:

- **Dynamic sampling methods** achieve noise reduction by gradually increasing the mini-batch size used in the gradient computation, thus employing increasingly more accurate gradient estimates as the optimization process proceeds.

- **Gradient aggregation methods** improve the quality of the search directions by storing gradient estimates corresponding to samples employed in previous iterations, updating one (or some) of these estimates in each iteration, and defining the search direction as a weighted average of these estimates.

Rate of convergence remains sublinear but reduces variance of iterates wrt SG:

- **iterate averaging methods** maintain an average of iterates computed during the optimization process and employes a more aggressive stepsize sequence–of order $O(1/\sqrt{k})$ rather than $O(1/k)$.

Reducing Noise at a Geometric (Linear) Rate

rate of decrease in noise that allows a stochastic-gradient-type method to converge at a linear rate.

Lemma (4.2)

Under the assumption of Lipschitz-continuous objective gradients ∇F with Lipschitz constant $L > 0$, ie, $\|\nabla F(\boldsymbol{w}) - \nabla F(\bar{\boldsymbol{w}})\|_2 \leq L\|\boldsymbol{w} - \bar{\boldsymbol{w}}\|_2$ for all $\{\boldsymbol{w}, \bar{\boldsymbol{w}}\} \subseteq \mathbb{R}^d$, the iterates of SG satisfy the following inequality for all $k \in \mathbb{N}$:

$$\mathbb{E}_{\xi_k}[F(\boldsymbol{w}_{k+1})] - F(\boldsymbol{w}_k) \leq -\alpha_k \nabla F(\boldsymbol{w}_k)^T \mathbb{E}_{\xi_k}[g(\boldsymbol{w}_k, \xi_k)] + \frac{1}{2}\alpha_k^2 L\, \mathbb{E}_{\xi_k}[\|g(\boldsymbol{w}_k, \xi_k)\|_2^2]$$

We want to make the left hand side small (sequence of expected optimality gaps).

Reducing Noise at a Geometric (Linear) Rate

Theorem (5.1, Strongly Convex Objective, Noise Reduction)

Under assumptions of F strongly convex with Lipschitz-continuous gradients the SG method with an appropriately chosen fixed step size $\alpha_k = \bar{\alpha}$ and a variance of the stochastic vectors that decreases geometrically

$$Var_{\xi_k}[g(\boldsymbol{w}_k, \xi_k)] \leq M\zeta^{k-1}$$

has a sequence of expected optimality gaps that vanishes at a linear rate:

$$\mathbb{E}[F(\boldsymbol{w}_k) - F^*] \leq \omega\rho^{k-1}$$

Dynamic Sample Size Methods

Can we design efficient optimization methods attaining the critical bound on the variance?

Mini-batch stochastic gradient:

$$\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \bar{\alpha} g(\boldsymbol{w}_k, \xi_k)$$

where the stochastic directions are computed for some $\tau > 1$ as

$$g(\boldsymbol{w}_k, \xi_k) \stackrel{def}{=} \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla f(\boldsymbol{w}_k; \xi_{k,i}) \qquad \text{with } n_k \stackrel{def}{=} |\mathcal{S}_k| = \lceil \tau^{k-1} \rceil.$$

the mini-batch size increases geometrically as a function of the iteration counter k

Corollary 5.2. Let $\{\boldsymbol{w}_k\}$ be the iterates generated with unbiased gradient estimates, i.e., $\mathbb{E}_{\xi_{k,i}}[\nabla f(\boldsymbol{w}_k; \xi_{k,i})] = \nabla F(\boldsymbol{w}_k)$ for all $k \in \mathbb{N}$ and $i \in \mathcal{S}_k$ (expectation of each stochastic gradient is equal to the true gradient $\nabla F(\boldsymbol{w}_k)$). Then, the variance condition is satisfied, and if all other assumptions of Theorem 5.1 hold, then the expected optimality gap vanishes linearly.

Dynamic Sample Size Methods

Note: we described a method as linearly convergent but the per-iteration cost increases without bound.

Theorem 5.3 Suppose that the dynamic sampling SG method is run with a stepsize $\bar{\alpha}$ satisfying "some" bounds and some τ. In addition, suppose that other Assumptions hold. Then, the total number of evaluations of a stochastic gradient of the form $\nabla f(\boldsymbol{w}_k; \xi_{k,i})$ required to obtain $\mathbb{E}[F(\boldsymbol{w}_k) - F^*] \leq \epsilon$ is $O(\epsilon^{-1})$.

Dynamic Sample Size Guidelines

Given the rate of convergence of a batch optimization algorithm on strongly convex functions (i.e., linear, superlinear, etc.), what should be the sampling rate so that the overall algorithm is **efficient** in the sense that it results in the lowest computational complexity?

- if the optimization method has a sublinear rate of convergence, then there is no sampling rate that makes the algorithm "efficient";
- if the optimization algorithm is linearly convergent, then the sampling rate must be geometric (with restrictions on the constant in the rate) for the algorithm to be "efficient";
- for superlinearly convergent methods, increasing the sample size at a rate that is slightly faster than geometric will yield an "efficient" method.

Design in Practice

- presetting the sampling rate, ie, $\tau > 1$ before running the optimization algorithm, requires some experimentation. Care must be put in preventing the full sample set from being employed too soon

- adaptive mechanisms to produce descent directions sufficiently often
  - any direction $g(\boldsymbol{w}_k, \xi_k)$ is a descent direction for F at $\boldsymbol{w}_k$ if, for some $\chi \in [0, 1)$, one has

    $$\delta(\boldsymbol{w}_k, \xi_k) \stackrel{def}{=} \|g(\boldsymbol{w}_k, \xi_k) - \nabla F(\boldsymbol{w}_k)\|_2 \leq \chi \|g(\boldsymbol{w}_k, \xi_k)\|_2$$

    verifying the inequality may be costly because involves the evaluation of $\nabla F(\boldsymbol{w}_k)$, one can estimate the left-hand side $\delta(\boldsymbol{w}_k, \xi_k)$, and then choose n_k so it holds sufficiently often.
  - The sample variance obtained by sampling without replacements is bounded above by $\chi^2 \|g(\boldsymbol{w}_k, \xi_\epsilon)\|_2^2$
  - If this condition is not satisfied, then increase the sample size to a size that one might predict would satisfy such a condition.
  - no guarantee that the size n_k increases at a geometric rate. Remedy: if the adaptive increases the sampling rate more slowly than a preset geometric sequence, then a growth in the sample size is imposed.

Gradient Aggregation

- Rather than compute increasingly more **new** stochastic gradient information in each iteration, achieve a lower variance by **reusing and/or revising** previously computed information
- achieve a linear rate of convergence on strongly convex problems.
- improved rate is achieved primarily by either an increase in computation or an increase in storage.
- works on finite sums like R_n

stochastic variance reduced gradient (SVRG)

Procedure SVRG ; # Methods for Minimizing an Empirical Risk R_n

Choose an initial iterate $\boldsymbol{w}_1 \in \mathbb{R}^d$, stepsize $\alpha > 0$ and a positive integer m;

for $k = 1, 2, \ldots$ **do**

  Compute the batch gradient $\nabla R_n(\boldsymbol{w}_k)$;

  Initialize $\tilde{\boldsymbol{w}}_1 \leftarrow \boldsymbol{w}_k$;

  for $j = 1, \ldots, m$ **do**

    $\tilde{\boldsymbol{g}}_j \leftarrow \nabla f_{i_j}(\tilde{\boldsymbol{w}}_j) - (\nabla f_{i_j}(\boldsymbol{w}_k) - \nabla R_n(\boldsymbol{w}_k))$; # $\nabla R_n(\boldsymbol{w}_k)$ from batch gradient

    $\tilde{\boldsymbol{w}}_{j+1} \leftarrow \tilde{\boldsymbol{w}}_j - \alpha \tilde{\boldsymbol{g}}_j$;

  Option (a): Set $\boldsymbol{w}_{k+1} = \tilde{\boldsymbol{w}}_{m+1}$;

  Option (b): Set $\boldsymbol{w}_{k+1} = \frac{1}{m}\sum_{j=1}^{m} \tilde{w}_{j+1}$;

  Option (c): Choose j uniformly from $\{1, \ldots, m\}$ and set $\boldsymbol{w}_{k+1} = \tilde{\boldsymbol{w}}_{j+1}$;

- since $E_{i_j}[\nabla f_{i_j}(\boldsymbol{w}_k)] = \nabla R_n(\boldsymbol{w}_k)$, one can view $\nabla f_{i_j}(\boldsymbol{w}_k) - \nabla R_n(\boldsymbol{w}_k)$ as the bias in the gradient estimate $\nabla f_{i_j}(\boldsymbol{w}_k)$.
- sampled gradient $\nabla f_{i_j}(\tilde{\boldsymbol{w}}_j)$ is corrected based on a perceived bias. Overall, $\tilde{\boldsymbol{g}}_j$ represents an unbiased estimator of $\nabla R_n(\tilde{\boldsymbol{w}}_j)$, but with a variance that one can expect to be smaller than as in simple SG
- Without explicit knowledge of L and c, the length of the inner cycle m and the stepsize α are typically both chosen by experimentation.

Stochastic average gradient algorithm (SAGA)

in each iteration, it computes a stochastic vector $\boldsymbol{g}_k$ as the average of stochastic gradients evaluated at previous iterates.

Procedure SAGA ; # Method for Minimizing an Empirical Risk R_n

Choose an initial iterate $\boldsymbol{w}_1 \in \mathbb{R}^d$ and stepsize $\alpha > 0$;

for $i = 1, \ldots, n$ **do**

  Compute $\nabla f_i(\boldsymbol{w}_1)$;

  Store $\nabla f_i(\boldsymbol{w}_{[i]}) \leftarrow \nabla f_i(\boldsymbol{w}_1)$; # $\boldsymbol{w}_{[i]}$ represents the latest iterate when ∇f_i was evaluated

for $k = 1, 2, \ldots$ **do**

  Choose j uniformly in $\{1, \ldots, n\}$;

  Compute $\nabla f_j(\boldsymbol{w}_k)$;

  Set $\boldsymbol{g}_k \leftarrow \nabla f_j(\boldsymbol{w}_k) - \nabla f_j(\boldsymbol{w}_{[j]}) + \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\boldsymbol{w}_{[i]})$; # $\mathbb{E}[g_k] = \nabla R_n(\boldsymbol{w}_k)$

  Store $\nabla f_j(\boldsymbol{w}_{[j]}) \leftarrow \nabla f_j(\boldsymbol{w}_k)$;

  Set $\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha \boldsymbol{g}_k$;

As in SVRG, the method employs unbiased gradient estimates, with variances that are expected to be less than the stochastic gradients that would be employed in a basic SG routine

SAGA

- Same per-iteration costs as basic SG
- on strongly convex R_n can achieve a linear rate of convergence but needs knowledge of at least L.
- More effective initialization instead of evaluating all the gradients $\{\nabla f_i\}_{i=1}^n$ at the initial point. For example, one could perform one epoch of simple SG, or one can assimilate iterates one-by-one and compute $\boldsymbol{g}_k$ only using the gradients available up to that point.
- SAGA needs to store n stochastic gradient vectors
- for very large n, gradient aggregation methods are comparable to batch algorithms and therefore cannot beat SG in this regime

Iterated Averaging Methods

- for minimizing a continuously differentiable F with unbiased gradient estimates, the idea is to employ the iteration:

$$\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha_k g(\boldsymbol{w}_k, \xi_k)$$
$$\tilde{\boldsymbol{w}}_{k+1} \leftarrow \frac{1}{k+1} \sum_{j=1}^{k+1} \boldsymbol{w}_j$$

  where $\{\tilde{\boldsymbol{w}}_k\}$ has no effect on the computation of the SG iterate sequence $\{\boldsymbol{w}_k\}$

- with stepsizes diminishing at a slow rate of $\mathcal{O}(1/(k^a))$ for some $a \in (\frac{1}{2}, 1)$ on strongly convex objectives, yields that $\mathbb{E}[\|\boldsymbol{w}_k - \boldsymbol{w}^*\|_2^2] = \mathcal{O}(1/(k^a))$ while $\mathbb{E}[\|\tilde{\boldsymbol{w}}_k - \boldsymbol{w}^*\|_2^2] = \mathcal{O}(1/k)$.

- in certain cases this combination of long steps and averaging prevent the adverse effects caused by ill-conditioning.

Second Order Methods

- Address the adverse effects of high nonlinearity and ill-conditioning of the objective function through the use of second-order information.

- improve convergence rates of batch methods or the constants involved in the sublinear convergence rate of stochastic methods

- First-order methods are **not scale invariant**. Consider:
  F continuously differentiable function $F : \mathbb{R}^d \to \mathbb{R}$

  $$\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha_k \nabla F(\boldsymbol{w}_k)$$

  linear transformation of the variables $\{\boldsymbol{w}_k\} = \{B\tilde{\boldsymbol{w}}_k\}$. $\min_{\tilde{w}} F(B\tilde{\boldsymbol{w}}_k)$. B SPD matrix.

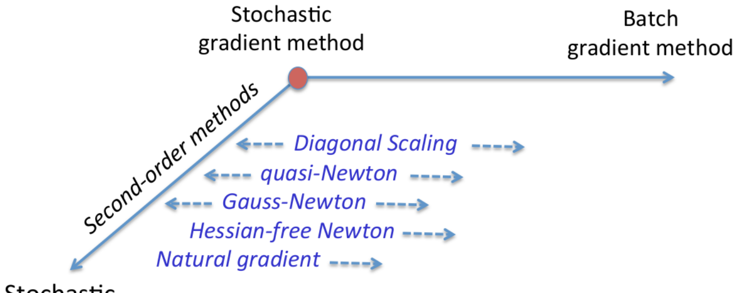
  $$\tilde{\boldsymbol{w}}_{k+1} \leftarrow \tilde{\boldsymbol{w}}_k - \alpha_k B \nabla F(B\tilde{\boldsymbol{w}}_k) \qquad \Longrightarrow \qquad \boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha_k B^2 \nabla F(\boldsymbol{w}_k)$$

  They will perform differently. With $\alpha = 1$ and $B = (\nabla^2 F(\boldsymbol{w}_1))^{-1/2}$ we get Newton's method. If F is strongly convex quadratic it gets to w^* in one step.

- Deterministic (batch) methods like Newton's method can achieve a quadratic rate of convergence if $\boldsymbol{w}_1$ is sufficiently close to a strong minimizer.

- Stochastic methods like the SG method cannot achieve a convergence rate that is faster than sublinear, regardless of the choice of B.

- careful use of successive re-scalings based on (approximate) second-order derivatives can be beneficial between the stochastic and batch regimes.



Hessian-Free Inexact Newton Methods

- Newton's method:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + \alpha_k \mathbf{s}_k$$
$$\text{where } \mathbf{s}_k \text{ satisfies } \nabla^2 F(\mathbf{w}_k)\mathbf{s}_k = -\nabla F(\mathbf{w}_k).$$

- one can solve the linear system inexactly through an iterative approach such as the conjugate gradient (CG) method.
- By ensuring that the linear solves are accurate enough, such an inexact Newton-CG method can enjoy a superlinear rate of convergence
- For a smooth objective function F, one can compute $\nabla^2 F(\mathbf{w})\mathbf{d}$ at a cost that is a small multiple of the cost of evaluating ∇F, and without forming the Hessian, which would require $\mathcal{O}(d^2)$ storage
- exploit structure of risk measures

- iterations are more tolerant to noise in the hessian estimate than it is to noise in the gradient estimate

- employs a smaller, conditionally (given w_k) uncorrelated, sample for defining the Hessian than for the stochastic gradient estimate

- can be combined with a backtracking (Armijo) line search or trust region

- (subsampled) Hessian-vector products can be computed efficiently in ML tasks

Example 6.2. *Consider a binary classification problem where the training function is given by the logistic loss with an ℓ_2-norm regularization parameterized by $\lambda > 0$:*

$$R_n(w) = \frac{1}{n}\sum_{i=1}^{n}\log(1+\exp(-y_i w^T x_i)) + \frac{\lambda}{2}\|w\|^2. \tag{6.8}$$

A (subsampled) Hessian-vector product can be computed efficiently by observing that

$$\nabla^2 f_{\mathcal{S}_k^H}(w_k;\xi_k^H)d = \frac{1}{|\mathcal{S}_k^H|}\sum_{i\in\mathcal{S}_k^H}\frac{\exp(-y_i w^T x_i)}{(1+\exp(-y_i w^T x_i))^2}(x_i^T d)x_i + \lambda d.$$

- Stochastic Quasi-Newton Methods:
  construct approximations to the Hessian using only gradient information. Eg, L-BFGS (limited memory BFGS)

- Gauss-Newton Methods
  works for problems in which the objective function is a sum of squares.
  constructs an approximation to the Hessian using only first-order information, and this approximation is guaranteed to be positive semidefinite, even when the full Hessian itself may be indefinite.
  The price to pay for this convenient representation is that it ignores second-order interactions between elements of the parameter vector $\boldsymbol{w}$, which might mean a loss of curvature information that could be useful for the optimization process.

- Natural Gradient Methods
  aim to be invariant with respect to all differentiable and invertible transformations
  formulate the gradient descent algorithm in the space of prediction functions rather than specific parameters.

Summary

- Ways to cope with the problems in machine learning
- SG might not be the best choice for parallelization
- How about other methods like CMA-ES? Why not used?

Other Methods

- gradient methods with momentum,
- accelerated gradient methods, and
- coordinate descent methods

12. Convergence Analysis of Stochastic Gradient

Outline

Theoretical Analysis — Preliminaries

convergence properties and worst-case iteration complexity bounds.

$$F(\boldsymbol{w}) = \begin{cases} R_n(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\boldsymbol{w}) & \text{Empirical Risk} \\ R(\boldsymbol{w}) = \mathbb{E}_{\xi}[f(\boldsymbol{w};\xi)] & \text{Expected Risk} \end{cases}$$

sampling uniformly with replacement from training set $\rightsquigarrow R_n$
sampling with $P(\xi)$ with replacement from training set $\rightsquigarrow R$.

Procedure SG(...);
Choose an initial iterate $\boldsymbol{w}_0$;
for $k = 0, 1, \ldots$ **do**
- Generate a realization of the random variable ξ_k;
- Compute a stochastic vector $g(\boldsymbol{w}_k, \xi_k)$;
- Choose a stepsize $\alpha_k > 0$;
- Set the new iterate as $\boldsymbol{w}_{k+1} \leftarrow \boldsymbol{w}_k - \alpha_k g(\boldsymbol{w}_k, \xi_k)$;

Theoretical Analysis — Preliminaries

ξ_k may represent a single sample or a mini-batch
g may represent a stochastic gradient (biased estimator of $\nabla F(\boldsymbol{w}_k)$ or a stochastic Netwon or quasi-Newton direction).

$$g(\boldsymbol{w}_k, \xi_k) = \begin{cases} \nabla f(\boldsymbol{w}_k; \xi_k) \\ \frac{1}{n_k} \sum_{i=1}^{n_k} \nabla f(\boldsymbol{w}_k; \xi_{k,i}) \\ H_k \frac{1}{n_k} \sum_{i=1}^{n_k} \nabla f(\boldsymbol{w}_k; \xi_{k,i}) \end{cases}$$

H_k a symmetric positive definite scaling matrix (we called it B_k^{-1} earlier in this course)
α_k fixed stepsize or diminishing stepsizes
$\boldsymbol{w}_k$ can have influence on the sample selection (active learning)

Convergence Analysis – Assumptions

- Assumption 4.1 Lipschitz-continuous objective gradients:
  $\|\nabla F(\boldsymbol{w}) - \nabla F(\bar{\boldsymbol{w}})\|_2 \leq L\|\boldsymbol{w} - \bar{\boldsymbol{w}}\|_2$ for all $\{\boldsymbol{w}, \bar{\boldsymbol{w}}\} \subseteq \mathbb{R}^d$
- Assumption 4.3 First and second moment limits. The objective function and Procedure SG satisfy the following:
  - objective function is bounded from below by a scalar F_{inf} over the region explored by the algorithm.
  - in expectation, the vector $-g(\boldsymbol{w}_k, \xi_k)$ is a direction of sufficient descent for F from $\boldsymbol{w}_k$ with a norm comparable to the norm of the gradient
  - the variance of $g(\boldsymbol{w}_k, \xi_k)$ is restricted, but in a relatively minor manner.

$$\mathrm{Var}_{\xi_k}[g(\boldsymbol{w}_k, \xi_k)] \leq M + M_V \, \|\nabla F(\boldsymbol{w}_k)\|_2^2 \,, \; M > 0, M_V > 0 \text{ for all } k \in \mathbb{N}$$

- Lemma: SG is a Markovian process in the sense that $\boldsymbol{w}_{k+1}$ is a random variable that depends only on the iterate $\boldsymbol{w}_k$, the seed ξ_k, and the stepsize α_k and not on any past iterates.

Convergence Analysis – Assumptions

- Assumption 4.5 Strong convexity.
  The objective function $F : \mathbb{R}^d \to \mathbb{R}$ is **strongly convex** in that there exists a constant $c > 0$ such that

  $$F(\bar{\boldsymbol{w}}) \geq F(\boldsymbol{w}) + \nabla F(\boldsymbol{w})^T(\bar{\boldsymbol{w}} - \boldsymbol{w}) + \frac{1}{2}c\,\|\bar{\boldsymbol{w}} - \boldsymbol{w}\|_2^2 \qquad \text{for all } (\bar{\boldsymbol{w}}, \boldsymbol{w}) \in \mathbb{R}^d \times \mathbb{R}^d$$

  or equivalently if there exists $c > 0$:

  $$\nabla^2 F(\boldsymbol{w}) \succeq c$$

  (for univariate case: $f''(w) \geq c$), ie, grows at least quadratically.

  Hence, F has a unique minimizer, denoted as $\boldsymbol{w}^* \in \mathbb{R}^d$ with $F^* \stackrel{def}{=} F(w^*)$.

Convergence Analysis – Results

- Theorem 4.6 (Strongly Convex Objective, Fixed Stepsize).
  - optimality gap has an asymptotic bound in expectation depending on $\bar{\alpha}$.
  - sublinear convergence rate of the suboptimality gap
- Theorem 4.7 (Strongly Convex Objective, Diminishing Stepsizes)
  SG with diminishing step size converges in expectation.
  - role of strong convexity
  - role of initial point
  - trade-offs of mini batches
- Theorem 4.8 (Nonconvex Objective, Fixed Stepsize)
  - While one cannot bound the expected optimality gap as in the convex case, inequality (4.28b) bounds the average norm of the gradient of the objective function observed on $\{\boldsymbol{w}_k\}$ visited during the first K iterations.
  - classical result for the full gradient method applied to nonconvex functions, namely, that the sum of squared gradients remains finite, implying that

$$\{\|\nabla F(\boldsymbol{w}_k)\|_2\} \to 0.$$

- Theorem 4.9 (Nonconvex Objective, Diminishing Stepsizes)
  - for the SG method with diminishing stepsizes, the expected gradient norms cannot stay bounded away from zero
  - the weighted average norm of the squared gradients converges to zero even if the gradients are noisy, (i.e., if $M > 0$ in the Variance upper bounding assumption) one can still conclude that the expected gradient norms cannot asymptotically stay far from zero.

Computational Complexity Analysis

- consider a big data scenario with an infinite supply of training examples, but a **limited computational time budget**. what type of algorithm — e.g., a simple SG or batch gradient method — would provide the best guarantees in terms of achieving a low expected risk?

- $\boldsymbol{w}^* \in \operatorname{argmin} R(\boldsymbol{w})$; $\boldsymbol{w}_n \in \operatorname{argmin} R_n(\boldsymbol{w})$, $\tilde{\boldsymbol{w}}_n$ approximate empirical risk minimizer returned by a given optimization algorithm at $\mathcal{T}_{max}$

- The tradeoffs associated with this scenario can be formalized as choosing the family of prediction functions $\mathcal{H}$, the number of examples n, and the optimization accuracy $\epsilon \stackrel{def}{=} E[R_n(\tilde{\boldsymbol{w}}_n) - R_n(\boldsymbol{w}_n)]$ in order to minimize the total error:

$$\underset{\mathcal{H}, n \in \mathbb{N}, \epsilon}{\text{minimize}}\ E[R(\tilde{\boldsymbol{w}}_n)] = \overbrace{R(\boldsymbol{w}^*)}^{\mathcal{E}_{app}(\mathcal{H})} + \overbrace{E[R(\boldsymbol{w}_n) - R(\boldsymbol{w}^*)]}^{\mathcal{E}_{est}(\mathcal{H}, n)} + \overbrace{E[R(\tilde{\boldsymbol{w}}_n) - R(\boldsymbol{w}_n)]}^{\mathcal{E}_{opt}(\mathcal{H}, n, \epsilon)}$$

$$\text{subject to } \mathcal{T}(n, \epsilon) \leq \mathcal{T}_{max}$$

Computational Complexity Analysis

- SG, with its sublinear rate of convergence, is more efficient for large-scale learning than (full, batch) gradient-based methods that have a linear rate of convergence.

- reducing the optimization error $\mathcal{E}_{opt}(\mathcal{H}, n, \epsilon)$ (evaluated with respect to R rather than R_n) one might need to make up for the additional computing time by: (i) reducing the sample size n, potentially increasing the estimation error $\mathcal{E}_{est}(\mathcal{H}, n)$; or (ii) simplifying the function family $\mathcal{H}$, potentially increasing the approximation error $\mathcal{E}_{app}(\mathcal{H})$.

Computational Complexity Analysis

Instead, keep fixed $\mathcal{H}$ carrying out a worst-case analysis on the influence of the sample size n and optimization tolerance ϵ, which together only influence the estimation and optimization errors.

- this leads to an upper bound:

$$\mathcal{E}(n, \epsilon) = \mathcal{O}\left(\frac{\sqrt{\log n}}{n} + \epsilon\right)$$

- when the loss function is strongly convex

$$\mathcal{E}(n, \epsilon) = \mathcal{O}\left(\frac{\log n}{n} + \epsilon\right) \qquad \text{and when } n \text{ large to} \quad \mathcal{E}(n, \epsilon) \sim \frac{1}{n} + \epsilon$$

Computational Complexity Analysis

Recall:

- the total work required to obtain ϵ-optimality for SG is $\mathcal{T}_{stoch} \sim 1/\epsilon$,
- the total work required to obtain ϵ-optimality for a batch gradient method is $\mathcal{T}_{batch} \sim n \log(1/\epsilon)$.

	Batch	Stochastic
$\mathcal{T}(n,\epsilon)$	$\sim n \log\left(\frac{1}{\epsilon}\right)$	$\frac{1}{\epsilon}$
$\mathcal{E}^*$	$\sim \frac{\log(\mathcal{T}_{\max})}{\mathcal{T}_{\max}} + \frac{1}{\mathcal{T}_{\max}}$	$\frac{1}{\mathcal{T}_{\max}}$

A stochastic optimization algorithm performs better that batch stochastic in terms of expected error

Large gap between asymptotical behavior and practical realities.

Remarks

- Fragility of the Asymptotic Performance of SG
  ok if objective function includes a squared L_2-norm regularizer (related to constant c) but regularization parameter should be lowered when the number of samples increases.

- SG good for GPUs but ill-conditioning erodes efficiency of SG

- Distributed computing not working with basic SG because of too frequent updates of $\boldsymbol{w}$, more promising with mini-batch.

- Alternatives with Faster Convergence: minimizing empirical risk R_n there is information from previous gradients.
  - gradient aggregation methods
  - dynamic sampling approach

Thery vs Practice

- A lot of work and results on convex functions (strongly and Lipschitz smooth) but not much on non-convex functions, which are more common in ML applications.
- Note however that locally, local optima are in convex parts of the landscape. Hence, theoretical analysis and results are important. Ideally we would ike to have algorithms that work well for non-convex functions, but also that they can switch to a more efficient behavior when the landscape is locally convex.
- we want that our algorithms work well at least in convex regions.
- problem because of noisy and local information of the gradients.

13. Constrained Optimization

Constrained Optimization

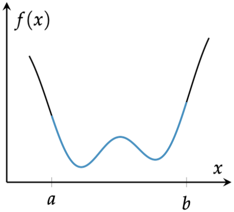
- Minimizing an objective subject to design point restrictions called **constraints**
- A variety of techniques transform constrained optimization problems into unconstrained problems
- New optimization problem statement

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{x} \in \mathcal{X} \end{aligned}$$

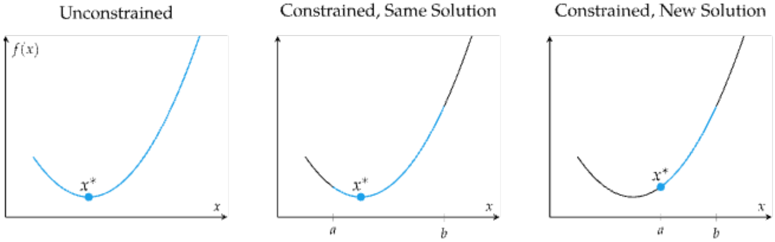
- The set $\mathcal{X} \subset \mathbb{R}$ is called the **feasible set**

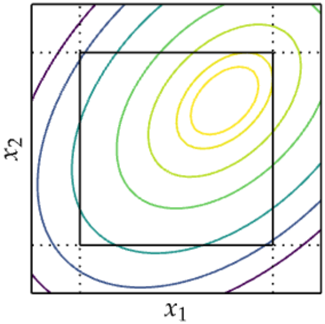
Constrained Optimization

Constraints that bound feasible set can change the optimizer



$$\underset{x}{\text{minimize}}\ f(x)$$
$$\text{subject to } x \in [a, b]$$





Constraint Types

- Generally, constraints are formulated using two types:
    1. **Equality constraints**: $h(\boldsymbol{x}) = 0$
    2. **Inequality constraints**: $g(\boldsymbol{x}) \le 0$

- Any optimization problem can be written as

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & g_i(\boldsymbol{x}) \le 0 \text{ for all } i \text{ in } \{1, \dots, m\} \\ & h_j(\boldsymbol{x}) = 0 \text{ for all } j \text{ in } \{1, \dots, \ell\} \end{aligned}$$

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{g}(\boldsymbol{x}) \le \mathbf{0} \\ & \boldsymbol{h}(\boldsymbol{x}) = \mathbf{0} \end{aligned}$$

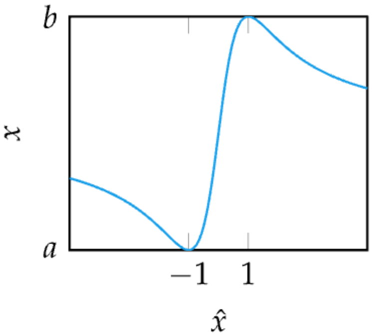
f and the functions h and g are all smooth, real-valued functions on a subset of $\mathbb{R}^n$

Transformations to Remove Constraints

- If necessary, some problems can be reformulated to incorporate constraints into the objective function

- If x is constrained between a and b

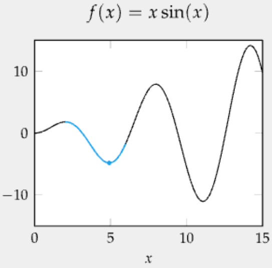
$$x = t_{a,b}(\hat{x}) = \frac{b+a}{2} + \frac{b-a}{2}\left(\frac{2\hat{x}}{1+\hat{x}^2}\right)$$

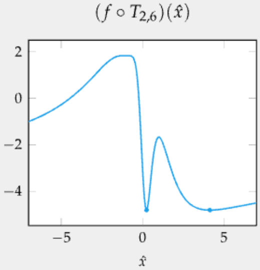


Transformations to Remove Constraints

Example

$$\underset{x}{\text{minimize}} \;\; x\sin(x)$$
$$\text{subject to} \;\; 2 \le x \le 6$$





$$\underset{\hat{x}}{\text{minimize}} \; t_{2,6}(\hat{x})\sin(t_{2,6}(\hat{x}))$$

$$\underset{\hat{x}}{\text{minimize}} \;\; \left(4 + 2\left(\frac{2\hat{x}}{1+\hat{x}^2}\right)\right)\sin\left(4 + 2\left(\frac{2\hat{x}}{1+\hat{x}^2}\right)\right)$$

Transformations to Remove Constraints

Example

$$\begin{aligned} \underset{x}{\text{minimize}} \;\; & f(\boldsymbol{x}) \\ \text{subject to} \;\; & h(\boldsymbol{x}) = x_1^2 + x_2^2 + \ldots + x_n^2 - 1 = 0 \end{aligned}$$

- Solve for one of the variables to eliminate constraint:

$$x_n = \pm\sqrt{1 - x_1^2 - x_2^2 - \ldots - x_{n-1}^2}$$

- Transformed, unconstrained optimization problem:

$$\underset{x}{\text{minimize}} \;\; \left(\left[x_1, x_2, \ldots, x_{n-1}, \pm\sqrt{1 - x_1^2 - x_2^2 - \ldots - x_{n-1}^2}\right]\right)$$

Lagrangian Multiplier Method

- With only equality constraints, critical points (local minima, global minima, or saddle points optimal) where gradient of f and the gradient of h are aligned

- The method of Lagrangian Multipliers is used to optimize a function subject to (equality) constraints

- Lagrangian multipliers refer to the variables introduced by the method denoted by λ

$$\underset{\boldsymbol{x}}{\text{minimize}} \; f(\boldsymbol{x})$$
$$\text{subject to} \; h(\boldsymbol{x}) = 0$$

1. Form **Lagrangian function**

$$\mathcal{L}(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) - \lambda h(\boldsymbol{x})$$

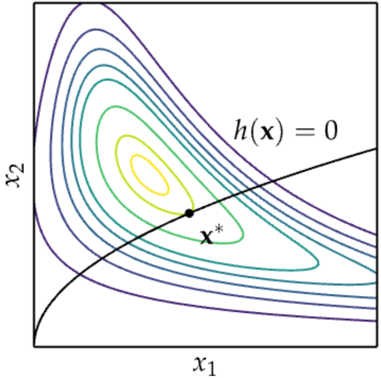
2. Set $\nabla_{\boldsymbol{x}}\mathcal{L}(\boldsymbol{x}, \lambda) = 0$ and $\nabla_{\lambda}\mathcal{L}(\boldsymbol{x}, \lambda) = 0$ to get

$$\nabla f(\boldsymbol{x}) = \lambda \nabla h(\boldsymbol{x}) \qquad h(\boldsymbol{x}) = 0$$

3. solve for $\boldsymbol{x}$ and λ

Lagrangian Multiplier Method

Intuitively, the method of Lagrange multipliers finds the point x^* where the constraint function is orthogonal to the gradient



Lagrangian Multiplier Method with Inequality Constraints

$$
\begin{aligned}
\underset{\boldsymbol{x}}{\text{minimize}} \;\; & f(\boldsymbol{x}) \\
\text{subject to} \;\; & g(\boldsymbol{x}) \leq 0
\end{aligned}
$$

- If solution lies at the constraint boundary, the constraint is called **active**, and the Lagrangian condition holds for a non-negative constant μ:

$$\nabla f(\boldsymbol{x}) + \mu \nabla g(\boldsymbol{x}) = 0$$

- If the solution lies within the boundary, the constraint is called **inactive**, and the optimal solution simply lies where

$$\nabla f(\boldsymbol{x}) = 0$$

that is, the Lagrangian condition holds with $\mu = 0$

Lagrangian Multiplier Method with Inequality Constraints

$$\underset{\boldsymbol{x}}{\text{minimize}} \; f(\boldsymbol{x})$$
$$\text{subject to } \; g(\boldsymbol{x}) \leq 0$$

- We create the Lagrangian function such that it goes to ∞ outside the feasibility set ($g(\boldsymbol{x}) \not\leq 0$):

$$\mathcal{L}_\infty(\boldsymbol{x}) = f(\boldsymbol{x}) + \infty(g(\boldsymbol{x}) > 0)$$

impractical: discontinuous and nondifferentiable.

- Instead, for $\mu \geq 0$:

$$\mathcal{L}(\boldsymbol{x}, \mu \geq 0) = f(\boldsymbol{x}) + \mu g(\boldsymbol{x})$$

$$\mathcal{L}_\infty(\boldsymbol{x}) = \underset{\mu \geq 0}{\text{maximize}} \; \mathcal{L}(\boldsymbol{x}, \mu)$$

for $\boldsymbol{x}$ infeasible, $\mathcal{L}_\infty(\boldsymbol{x}) = \infty$; for $\boldsymbol{x}$ feasible, $\mathcal{L}_\infty(\boldsymbol{x}) = f(\boldsymbol{x})$

- The new optimization problem becomes

$$\boxed{\underset{\boldsymbol{x}}{\text{minimize}} \; \underset{\mu \geq 0}{\text{maximize}} \; \mathcal{L}(\boldsymbol{x}, \mu)}$$

This is called the **primal problem**

Lagrangian Multiplier Method – General Case

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \;& f(\boldsymbol{x}) \\ \text{subject to} \;& \boldsymbol{g}(\boldsymbol{x}) \leq \boldsymbol{0} \\ & \boldsymbol{h}(\boldsymbol{x}) = \boldsymbol{0} \end{aligned}$$

- Generalized **Lagrangian Function** (with both equality and inequality constraints):

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\boldsymbol{x}) + \sum_i \mu_i g_i(\boldsymbol{x}) + \sum_j \lambda_j h_j(\boldsymbol{x})$$

(note we changed the sign of the λ term since λ is anyway free to be positive or negative)

- the **primal form** of the optimization problem

$$\underset{\boldsymbol{x}}{\text{minimize}} \underset{\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda}}{\text{maximize}} \; \mathcal{L}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{\lambda})$$

Necessary Conditions – KKT Conditions

$$
\begin{aligned}
\underset{\boldsymbol{x}}{\text{minimize}} \quad & f(\boldsymbol{x}) \\
\text{subject to} \quad & \boldsymbol{g}(\boldsymbol{x}) \leq \boldsymbol{0} \\
& \boldsymbol{h}(\boldsymbol{x}) = \boldsymbol{0}
\end{aligned}
$$

Any critical point $\boldsymbol{x}^*$ must satisfy the **Karush-Kuhn-Tucker conditions**

1. primal feasibility: $\boldsymbol{g}(\boldsymbol{x}^*) \leq 0$ and $\boldsymbol{h}(\boldsymbol{x}^*) = 0$
2. dual feasibility: penalization is towards feasibility $\boldsymbol{\mu} \geq 0$
3. complementary slackness: either μ_i or $g_i(\boldsymbol{x}^*)$ is zero.

   $$\boldsymbol{\mu} \cdot \boldsymbol{g}(\boldsymbol{x}) = 0 \implies \mu_i g_i(\boldsymbol{x}^*) = 0, \text{ for } i = 1, \ldots, m.$$

4. stationarity: objective function tangent to each active constr.

   $$\nabla f(\boldsymbol{x}^*) + \sum_i \mu_i \nabla g_i(\boldsymbol{x}^*) + \sum_j \lambda_j \nabla h_j(\boldsymbol{x}^*) = 0$$

These are the First Order Necessary Conditions (FONC) for Constrained Optimization

Necessary Conditions – KKT Conditions

In vector form:

$$\begin{cases} \nabla f(\boldsymbol{x}^*) + \boldsymbol{\mu} \cdot \nabla \boldsymbol{g}(\boldsymbol{x}^*) + \boldsymbol{\lambda} \cdot \nabla \boldsymbol{h}(\boldsymbol{x}^*) = \boldsymbol{0} \\ \boldsymbol{\mu} \cdot \boldsymbol{g}(\boldsymbol{x}^*) = 0 \\ \boldsymbol{g}(\boldsymbol{x}^*) \leq \boldsymbol{0}, \;\; \boldsymbol{h}(\boldsymbol{x}^*) = \boldsymbol{0} \\ \boldsymbol{\mu} \geq \boldsymbol{0} \end{cases}$$

Particular cases:

- f concave, $\boldsymbol{g}$ convex: then KKT are also sufficient
- $\boldsymbol{x}^*$ interior point
- Patological cases where they do not hold (formal expressions of the conditions include assumptions to avoid these cases).

Duality

- Generalized Lagrangian Function:

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\boldsymbol{x}) + \sum_i \mu_i g_i(\boldsymbol{x}) + \sum_j \lambda_j h_j(\boldsymbol{x})$$

- the **primal form** of the optimization problem

$$\underset{\boldsymbol{x}}{\text{minimize}}\ \underset{\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda}}{\text{maximize}}\ \mathcal{L}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{\lambda})$$

- Reversing the order of operations leads to the **dual form**

$$\underset{\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda}}{\text{maximize}}\ \underset{\boldsymbol{x}}{\text{minimize}}\ \mathcal{L}(\boldsymbol{x}, \boldsymbol{\mu}, \boldsymbol{\lambda})$$

- What is the relationship between the two? (See next slide)
- In some cases, the dual problem is easier to solve computationally than the original problem.
  In other cases, the dual can be used to obtain easily to gain information on the optimal value of the objective for the primal problem.
  The dual has also been used to design algorithms for solving the primal problem.

Min-Max Inequality

Theorem (Max-min inequality)

For any function $f : Z \times W \to \mathbb{R}$,

$$\sup_{z \in Z} \inf_{w \in W} f(z, w) \leq \inf_{w \in W} \sup_{z \in Z} f(z, w) \ .$$

<u>Proof</u>

For all $z \in Z$, $\inf_{w \in W} f(z, w) \leq f(z, w)$ for all $w \in W$ by definition of infimum being a lower bound.

For all $w \in W$, $f(z, w) \leq \sup_{z \in Z} f(z, w)$ for all $z \in Z$ by definition of supremum being an upper bound.

Let $g(z) \triangleq \inf_{w \in W} f(z, w)$. Let $h(w) \triangleq \sup_{z \in Z} f(z, w)$. Then, for all $z \in Z$ and $w \in W$, $g(z) \leq h(w)$

$\sup_{z \in Z} g(z) \leq h(w)$ holds for all $w \in W$ because the supremum is the least upper bound.

$\sup_{z \in Z} g(z) \leq \inf_{w \in W} h(w)$ because the infimum is the greatest lower bound

$\square$

Min-Max Inequality and Duality

- When f, W, and Z are convex the inequality becomes equality and we have a **strong max-min property** (or a **saddle-point property**).
- For us:

$$\underset{\boldsymbol{\mu}\geq 0,\boldsymbol{\lambda}}{\text{maximize}}\ \underset{\boldsymbol{x}}{\text{minimize}}\ \mathcal{L}(\boldsymbol{x},\boldsymbol{\mu},\boldsymbol{\lambda}) \leq \underset{\boldsymbol{x}}{\text{minimize}}\ \underset{\boldsymbol{\mu}\geq 0,\boldsymbol{\lambda}}{\text{maximize}}\ \mathcal{L}(\boldsymbol{x},\boldsymbol{\mu}.\boldsymbol{\lambda})$$

- Therefore, the solution to the dual problem d^* is a **lower bound** to the primal solution p^*
- The inner part of the dual problem can be used to define the **dual function** or **dual objective**

$$\mathcal{D}(\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda}) = \underset{\boldsymbol{x}}{\text{minimize}}\ \mathcal{L}(\boldsymbol{x},\boldsymbol{\mu},\boldsymbol{\lambda})$$

Duality

- The dual function is concave. Gradient ascent on a concave function always converges to the global maximum.

- **Dual Problem**: $\max \mathcal{D}(\boldsymbol{\mu} \geq 0, \boldsymbol{\lambda})$

- Optimizing the dual problem is easy whenever minimizing the Lagrangian with respect to $\boldsymbol{x}$ is easy.

- For any $\boldsymbol{\mu} \geq \mathbf{0}$ and any $\boldsymbol{\lambda}$, we have

$$\mathcal{D}(\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}) \leq p^*$$

- The difference between dual and primal solutions d^* and p^* is called the **duality gap**

- Showing zero-duality gap is a "certificate" of optimality

Penalty methods

- A way to optimize the Lagrangiann function is to alternate between optimizing with respect to $\boldsymbol{x}$ and $\boldsymbol{\mu}$ and $\boldsymbol{\lambda}$ (called **primal-dual methods**)
- Penalty methods are a **heuristic** way of reformulating a constrained optimization problem as an unconstrained problem by penalizing the objective function value when constraints are violated

Example

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{g}(\boldsymbol{x}) \leq \mathbf{0} \\ & \boldsymbol{h}(\boldsymbol{x}) = \mathbf{0} \end{aligned}$$

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & f(\boldsymbol{x}) + \rho \cdot p_{count}(\boldsymbol{x}) \\ \text{s.t.} \quad & p_{count}(\boldsymbol{x}) = \sum_i (g_i(\boldsymbol{x}) > 0) + \sum_j (h_j(\boldsymbol{x}) \neq 0) \end{aligned}$$

Penalty Methods

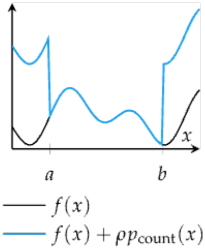
Penalty methods optimize on the $\boldsymbol{x}$ space only, but the penalty term is designed (heuristically) to push the solution towards the feasible set.

Procedure penalty_method;
Input: $f, p, \boldsymbol{x}, k_{max}; \rho = 1, \gamma = 2$
Output: $\boldsymbol{x}$ solution
for k in $1, \ldots, k_{max}$ **do**
    $\boldsymbol{x} \leftarrow \text{minimize}_{\boldsymbol{x}}\{f(\boldsymbol{x}) + \rho \cdot p(\boldsymbol{x})\}$;
    $\rho \leftarrow \rho \cdot \gamma$;
    if $p(\boldsymbol{x}) = 0$ **then**
        return $\boldsymbol{x}$;
return $\boldsymbol{x}$;

Penalty methods

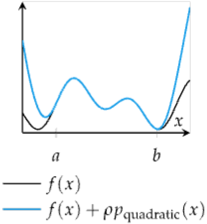
- Count penalty:

$$p_{count}(\boldsymbol{x}) = \sum_i (g_i(\boldsymbol{x}) > 0) + \sum_j (h_j(\boldsymbol{x}) \neq 0)$$



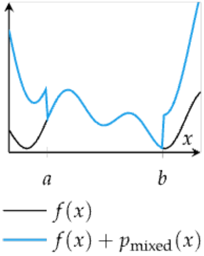
- Quadratic penalty:

$$p_{quadratic}(\boldsymbol{x}) = \sum_i \max(g_i(\boldsymbol{x}), 0)^2 + \sum_j h_j(\boldsymbol{x})^2$$



- Mixed Penalty:

$$p_{mixed}(\boldsymbol{x}) = \rho_1 p_{count}(\boldsymbol{x}) + \rho_2 p_{quadratic}(\boldsymbol{x})$$



Augmented Lagrange Method

- Adaptation of penalty method for equality constraints

$$p_{Lagrangian}(\boldsymbol{x}) \stackrel{def}{=} \frac{1}{2}\rho \sum_i (h_i(\boldsymbol{x}))^2 - \sum_i \lambda_i h_i(\boldsymbol{x})$$

Procedure augmented_lagrange_method;
Input: $f, h, \boldsymbol{x}, k_{max}; \rho = 1, \gamma = 2)$
$\boldsymbol{\lambda} \leftarrow \boldsymbol{0}$;
for k in $1, \ldots, k_{max}$ **do**
  $p \leftarrow (x \mapsto \rho/2 \cdot \sum_i (h_i(\boldsymbol{x})^2) - \boldsymbol{\lambda} \cdot h(\boldsymbol{x}))$;
  $\boldsymbol{x} \leftarrow \text{minimize}_{\boldsymbol{x}}\{f(\boldsymbol{x}) + p(\boldsymbol{x})\}$;
  $\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} - \rho \cdot \boldsymbol{h}(\boldsymbol{x})$;
  $\rho \leftarrow \rho \cdot \gamma$;
return $\boldsymbol{x}$;

- $\boldsymbol{\lambda}$ converges towards the Lagrangian multiplier

Interior Point Methods

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{g}(\boldsymbol{x}) \leq \boldsymbol{0} \end{aligned}$$

- Also called **barrier methods**, interior point methods ensure that each step is feasible
- This allows premature termination to return a nearly optimal, feasible point
- Barrier functions are implemented similar to penalties but must meet the following conditions:
    1. Continuous
    2. Non-negative in the feasible region, i.e., $p_{barrier}(\boldsymbol{x}) \geq 0$ for all $\boldsymbol{x}$ such that $g_i(\boldsymbol{x}) \leq 0$ for all i
    3. Approach infinity as $\boldsymbol{x}$ approaches boundary

Interior Point Methods – Barrier Functions

- Inverse Barrier:

$$p_{barrier}(\boldsymbol{x}) = -\sum_i \frac{1}{g_i(\boldsymbol{x})}$$

- Standard Log Barrier:

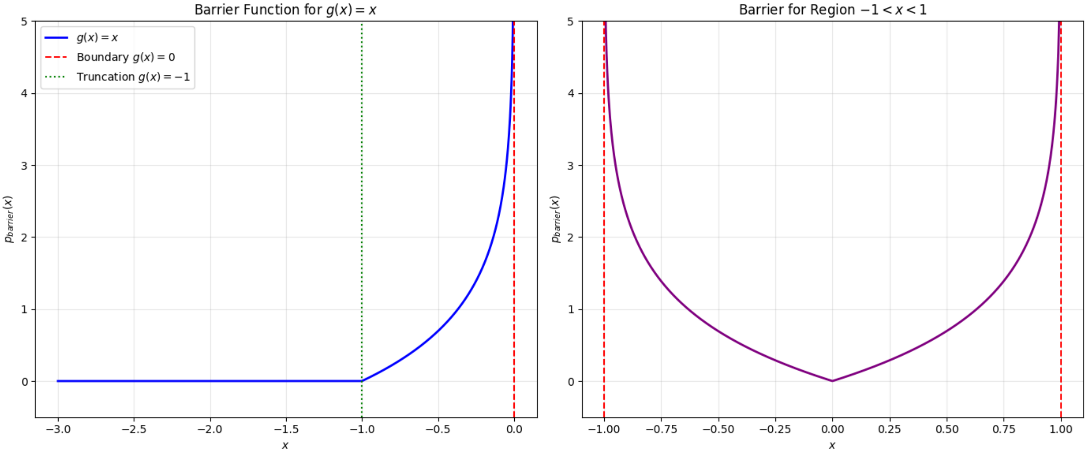
$$p_{barrier}(\boldsymbol{x}) = -\sum_i \log(-g_i(\boldsymbol{x}))$$

- Truncated Log Barrier:

$$p_{barrier}(\boldsymbol{x}) = -\sum_i \begin{cases} \log(-g_i(\boldsymbol{x})) & \text{if } g_i(\boldsymbol{x}) \geq -1 \\ 0 & \text{otherwise} \end{cases}$$

  - Feasible Region: The function is primarily defined for $g_i(\boldsymbol{x}) < 0$. As $g_i(\boldsymbol{x}) \to 0$ (approaching the boundary of the constraint), the value $-\log(-g_i(\boldsymbol{x}))$ approaches $+\infty$, creating a penalty "wall"
  - Truncation at -1: The barrier only exerts influence when the variable is relatively close to the boundary (specifically within a distance where $g_i(\boldsymbol{x})$ is between -1 and 0).
  - Continuity: At $g_i(\boldsymbol{x}) = -1$, the function is continuous because $\log(-(-1)) = \log(1) = 0$.

Interior Point Methods – Truncated Log Barrier

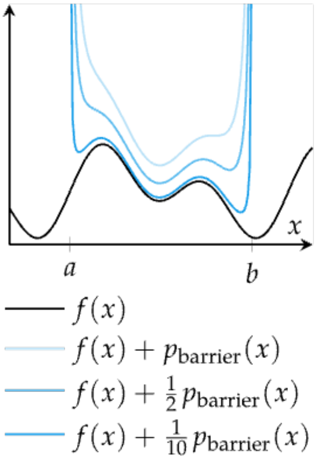


Left: a single constraint $g(x) = x$. Right: the summation of two constraints $g_1(x) = x - 1$ and $g_2(x) = -x - 1$

Interior Point Methods – Optimization Problem

New optimization problem:

$$\underset{\boldsymbol{x}}{\text{minimize}}\ f(\boldsymbol{x}) + \frac{1}{\rho} p_{barrier}(\boldsymbol{x})$$



Interior Point Methods

Procedure interior_point_method;
Input: $f, p, \boldsymbol{x}; \rho = 1, \gamma = 2, \epsilon = 0.001$
$\Delta \leftarrow \infty$;
while $\Delta > \epsilon$ **do**
    $\boldsymbol{x}' \leftarrow \text{minimize}_{\boldsymbol{x}}\{f(\boldsymbol{x}) + p(\boldsymbol{x})/\rho\}$;
    $\Delta \leftarrow \|\boldsymbol{x}' - \boldsymbol{x}\|$;
    $\boldsymbol{x} \leftarrow \boldsymbol{x}'$;
    $\rho \leftarrow \rho \cdot \gamma$;
return $\boldsymbol{x}$;

- Line searches $f(\boldsymbol{x} + \alpha\boldsymbol{d})$ are constrained to the interval $\alpha = [0, \alpha_u]$, where α_u is the step to the nearest boundary.
  In practice, α_u is chosen such that $\boldsymbol{x} + \alpha\boldsymbol{d}$ is just inside the boundary to avoid the boundary singularity.
- Needs an initial **feasible** solutions. Typically, found by solving:

$$\underset{\boldsymbol{x}}{\text{minimize}}\ p_{quadratic}(\boldsymbol{x})$$

Summary

- Constraints are requirements on the design points that a solution must satisfy
- Some constraints can be transformed or substituted into the problem to result in an unconstrained optimization problem
- Analytical methods using Lagrange multipliers yield the generalized Lagrangian and the necessary conditions for optimality under constraints
- A constrained optimization problem has a dual problem formulation that is easier to solve and whose solution is a lower bound of the solution to the original problem
- Penalty methods penalize infeasible solutions and often provide gradient information to the optimizer to guide infeasible points toward feasibility
- Interior point methods maintain feasibility but use barrier functions to avoid leaving the feasible set